IACR Communications in Cryptology
ISSN 3006-5496, Vol. 3, No. 1, 57 pages.

https://doi.org/10.62056/a6n56chdj

Check for updates

# Diving Deep Into UC: Uncovering and Resolving Issues in Universal Composability

Céline Chevalier[1,3,4,5,6] and Éric Sageloli[2,3,4,5,6]

[1] CRED, Paris-Panthéon-Assas University, Paris, France
[2] Thales, France
[3] DIENS, École normale supérieure, Paris, France
[4] CNRS, France
[5] PSL University, France
[6] Inria, France

**Abstract.** Introduced by Canetti in 2001, Universal Composability (UC) is a widely adopted security model that enables the specification and proof of security for a broad range of protocols, offering strong security guarantees. At its core lies the universal composition theorem (UC theorem), which ensures that protocols proven secure within the framework remain secure even when deployed in real-world environments with multiple instances of them.

In this work, we present two key contributions. First, we identify several problems with the UC framework, in particular the UC Theorem. They include counterexamples, limitations that make it unusable for important classes of protocols, and weaknesses in its proof. These problems reveal flaws in nearly all the fundamental concepts of UC.

Second, we propose a revised formulation of the main concepts of UC to address these issues. Although the resulting modifications are nontrivial, our updated definitions are designed to remain as faithful as possible to the structure and intent of the original model.

**Keywords:** Universal Composability · Composable Security · Protocol Security · Theoretical Foundations

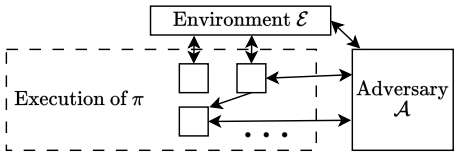
## 1 Introduction

Introduced in 2001 by Canetti in [Can01], Universal Composability (UC) is a security framework renowned for its ability to define and prove the security of a wide range of protocols, as well as for the strong security guarantees it provides. A protocol proven secure within this framework remains secure even when deployed in real-world environments, including in multiple concurrent instances. To illustrate the variety of protocols recently studied under UC, we can cite PAKEs [JKX18, Hes20, CNPR22, BGHJ24], non-interactive zero-knowledge arguments [CDG+25], oblivious transfer [BC15], the WhatsApp backup protocol [DFG+23], the Signal protocol [CJSV22] and the Bitcoin protocol [BMTZ24].

In UC, it is considered that if a protocol contains a vulnerability, this vulnerability should have an impact on some environment of execution, attacked by some adversary. Consequently, when studying a protocol $\pi$, we consider its execution alongside two machines: one modeling an adversary and the other modeling the execution environment.

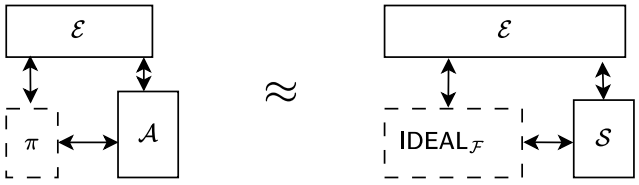
E-mail: eric.sageloli@protonmail.com (Éric Sageloli)

This work is licensed under a "CC BY 4.0" license.
Received: 2025-10-06 Accepted: 2025-12-02




To define the security of a protocol, such as a PAKE [JKX18, Hes20, CNPR22, BGHJ24], or an oblivious transfer [BC15], we must first define a special machine $\mathcal{F}$, known as an ideal functionality. This ideal functionality is then used to define the protocol $\mathsf{IDEAL}_{\mathcal{F}}$, referred to as the ideal protocol associated with $\mathcal{F}$, which represents a idealized version of the protocol (*e.g.*, a ideal PAKE or oblivious transfert). The references listed at the beginning of this paragraph provide examples of ideal functionalities.

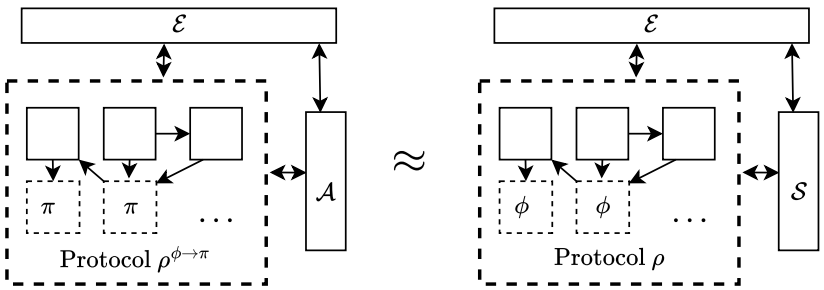
Then, the security of a protocol $\pi$ is proven by showing that $\pi$ is "as secure as" $\mathsf{IDEAL}_{\mathcal{F}}$ through a notion called UC-emulation. $\pi$ is said to UC-emulate $\mathsf{IDEAL}_{\mathcal{F}}$ if, for any adversary $\mathcal{A}$ of $\pi$, there is an adversary $\mathcal{S}$ of $\mathsf{IDEAL}_{\mathcal{F}}$ such that no environment $\mathcal{E}$ can efficiently distinguish if it is interacting with $\pi$ and $\mathcal{A}$ or $\mathsf{IDEAL}_{\mathcal{F}}$ and $\mathcal{S}$.




While the aim of $\mathcal{E}$ is to model any execution environment, such environments are typically not composed of a single machine, nor do they involve only one instance of the protocol. Instead, they can be viewed as protocols $\rho$ with multiple machines, each using multiple instances of the studied protocol.

To justify modeling the environment as a single machine that can only use one instance of the protocol $\pi$, UC must ensure that this modeling is sufficient to cover situations involving complex execution environments. This is the goal of the main theorem of the framework, called the universal composition theorem [Can20, (Theorem 22)] (UC theorem).

The UC Theorem states that for protocols $\phi$, $\pi$, and $\rho$, under certain hypotheses, if $\pi$ UC-emulates $\phi$, then a modification of $\rho$, denoted by $\rho^{\phi \to \pi}$, where uses of $\phi$ are replaced by uses of $\pi$, will also UC-emulate $\rho$.




Taking $\phi = \mathsf{IDEAL}_{\mathcal{F}}$, this implies that if $\pi$ is secure within a simple execution environment – *i.e.*, if $\pi$ UC-emulates $\mathsf{IDEAL}_{\mathcal{F}}$ – it remains secure when used in complex execution environments.

**Table 1:** Issues, Limitations and Weaknesses described in the main body.

| Notions | Issues | Limitations | Weaknesses | |
|---|---|---|---|---|
| Extended Session and Subroutine Respecting | (I.1) (I.2) (I.3) | (L.1) | | (Section 5.1) |
| UC Operator | (I.4) | | (W.1) | (Section 5.2) |
| Subroutine Exposing | | (L.2) | (W.2) | (Section 5.3) |
| Compliance | | | (W.3) (W.4) | (Section 5.4) |

## 1.1 Our Contributions

As a first and main contribution, we identify several problems with the UC theorem [Can20, (Theorem 22)], classified as follows:

- *Issues (I):* problems that lead to counterexamples of the UC theorem.
- *Limitations (L):* difficulties that limit the applicability of the UC theorem.
- *Weaknesses (W):* gaps in the proof of the UC theorem.

These problems affect all security concepts associated with the UC Theorem, with multiple concerns identified for each. Our focus will be on these concepts within the initial and most detailed formulation of the UC framework that was last updated in 2020 [Can20], which we refer to as "Plain UC ". However, these problems naturally extend to its generalizations, which encompass it. These include UC with joint state [CR03], UC with global setup [CDPW07], and multi-protocol UC [CDT19].

Since the UC Theorem ensures that protocols proven UC-secure maintain their security during composition, our findings reveal that the current UC framework does not guarantee the preservation of security under composition – the very goal of UC.

To re-establish these security guarantees, in a second contribution, we propose an update to each security concept within Plain UC. These revised definitions address the identified issues. To support these updates, we also present an updated UC Theorem.

These updates restrict the applicability of the UC Theorem in certain respects – eliminating some of the issues identified earlier – while simultaneously extending it to protocols that were previously incompatible with the framework.

Finally, although case-by-case verification remains necessary, we believe that protocols proven secure under Plain UC generally satisfy the hypotheses of the UC Theorem in Updated UC.

Moreover, we believe that these updates offer a solid foundation for extending the main generalizations of Plain UC.

### 1.1.1 Problems in Plain UC and its generalizations.

The identified issues, limitations, and weaknesses presented in the main body exhibit problems with all the properties required by the UC Theorem. They are listed and classified into four categories in Table 1, and outlined in the Technical Overview (Section 2).

As a side contribution, the extended version [CS25, Appendix D] describes additional problems related to other concepts, such as the time-complexity notion used in UC and the concept of a Balanced Environment (See Technical Overview, Section 2.8).

### 1.1.2 Updated UC.

To address the identified problems, we propose updated definitions for Plain UC, as summarized in Table 2, where even similarly named notions exhibit differences. We refer to the framework with these updated definitions as Updated UC.

**Table 2:** Correspondence between Plain UC and Updated UC notions.

| Plain UC [Can20] | Updated UC (this work) |
|---|---|
| Extended Session [Can20, (Section 3.1.2)] | Extended Session (Definition 3) |
| – | Panoptic Session (Definition 5) |
| Subroutine Respecting [Can20, (Section 5.2)] | Sub-party Respecting (Definition 9) |
| Universal Composition Operator [Can20, (Section 6.1)] | Universal Composition Operator (Section 6.7) |
| Subroutine Exposing [Can20, (Section 6.1)] | Sid-Exposing (Definition 8) |
| Compliant [Can20, (Section 6.1)] | Compliant (Definition 10) |
| – | Disjunction (Definition 6) |
| – | Separation (Definition 7) |
| Balanced Environment [Can20, (Section 4.2)] | – |
| – | Replacement Execution (Section 6.2) |

Although most of these updates introduce non-trivial modifications, they remain grounded in the same computational model and core design principles. Moreover, while some updates restrict the framework's generality, this is done deliberately to exclude the counterexamples to the UC Theorem that we identified.

While individual verification is still required, we expect that existing protocols proven secure under Plain UC already meet the assumptions of the UC Theorem as formulated in Updated UC.

The justification for these updates is twofold.

First, we motivate the vast majority of the definitional changes by showing how they resolve specific issues previously identified. These explanations illustrate that, although some modifications are nontrivial, Updated UC should not be viewed as a distinct model, but rather as a necessary refinement of Plain UC to guarantee its correctness.[1]

Second, we provide a new formulation of the UC Theorem based on these updated definitions (and proven in the extended version [CS25, Appendix C]). While the main ideas remain those of Plain UC, adapted to Updated UC, our formulation of the UC Theorem relies on a definition of the UC operator that we introduce alongside the presentation of Updated UC. This definition allows the UC Theorem to be stated and proven independently of any specific implementation of the UC operator, thereby adding modularity and simplifying the overall analysis.

## 1.2 Related Work

Aside from the current version of Plain UC, widely used in this work, we can classify previously known publications into three categories:

- Generalizations of Plain UC, such as UC with joint state [CR03], UC with global setup [CDPW07], and multi-protocol UC [CDT19]. While the problems we have identified in Plain UC can be directly adapted to these generalizations, updating their UC theorems requires additional work. Although further research is needed, we believe that the updates to Plain UC offer a solid foundation for updating these

[1] A few minor changes are not directly related to the identified issues but were found necessary to ensure the correctness of the proof of the new UC Theorem. The fact that these changes are not implemented in Plain UC may itself be the source of additional issues for Updated UC; investigating this is left for future work.

generalizations. This confidence is partly justified by [BCH+20], which demonstrates that the UC theorem from Plain UC can be used to prove the UC theorem for UC with global setup.

- Alternative frameworks to Plain UC that rely on different computational models, such as iUC [CKKR19] (based on the IITM model [KTR20, RKC22]), CC [Mau12], and GNUC [HS15]. Both [HS15] and [KTR20] identify significant issues in older versions of Plain UC (specifically the 2005 and 2013 revisions), leading to counterexamples of the UC theorem. These issues are addressed by the current version of Plain UC that we analyze, and they arise from different ideas than the problems we present in this work. Appendix J of [RKC22] highlights minor issues distinct from those we identified, which can be resolved by slightly refining Plain UC 's protocol-execution definition; our description of Plain UC incorporates those refinements. Given that the problems of Plain UC identified in our work are new, it may be worthwhile to investigate if any of them can be adapted to these alternative frameworks. Finally, we emphasize that our aim is not to create a new framework, but to ensure the correctness of Plain UC by updating its definitions.
- Simplification of Plain UC, such as the one described in [CCL15], which presents a simplified variant within the context of standard multiparty computation. This variant deliberately restricts the expressiveness of Plain UC, for instance, by removing the possibility for protocols to have sub-parties [CCL15, (Section 3.2)]. Since its security is based on embedding this variant into Plain UC, we believe some of the issues we identified may also be relevant to this simplified version. Moreover, our updates to Plain UC pursue different goals: they aim to preserve as much expressiveness as possible.
- Comparison of UC with other frameworks, such as [RKC22] that compare the UC and IITM frameworks. It would be interesting to explore how the updated definitions of UC impact such comparisons.

## Acknowledgements

This work was supported in part by the French ANR project SecNISQ (ANR-21-CE47-0014) and through the France 2030 program under the RESQUE project.

We express our deepest appreciation to Thomas Ricosset, whose insightful comments improved the presentation of the paper throughout its creation; Hugo Beguinet, whose feedback enhanced the flow of the main body; Erkan Tairi, for his constructive advice that refined the introduction and technical details; and Paula Celles, for her invaluable assistance in elevating the overall writing quality.

**Outline of the paper.**

Section 2 describes the main contributions of the paper, in the form of a technical overview.

The following sections elaborate on our contributions. First, Section 3 provides preliminary notations, followed by a description of Plain UC in Section 4. Then, Section 5 identifies various issues, limitations, and weaknesses within Plain UC. Section 6 introduces Updated UC, and Section 7 presents the UC Theorem associated with Updated UC.

Moreover, Appendix A contains the missing details related to the issues, limitations, and weaknesses discussed in the main body.

Note that the extended version [CS25] includes additional contributions. Appendix C[2] presents a comprehensive proof of the UC Theorem of Updated UC. Appendix D discusses further issues, limitations, and weaknesses that, while not essential for understanding the

[2]It begins with C, rather than B, because the extended version includes in Appendix A a quick guide to the appendices.

main content, offer valuable insights. These include analyses of the notion of time complexity within Plain UC, the concept of a balanced environment, and the possibility of specifying the receiver of an external write via a predicate rather than by its extended identity. Finally, Appendix E defines a UC operator that addresses the problems associated with Plain UC's operator and aligns with our general definition of a UC operator (Definition 13).

Finally, the table of contents is provided at the end of the document.

# 2 Technical Overview

In this technical overview, we provide a detailed account of the main results of our article. Due to the technical nature of our contributions, some familiarity with Plain UC is required. To facilitate understanding, a comprehensive overview of Plain UC is provided in Section 4. Furthermore, from this point onward and throughout the rest of the article, we focus exclusively on Plain UC and the update proposed in this paper.

## 2.1 A Flaw in the Extended Session Definition Making the UC Theorem Unusable for Practical Protocols

In Plain UC, the extended session of a protocol $\pi$ and session identifier $\mathsf{s}$ for a fixed execution is a set denoted by $\mathsf{Ext}(\pi, \mathsf{s})$.[3] It is constructed interactively during the execution by observing the ITIs that are invoked. All ITIs with code $\pi$ and session identifier $\mathsf{s}$, known as the main parties, are included and form the subset $\mathsf{MainParty}(\pi, \mathsf{s}) \subset \mathsf{Ext}(\pi, \mathsf{s})$. Additionally, ITIs that serve to perform computations for the main parties or other members of the extended session are also added. These are referred to as sub-parties and form the subset $\mathsf{ExtSub}(\pi, \mathsf{s}) \subset \mathsf{Ext}(\pi, \mathsf{s})$.

According to the precise definition of an extended session, an ITI $N$ cannot be part of $\mathsf{ExtSub}(\pi, \mathsf{s})$ if it has not been invoked by a member of $\mathsf{Ext}(\pi, \mathsf{s})$ – see the discussion on page 17 in the overview of Plain UC.

In our work, we show that this condition is overly restrictive. In particular, we demonstrate that most ideal protocols are not subroutine respecting, and that the method described in [Can20, Section 5.2, Paragraph "Writing subroutine respecting protocols"] fails – contrary to its stated goal – to generically transform an arbitrary protocol into a subroutine respecting one.

To rectify this, we introduce the concept of "sleeping ITI" in Section 6.1. A sleeping ITI is one that has either not been invoked or has rejected all received messages thus far. In the earlier example, after being invoked by any ITI $U$ that does not belong to $\mathsf{Ext}(\pi, s)$, $N$ remains a sleeping ITI because it rejects U's message. Based on this, we revise the definition of an extended session in Section 6.1.

## 2.2 Protocols with Dynamic Sub-Parties: A Family of Protocols Unfit for Plain UC

Before delving into the details, we summarize our contribution in a single paragraph. We show that the standard notion of subroutine respecting is insufficiently restrictive, allowing the UC Theorem to be applied to a family of protocols that do not properly fit the UC framework. Specifically, Issue (I.2) (Section 5.1) utilizes one such protocol to construct a counterexample to the UC Theorem.

In simple terms, a protocol $\pi$ is considered subroutine respecting if for any execution of any protocol – not necessarily $\pi$ itself – and for any sid $\mathsf{s}$, any communication between

[3] Formally, the notation should indicate the specific execution being considered.

an ITI within $\mathsf{Ext}(\pi, \mathsf{s})$ and any other ITI must go through the main parties of $\mathsf{Ext}(\pi, \mathsf{s})$. In particular, for any execution[4]:

- Any input or output sent by an ITI $U \notin \mathsf{Ext}(\pi, \mathsf{s})$ to an ITI in $\mathsf{ExtSub}(\pi, \mathsf{s})$ is rejected.
- No ITI in $\mathsf{ExtSub}(\pi, \mathsf{s})$ sends an input or output to an ITI $U \notin \mathsf{Ext}(\pi, \mathsf{s})$.

This concept is of crucial importance for the UC Theorem because it allows us to "isolate" the members of $\mathsf{ExtSub}(\pi, \mathsf{s})$ from any execution, simulating any other ITI of the execution by an environment $\mathcal{E}_{\mathsf{sim}}$, as shown in Figure 1.[5]

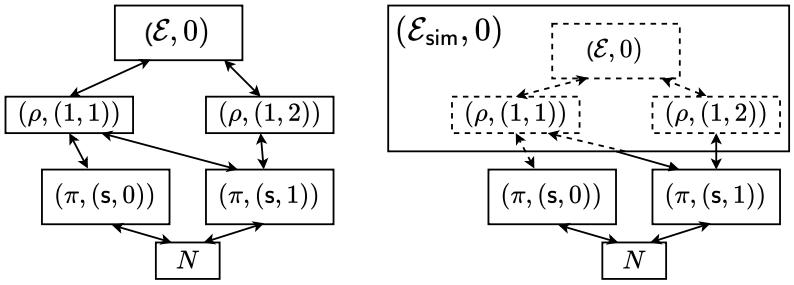



**Figure 1:** An example of "isolation": all ITIs are simulated except those within $\mathsf{Ext}(\pi, \mathsf{s})$

However, we discovered that the conditions required for being subroutine respecting are not sufficiently restrictive. This lack of constraint allows protocols that are not well-suited for analysis within the UC framework to satisfy the hypotheses of the UC Theorem, leading to counterexamples to the theorem.

We refer to these as protocols with "dynamic subparties". A protocol $\pi$ has a dynamic sub-party $N$ if there exists a session identifier $\mathsf{s}$ such that:

(a) In some executions, $N \in \mathsf{ExtSub}(\pi, \mathsf{s})$.

(b) In other executions, $N$ is not sleeping (i.e., it exists and has not rejected all received messages) and does not belong to $\mathsf{ExtSub}(\pi, \mathsf{s})$.

While we believe that no real-world protocols have dynamic sub-parties, it is possible to construct simple protocols with this characteristic that lead to counterexamples of the UC Theorem. Such a protocol is constructed in Issue (I.2), as detailed in Section 5.1.

These protocols do not fit the UC framework because, during the execution of a protocol $\pi$ with any adversary and environment, the execution rules prevent instances of condition (b) from occurring. These instances are only observable when considering the execution of another protocol $\rho$ that uses $\pi$. As a result, the full behavior of $\pi$ is unobservable when examining its UC-emulation by another protocol.

In order to exclude protocols with dynamic sub-parties from being considered subroutine respecting, we updated the notion of subroutine respecting protocols to develop the concept of sub-party respecting protocols (see Section 6.5) used in Updated UC. It uses the new concept of Panoptic Session, which is detailed in the next section.

## 2.3 Panoptic Session: A Static Variant of Extended Session

The concept of an extended session is dynamic because it is tied to a specific execution of the protocol. However, as discussed in the previous section on dynamic sub-parties, it

[4]For simplicity, we do not describe all the conditions.

[5]Strictly speaking, achieving isolation requires additional concepts, such as compliance. However, the subroutine respecting notion remains the principal component.

might be beneficial to consider not only the *actual* sub-parties of an execution but also the *potential* sub-parties – that is, those that appear in some execution, without specifying which one.

To capture this idea, we introduce the notion of Panoptic Session (Section 6.3). For a protocol $\pi$ and session identifier $\mathsf{s}$, the panoptic session $\mathsf{Pan}(\pi, \mathsf{s})$ is defined as the union of all extended sessions $\mathsf{Ext}(\pi, \mathsf{s})$ across all executions of $\pi$ with any environment and adversary. Therefore, the panoptic session is a static concept that provides information about the potential sub-parties of a protocol $\pi$.

For example, in the situation depicted in Figure 2, $\mathsf{Ext}(\pi, 0) = \{((\pi, (0,0)), N_i\}$ for each execution such that $(\pi, (0,0))$ sampled $i$, and $\{(\pi, (0,0))\} \cup \{N_i : i \in \{1, \ldots, 100\}\} \subset \mathsf{Pan}(\pi, 0)$.

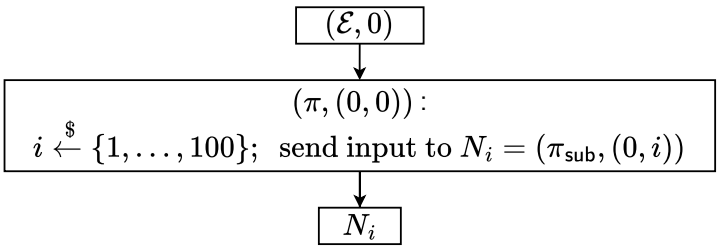



**Figure 2:** Example to illustrate the difference between Extended and Panoptic Sessions.

We assert that computing the panoptic sessions is straightforward for practical protocols. In fact, for any protocol analyzed under Plain UC, the panoptic sessions are already effectively determined, since checking the subroutine respecting condition requires computing the extended sessions that comprise it.

In the previous section, we explained that panoptic sessions are used to update the notion of subroutine respecting. In the upcoming section, we will demonstrate their applicability in addressing a flaw related to the subroutine exposing notion.

## 2.4 Problems with the Subroutine Exposing Property

In this section, we identify two flaws in the subroutine-exposing property – a requirement of the UC Theorem – and explain how they are addressed by the sid-exposing property, which updates it in Updated UC.

For a protocol $\alpha$, the subroutine exposing property requires that for each sid $\mathsf{s}$, one of the ITIs in the extended session $\mathsf{Ext}(\alpha, \mathsf{s})$ can inform – upon request – the adversary whether an ITI $N$ belongs to the extended session $\mathsf{Ext}(\alpha, \mathsf{s})$ at the time of the request. We refer to this ITI as the exposing ITI of the extended session $\mathsf{Ext}(\alpha, \mathsf{s})$, or simply the exposing ITI when the context is clear.[6] It is also required that the extended identity of the exposing ITI for the extended session $\mathsf{Ext}(\alpha, \mathsf{s})$ can be determined using only $\alpha$ and $\mathsf{s}$.

### 2.4.1 A Flawed Generic Transformation for Adding the Subroutine Exposing Property.

There is a straightforward mechanism, described in [Can20, Section 6.1], that transforms any protocol $\phi$ into a subroutine-exposing protocol $\phi_{\mathsf{SE}}$. To our knowledge, this is the only generic method for adding the subroutine-exposing property.

[6] Note that the exposing ITI is sometimes called a "directory ITI" in Plain UC.

In summary, $\phi_{\mathsf{SE}}$ introduces in any extended session $\mathsf{Ext}(\phi_{\mathsf{SE}}, \mathsf{s})$ a middle-man $M_{\phi,\mathsf{s}}$ which is aware of all ITIs belonging to the extended session $\mathsf{Ext}(\phi_{\mathsf{SE}}, \mathsf{s})$ and can therefore act as an exposing ITI, telling the adversary whether any ITI $N$ belongs to $\mathsf{Ext}(\phi_{\mathsf{SE}}, \mathsf{s})$.

To the best of our knowledge, most protocols in the UC framework are not subroutine exposing. We substantiate this claim in Appendix A.6 by presenting numerous examples of ideal protocols that lack this property, including those discussed in [Can20, Section 6.1]. Consequently, applying this transformation is essential to endow them with the property and ensure they benefit from the UC Theorem's security guarantees – although its use is almost always implicit.

For the transformation to be sound, it must preserve UC-emulation: whenever $\pi$ UC-emulates $\phi$, it should follow that $\pi_{\mathsf{SE}}$ UC-emulates $\phi_{\mathsf{SE}}$. However, as shown by Limitation (L.2) (Section 5.3), this preservation is not guaranteed. While [Can20, Section 6.1] does not claim this property, we are not aware of prior works that explicitly analyze why it fails.

More importantly, because most protocols in the literature are not subroutine-exposing and no other generic method exists to achieve this property, such protocols cannot formally benefit from the composable security guarantees offered by the UC Theorem. To ensure they do, the preservation of UC-emulation must be analyzed on a case-by-case basis for each protocol[7]

We illustrate the problem with a simple example depicted in Figure 3. In this example, $\phi^*$ performs cryptographic operations without utilizing any subroutines, whereas the protocol $\pi^*$ executes the same operations using two specialized subroutines. These subroutines are selected based on the type of input – for example, $\alpha$ might use an algorithm optimized for inputs of type 1, while $\beta$ is more efficient for inputs of type 2.

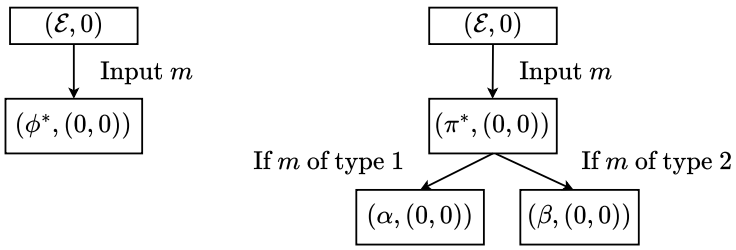



**Figure 3:** Protocols $\phi^*$ and $\pi^*$ used to describe a flaw in the transformation $\mu \to \mu_{\mathsf{SE}}$.

By definition of $\pi^*$, $(\alpha, (0,0))$ (*resp.* $(\beta, (0,0))$) is part of the extended session $\mathsf{Ext}(\pi, 0)$ if and only if the environment sent to $(\pi, (0,0))$ an input of type 1 (*resp.* of type 2). This implies that an adversary against $\pi^*_{\mathsf{SE}}$ can leverage the exposing ITI to determine whether the environment sent an input of type 1 or 2. In contrast, an adversary against $\phi^*_{\mathsf{SE}}$ does not gain additional information by using the exposing ITI.

This asymmetry in the information provided by the transformation illustrates why one cannot conclude that $\pi^*_{\mathsf{SE}}$ UC-emulates $\phi^*_{\mathsf{SE}}$ simply because $\pi^*$ UC-emulates $\phi^*$.

To resolve this problem, we leverage the Panoptic Session concept to refine the subroutine exposing property, yielding the sid-exposing property of Updated UC (Section 6.4). Under this notion, the exposing ITI can only determine whether an ITI is in $\mathsf{Pan}(\pi, 0)$ rather than in $\mathsf{Ext}(\pi, 0)$. Informally, the "updated" exposing ITI – its equivalent in the sid-exposing property – reveals if a subroutine *could* belong to $\mathsf{Ext}(\pi, 0)$, whereas the "original" exposing ITI indicates if it *actually* belongs to $\mathsf{Ext}(\pi, 0)$. In particular, the the updated exposing ITI provides only static information: it does not need to interact with

[7] We do not claim that such an analysis is complex, but rather that it is required – and generally absent from the literature.

the ITIs of the current execution.[8] Consequently, the updated exposing ITI do not need to be part of the protocol : it can be defined simply as a Turing machine, which the adversary may simulate if desired. This is illustrated by Figure 4.

Applying this updated definition to the example in Figure 3, we see that it is sid-exposing, with no need for any transformation. The updated exposing ITI cannot help distinguish the type of input received from the environment: it answers affirmatively for both $(\alpha, (0,0))$ and $(\beta, (0,0))$, since both belong to $\mathsf{Pan}(\pi, 0)$. Hence, our sid-exposing property enables a broader class of protocols to be analyzed within the UC framework.

More generally, all the protocols presented in Appendix A.6 are already sid-exposing, without any modification required, and we believe this also holds for most protocols analyzed within the UC framework in the literature.

As a result, our new notion eliminates the need for a transformation such as $\pi \mapsto \pi_{\mathsf{SE}}$, along with its associated limitations.

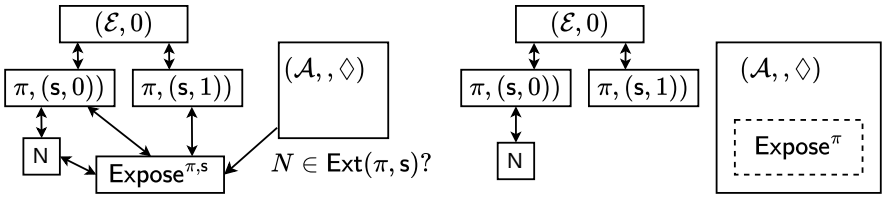



**Figure 4:** On the left-hand side, an execution illustrates the exposing ITI of the subroutine exposing property of Plain UC. On the right-hand side, an execution shows how an adversary can simulate the exposing ITI of the sid-exposing property of Updated UC.

### 2.4.2 A Problematic Dependence on the Session Identifier.

As stated in the definition of the subroutine exposing property, the exposing ITI for an extended session $\mathsf{Ext}(\pi, \mathsf{s})$ requires knowledge of both $\pi$ and $\mathsf{s}$ to be used.

However, Weakness (W.2) (Section 5.3) shows that this requirement compromises the soundness of the UC Theorem's proof. Specifically, one core construction of the demonstration implicitly assumes knowledge of some sids without any guarantee of such knowledge. Moreover, attempting to guess these sids can result in an exponential time complexity.

The sid-exposing notion in Updated UC (Section 6.4) addresses this by making the exposing ITI depend only on the protocol $\pi$. More precisely, when given an ITI $N$[9], it returns the sid $\mathsf{s}$ of the panoptic session $\mathsf{Pan}(\pi, \mathsf{s})$ containing $N$, if such a sid exists.[10]

## 2.5 Extending Security Definitions Beyond Protocol Executions

In Plain UC, subroutine-respecting and compliance properties require that certain conditions apply to every protocol execution. However, the UC Theorem's proof for Plain UC studies some executions of extended systems that are not protocol executions – recall that a protocol execution is just a special case of an extended-system execution. We call these *replacement executions*.

[8] In contrast, the exposing ITI of the subroutine exposing notion provides dynamic information tied to the current execution.

[9] Formally, the input is an extended identity.

[10] We ensure uniqueness by requiring $\mathsf{Pan}(\pi, \mathsf{s}) \cap \mathsf{Pan}(\pi, \mathsf{s}') = \emptyset$ for $\mathsf{s} \neq \mathsf{s}'$, this is the separation property (Definition 7).

Informally, a replacement execution of a protocol $\rho$ is one in which some subroutines with code $\phi$ are substituted by other subroutines with code $\pi$. Figure 5 illustrates this: the left-hand side shows a normal execution, while the right-hand side depicts a replacement execution.

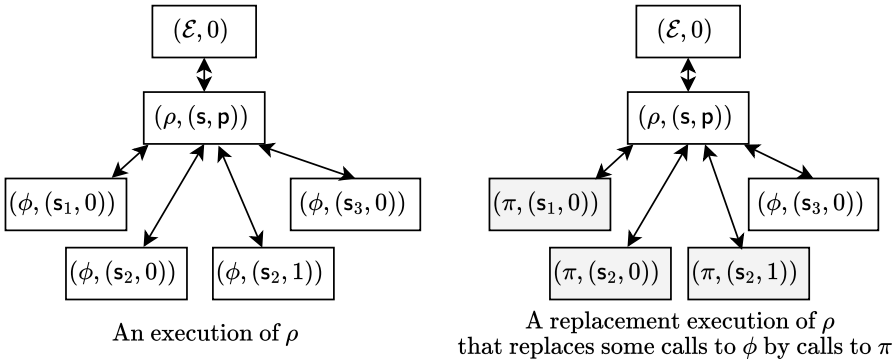



**Figure 5:** Illustration of a replacement execution.

In this work, we demonstrate that the conditions required by the subroutine respecting and compliance notions should apply not only to protocol executions but also to certain replacement executions.

In particular, Issue (I.3) (Section 5.1) exploits this omission in the subroutine respecting notion to construct a counterexample to the UC Theorem of Plain UC.[11]

In Updated UC, we update both the subroutine respecting and compliance notions to enforce their conditions on these additional executions as well. To that end, we formally define the notion of replacement execution in Section 6.2.[12]

## 2.6 Identifying Concerns in the UC Operator and Reimagining It for Enhanced Modularity

As explained in the introduction, the UC Theorem requires the construction of a protocol $\rho^{\phi \to \pi}$ for protocols $\rho$, $\phi$, and $\pi$. Simply stated, $\rho^{\phi \to \pi}$ is a modification of $\rho$ that replaces any call[13] to $\phi$ by a call to $\pi$. We refer to the function that, given $\rho, \phi, \pi$, returns $\rho^{\phi \to \pi}$ as the UC Operator.

While the purpose of the UC Operator can be simply described, its detailed construction [Can20, (Section 6.1)] is intricate, demanding careful attention to various challenges. This intricacy is the origin of two main contributions of our article. First, we identified a subtle issue in its conception, leading to a complete counterexample of the UC Theorem (Issue (I.4) in Section 5.2). Second, we found a problem in how the UC Operator is used within the proof of the UC Theorem (Weakness (W.1) in Section 5.2).

To mitigate these two problems, it would be sufficient to create a new UC Operator and revisit the proof of the UC Theorem. However, we believe these issues illustrate that the UC Theorem should not directly depend on any specific UC Operator construction. Instead, a more modular formulation is necessary.

[11] Regarding the compliance, the corresponding problem we found is treated as a weakness (Weakness (W.4) in Section 5.4), since we have not attempted to build a counterexample based on it.

[12] A related notion already appears in the proof of the UC Theorem [Can20, (Theorem 22)] within Plain UC; however, it was used only within that proof and not in the definition of any protocol property, such as subroutine respecting or compliance.

[13] Here, we simplify the explanation by omitting the concept of top-level sessions [Can20, (Section 6.1)], which is not essential for understanding and will not be required for the rest of the document. More formally, we should speak of any call to a top-level main party of $\pi$.

To achieve this, we introduce in Section 6.7 a general definition of the UC Operator. Our definition outlines the necessary properties for a UC Operator, regardless of how it is precisely constructed. This enables us to provide a formulation of the UC Theorem that depends solely on this abstract definition (Theorem 1). Any operator that satisfies this definition can then be utilized, resulting in a "concrete" version of the UC Theorem using this operator (Corollary 1).

This approach enhances modularity, offering several advantages. The proof of the UC Theorem becomes simpler because it does not need to address the construction nuances of a particular UC Operator, effectively addressing Weakness (W.1). Moreover, the correctness of the theorem remains unaffected by any potential errors in an individual UC Operator.

As a supplementary contribution, we provide in the extended version [CS25, Appendix E] the construction of a UC Operator that conforms to our definition. Notably, our construction is unaffected by Issue (I.4).

## 2.7 Revealing Hidden Scope Constraints in the UC Theorem

The UC Theorem states that for protocols $\phi$, $\pi$, and $\rho$, under specific hypotheses, if $\pi$ UC-emulates $\phi$, then $\rho^{\phi\to\pi}$ UC-emulates $\rho$. Recall that $\rho^{\phi\to\pi}$ is a modification of $\rho$ where instances of $\phi$ are replaced by instances of $\pi$.

In this section, we explain that the UC Theorem does not impose sufficient restrictions on which protocols $\rho$ can be used.

Indeed, while some restrictions apply to $\rho$, it is possible for $\rho$ to invoke ITIs with both codes $\phi$ and $\pi$. In $\rho^{\phi\to\pi}$, ITIs with code $\phi$ are replaced with $\pi$, whereas ITIs with code $\pi$ remain unchanged. Consequently, ITIs $N$ with code $\pi$ in $\rho^{\phi\to\pi}$ can be classified into two types:

- Type 1: $N$ was invoked due to a replacement $\phi \to \pi$,
- Type 2: $N$ is not the result of such a replacement.

This is illustrated by Figure 6.

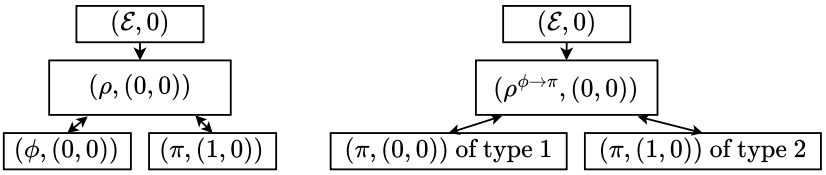



**Figure 6:** An illustration of the two types of ITIs with code $\pi$ described in Section 2.7.

The existence of these two types of ITIs gives rise to a subtle weakness in the proof of the UC Theorem (Weakness (W.3), Section 5.4). Specifically, the proof requires one adversary to distinguish between Type 1 and Type 2 ITIs, yet it lacks the information to make this distinction.[14]

As a result, contrary to its stated claim, the UC Theorem applies only to protocols $\rho$ whose executions exclude any Type 2 ITI.[15] This restriction still applies to Updated UC, and is captured by the update of the compliance notion (see Section 6.6).

[14] In specific cases – where additional information about $\rho$ is available – this restriction could be relaxed to distinguish between Type 1 and Type 2. However, such information is not always available, which creates the problem.

[15] It also formally applies to protocols excluding any Type 1 ITI, but that case is vacuous – $\rho^{\phi\to\pi}$ would never replace $\phi$ with $\pi$.

### 2.8 Additional Contributions

As a side contribution, we discuss additional problems in the extended version [CS25, Appendix D]. These are listed in Table 3 and cover three concepts not addressed by the problems described in the main body.[16] These concepts are:

- The ability of a machine to *designate a message receiver using a predicate* rather than specifying its full code and identity [Can20, Section 3.1.2].
- The *time-complexity* notion used in Plain UC [Can20, Section 3.2].
- The concept of *balanced environment* [Can20, Section 4.2], proposed to prevent situations where the adversary is not allocated sufficient execution time.

**Table 3:** Additional issues, limitations, and weaknesses described in the extended version [CS25, Appendix D]. They complete the main problems listed in Table 1.

| Notions | Issues | Limitations | Weaknesses |
|---|---|---|---|
| Extended Session Subroutine Respecting | | (L.3) | |
| UC Operator | (I.5) (I.6) | | |
| Predicates to Designate Receiver | (I.7) | (L.4) | |
| Time Complexity | | | (W.6) (W.7) |
| Balanced Environment | | | (W.8) |
| Other | | | (W.5) |

To address these additional problems, we implement the following changes in Updated UC:

- We remove the ability to use predicates to designate receivers. This simplifies our presentation, and we have not identified any protocols proven UC-secure that rely on them.
- Regarding the issues with the time complexity notion in Plain UC, which are not central to our article, we adopt a simpler notion, leaving a more general treatment for future work.
- Since we study a weakness related to the notion of balanced environments in the extended version [CS25, Appendix D.6], and because time-complexity concerns are not central to our work, we do not provide a direct analogue of the balanced environment concept from Plain UC. However, our definition of a PPT adversary includes guarantees that ensure sufficient computation time.

## 3 Preliminary Notations

The set of binary words is denoted by $\{0,1\}^*$ and the empty word by $\epsilon$. In both Plain UC and Updated UC, we need to represent pairs and additional symbols – such as $\Diamond$ or $\perp$ – on the tapes of Turing Machines, which can normally only contain binary words. We achieve this by implicitly using an encoding to represent them within $\{0,1\}^*$.

[16] Note that the category "other" used in the table refers to a weakness in the proof of the UC Theorem that is not directly related to any specific concept of UC. It addresses a deficiency in the definition of an adversary.

# 4 Overview of Plain UC

In this section, we recall some fundamental concepts of Plain UC, which will largely be utilized in Updated UC as well.

We made slight adjustments to the presentation, particularly in the definition of protocol execution. In addition, two more significant changes are made compared to Plain UC. First, as stated in the technical details (Section 2.8), we do not allow ITMs to designate recipients using predicates. Second, we use an alternative notion of time complexity along with a slightly modified notion of parametrized systems.

Because they are not necessary for understanding the problems of Plain UC discussed later, we omit the original time complexity definitions from the main body. However, they are recalled and discussed in the extended version [CS25, Appendix D.5].

Finally, note that some notions – significantly revised in Updated UC – are not described here. Instead, they will be introduced in the sections that address the corresponding issues in Plain UC. These include concepts such as subroutine respecting, subroutine exposing, and compliance.

**ITM, External Write, Protocol.** The machines evoked in introduction are Interactive Turing Machines (ITM) [Can20, (Definition 4)]. They are special Turing machines that can send messages to each other through an operation called *external-write*, described below. We generally denote the code of an ITM using lowercase Greek letters.

An ITM $\mu$ contains multiples tapes, including:

- A read-only *identity tape* containing its code $\mu$ and an identity $\mathsf{id} \in \{0,1\}^*$. This pair $(\mu, \mathsf{id})$ is known as its *extended identity*.
- An *activation tape* containing 0 or 1, used during execution to designate which ITM can perform computations – only one ITM at a time.
- A read-only, read-once *randomness tape*, initialized with random bits.
- An *outgoing message tape* used to (1) send messages via the external-write operation, and (2) indicate the execution result (see the paragraph "Extended System and Execution of an Extended System" page 15).
- Three externally-writable tapes (*i.e.* writable only by other ITMs through the external-write operation) for receiving messages sent by other ITMs, with one tape for each type of message: an *input*, a *subroutine-output* (often abbreviated as *output*), and a *backdoor* tape.

Before executing an external-write instruction, an ITM must specify all information regarding the message to be sent. This is achievied by writing a tuple $(\mathsf{forced\text{-}write}, \mathsf{extid}_{\mathsf{rec}}, \mathsf{tape}, \mathsf{reveal\text{-}ext\text{-}id}, \mathsf{extid}_{\mathsf{send}}, \mathsf{m})$ on the outgoing message tape, where:

- $\mathsf{extid}_{\mathsf{rec}}$ is the extended identity of the recipient.
- $\mathsf{tape} \in \{\mathsf{input}, \mathsf{subroutine\text{-}output}, \mathsf{backdoor}\}$ is the destination tape.
- $\mathsf{m} \in \{0,1\}^*$ is the message to be sent, commonly referred to as input, output, or backdoor, depending on the chosen destination tape.
- $\mathsf{reveal\text{-}ext\text{-}id} \in \{0,1\}$ is a flag indicating whether the sender's external identity is sent with $\mathsf{m}$.
- $\mathsf{forced\text{-}write} \in \{0,1\}$ is a flag indicating whether the recipient ITI should be created if it does not exist when the external-write is applied.

We denote by $\mathsf{SetExtWrites}$ the set of such tuples. The complete behavior of this instruction is described in paragraph "Extended System and Execution of an Extended System", page 15

We often use the term protocol to designate an ITM or a parametrized system (defined in paragraph "Parametrized Systems and Execution of a Parametrized System" page 16).

**Configuration, ITI.** A *configuration* [Can20, (Definition 5)] of an ITM consists of its current tape, the contents of all its tapes, and the location of the head on each tape. A configuration is active if the activation tape is set to 1, otherwise it is inactive.

An instance [Can20, (Definition 5)] of an ITM (ITI) $\mu$ consists of the content of its identity tape, whose is its extended identity $M = (\mu, \mathsf{id})$. By abusing notation, we often speak of "ITI" instead of "extended identity". For example, if an algorithm has an input which is an extended identity, we say that its input is an ITI. We generally denote ITIs using uppercase Latin letters.

**Extended System and Execution of an Extended System.** An extended system of ITMs [Can20, (section 3.1.2)] is a pair $(\mu, \mathcal{C})$, where $\mu$ is an ITM called the initial ITM and $\mathcal{C} : \{0,1\}^* \to \{\mathsf{allow}, \mathsf{disallow}\} \cup \mathsf{SetExtWrites}$ is a function called the control function, which role is described below.

To execute an extended system $(\mu, \mathcal{C})$ with an input $z \in \{0,1\}^*$, the ITI $(\mu, 0)$, called the initial ITI, is first activated – *i.e.* 1 is written on its activation tape –, with $z$ written on its input tape. During execution, there is no distinct ITIs with same extended identity and only the activated ITI can perform operations. An ITI will be deactivated in favor of another ITI in two occasions: when it makes an external-write or when it deactivates itself. We describe these two situations below and conclude the paragraph by defining the end and result of an execution.

External-write. When activated, an ITI can send inputs, outputs, or backdoors to another ITI via the external-write operation. If the recipient ITI does not exist and forced-write is set, the recipient ITI is created in its initial state and receives the message in the corresponding input, output or backdoor tape. If the recipient ITI does not exist and forced-write is unset, the initial ITI $(\mu, 0)$ is instead activated, with no message. The value of flag reveal-ext-id is written with the message and, if it is set, the extended identity of the sender is also written.[17] The effect of an external-write can be modified by the control function $\mathcal{C}$. When an external-write operation is made, $\mathcal{C}$ receives the history of all consecutive configurations of each activated ITIs and can decide to allow, disallow or replace the external-write operation with another. If the control function allows the external-write, nothing is changed. If it is disallowed, initial ITI $(\mu, 0)$ is activated next, with no message. If it is replaced by another external-write operation, the new external-write operation is applied.

Self-Deactivation, Reject and Halt. An activated ITI can deactivate itself. When this is done, the next activated ITI is the initial ITI $(\mu, 0)$, with no message. A *reject* [Can20, (Section 3.1.2)] is a specific case of deactivation, where the state of the ITI as well as all its non read-once tables are returned in their state prior to the last activation. Moreover, each ITI can enter in *halt* state. Upon entering the halt state, an ITI deactivates and will reject any subsequent messages.

End of Execution. The execution ends when the initial ITI halts. The result of the execution is the first bit of the outgoing message tape. We sometimes say "the environment returns $b$" to indicate the termination and result of the execution.

**Extended Execution.** By an *extended execution*, we mean the execution of an extended system.

**Execution Prefix.** An execution prefix of an extended execution is a complete description of the extended execution from its start up to a given point in time.

**Protocol Execution.** The most important example of extended execution is the protocol execution [Can20, (Section 4.1)].

Let $\alpha, \mathcal{A}, \mathcal{E}$ three ITMs. We will refer to any ITI different of $(\mathcal{E}, 0)$ and $(\mathcal{A}, \Diamond)$ as a

[17] The various elements written with the message are encoded in a way that ensures they are clearly separated. For example, it is not possible to perform an external-write with reveal-ext-id unset such that the recipient would interpret it as one with reveal-ext-id set.

protocol ITI.[18] The execution of the protocol $\alpha$ with adversary $\mathcal{A}$ and environment $\mathcal{E}$ is the execution of the extended system $(\mathcal{E}, \mathcal{C}_{\mathsf{EXEC}}^{\alpha,\mathcal{A},\mathcal{E}})$, the control function ensuring some restrictions:

- Unless explicitly stated otherwise, the identity of any recipient of an external-write operation must be structured as a pair $(\mathsf{s}, \mathsf{p}) \in \{0,1\}^* \times \{0,1\}^*$ where $\mathsf{s}$ is called the *session identifier* (sid) and $\mathsf{p}$ the *party identifier* (pid).
- $(\mathcal{E}, 0)$ always set forced-write and can only send inputs. When $(\mathcal{E}, 0)$ sends input to an ITI with identity $\Diamond$, the recipient ITI is replaced by $(\mathcal{A}, \Diamond)$. When $(\mathcal{E}, 0)$ sends an input to an ITI with identity distinct of $\Diamond$, the recipient's code is replaced by $\alpha$. When $\mathcal{E}$ sends an input to an ITI with identity distinct of $\Diamond$, the identity must be structured as a couple $(\mathsf{s}, \mathsf{p})$ and it must consistently use the same sid $\mathsf{s}$, usually denoted as $\mathsf{s}_{\mathsf{test}}$ and called the test sid.
  Finally, we require that $(\mathcal{E}, 0)$ always sets reveal-ext-id, but it has impersonation power: in all its external-write operations, $\mathcal{E}$ can use a sender ITI other than $(\mathcal{E}, 0)$.[19]
- $(\mathcal{A}, \Diamond)$ always unsets forced-write, reveal-ext-id and can only send backdoors.[20] If it sends a backdoor to an ITI with identity 0, the recipient is replaced by $(\mathcal{E}, 0)$.
- Protocol ITIs always set reveal-ext-id and forced-write.[21] [22] When a protocol ITI sends a backdoor, the recipient ITI is replaced by $(\mathcal{A}, \Diamond)$. When a protocol ITI with code $\alpha$ and sid $\mathsf{s}_{\mathsf{test}}$ (the test sid) sends an output to an ITI $M$ that (1) does not have code $\alpha$ and sid $\mathsf{s}_{\mathsf{test}}$, and (2) does not exist in the system, the message is written to the output tape of $(\mathcal{E}, 0)$, along with the target extended identity and the sender's identity – but not the sender's code.[23]

**Protocol Execution Prefix.** A $\alpha$-execution prefix is a prefix of the execution of $\alpha$. A protocol execution prefix is a $\alpha$-execution prefix for some protocol $\alpha$.

**Parametrized Systems and Execution of a Parametrized System.** As mentioned at the beginning of this section, the definition of parametrized system we use is slightly different from that of Plain UC [Can20, (Section 3.2)].

The UC-emulation property is asymptotic, both in terms of the size of messages and with respect to a security parameter $k \in \mathbb{N}$. The notion of parametrized system is a generalization of the notion of an ITM that takes into account this security parameter. Essentially, it is a sequence of ITMs indexed by the security parameter.

More formally, a parametrized system is a sequence of ITMs $\boldsymbol{\phi} = (\phi_k)_{k\in\mathbb{N}}$ such that the code of each $\phi_k$ can be written in polynomial time in $k$. More precisely, there exists a Turing machine and a polynomial $P$ such that, given $k$, it returns the code $\phi_k$ in fewer than $P(k)$ operations. Parametrized systems will generally be written using bold lowercase Greek letters.

The execution of a parametrized system $\boldsymbol{\phi}$ is defined as the execution of a protocol, with slight differences:

- All the inputs to $(\mathcal{E}, 0)$ must be pairs $(k, z)$, for $k \in \mathbb{N}$ and $z \in \{0,1\}^*$. $k$ is called the security parameter of the execution.

[18] Note that in plain UC, the identity of adversary is normally $(\mathsf{s}_{\mathsf{test}}, \Diamond)$ and not $\Diamond$. This is a cosmetic change.

[19] This simplifies the original formulation where the environment could set or unset this flag. It is essentially equivalent because the impersonation power allows the environment to conceal its extended identity.

[20] [RKC22, Appendix J.1] explains why reveal-ext-id needs to be unset.

[21] In the original formulation, forced-write could be unset for inputs and outputs. However, since we removed the ability to use predicates to designate the receiver, we do not consider this option relevant.

[22] In the original formulation, forced-write needs to be unset for backdoor and thus the adversary cannot be invoked by a protocol ITI. This might be because executions are made with balanced environments [Can20, Section 4.2], where the adversary is invoked before any protocol sends a backdoor. As we do not restrict to such environment, we take in account the possibility for a protocol ITI to invoke the adversary.

[23] Note that in this situation, the sender identity takes the form $(\mathsf{s}_{\mathsf{test}}, \mathsf{p})$ for some $\mathsf{p} \in \{0,1\}^*$.

- The adversary has a new read-only tape containing the security parameter of the execution.[24]
- When the security parameter of the execution is $k$, the rules of execution of $\phi_k$ are applied.

We denote by $\mathcal{C}_{\mathsf{EXEC}}^{\boldsymbol{\phi},\mathcal{A},\mathcal{E}}$ the control function and by $\mathsf{EXEC}_{\boldsymbol{\phi},\mathcal{A},\mathcal{E}}(k,z)$ the binary random-variable representing the result of the execution of $\boldsymbol{\phi}$ with adversary $\mathcal{A}$ and environment $\mathcal{E}$ on input $(k,z)$.

**Subroutine, Extended Session.** Let us consider $M$ and $M'$ two ITIs in an execution prefix. $M'$ is a *subroutine* [Can20, (Section 3.1.2)] of $M$ if $M$ has passed input to $M'$ and $M'$ has not rejected it, or if $M'$ has passed output to $M$ and $M$ has not rejected it.

The *extended session* [Can20, (Section 3.1.2)] of a protocol can be viewed as the set of ITIs that are executed by an instance of the protocol. More formally, considering $\mu$ an ITM, a sid $\mathsf{s} \in \{0,1\}^*$ and some prefix of execution $\theta$ of a protocol, the extended session $\mathsf{Ext}(\mu,\mathsf{s},\theta)$ is a set constructed inductively by inspecting the ITIs by order of invocation: each newly invoked ITI $M$ is added to the extended session if (a) $M$ has code $\mu$ and sid $\mathsf{s}$ (such an ITI is called a main party); or (b) $M$ is a subroutine of an ITI that is already in the extended session; or (c) $M$ is invoked by an ITI that is a member of the extended session, but not a main party. Note that determining whether an ITI belongs to an extended session only requires examining the first message received by the ITI. This is because ITIs are processed in the "order of invocation" and that we focus on "newly invoked ITI". Therefore, it is equivalent to replace item (b) by: "it is invoked with an input by a main party and does not reject the input".

The ITIs with code $\mu$ and sid $\mathsf{s}$ are called the main parties of $\mathsf{Ext}(\mu,\mathsf{s},\theta)$ and form a set $\mathsf{MainParty}(\mu,\mathsf{s})$. The members of the extended session that are not main parties are called *sub-parties* and form a set $\mathsf{ExtSub}(\mu,\mathsf{s},\theta)$.

We will often omit the prefix $\theta$ from $\mathsf{Ext}(\mu,\mathsf{s},\theta)$ when clear from context. In the context of an execution, we speak of an extended session of $\mu$ to means any extended session $\mathsf{Ext}(\mu,\mathsf{s})$ for some sid $\mathsf{s} \in \{0,1\}^*$.

Finally, note that in the execution of a protocol $\mu$ with test sid $\mathsf{s}_{\mathsf{test}}$, the extended session $\mathsf{Ext}(\mu,\mathsf{s}_{\mathsf{test}})$ is referred to as the extended test session.

**Probabilistic Polynomial Time ITM.** We introduce here the notions of PPT (Probabilistic Polynomial Time) adversary, environment, and protocol – more formally defined in the extended version [CS25, Appendix C.2].

These definitions differ from the notion of PPT ITM given in Plain UC [Can20, (Section 3.2)]. They are partially inspired by the concept of a polynomial time machine from [CCL15, (Section 2.1)] and focused on the context of parametrized system execution.

An adversary is PPT if there exists a polynomial $P$ such that at any moment in any possible execution, its time of execution is bounded by $\sum_i P(|w_i|+k)$ where $(w_i)_i$ is the sequence of external-writes it received and $k$ the security parameter written on its dedicated tape. Thus, the adversary can always perform at least $n_M P(k)$ operations, where $n_M$ is the number of messages it received.

An environment is PPT if there exists a polynomial $P$ such that at any moment in any possible execution, its execution time is bounded by $P(k+|z|)$, where $(k,z) \in \mathbb{N} \cup \{0,1\}^*$ is its input, given at its first activation.

A protocol $\boldsymbol{\alpha}$ is PPT if for any PPT environment $\mathcal{E}$ and PPT adversary $\mathcal{A}$, there exists a polynomial $R$ such that any execution of $\boldsymbol{\alpha}$ with environment $\mathcal{E}$, adversary $\mathcal{A}$ and input $(k,z) \in \mathbb{N} \cup \{0,1\}^*$ has an execution time bounded by $R(k+|z|)$.

**UC-emulation.** The definition of UC-emulation relation we give is similar to that in Plain UC, but it incorporates our variants of parametrized system and PPT notions for

[24] This differs from the usual formulation of Plain UC, and we believe this variant is more practical.

adversary and environment. Additionally, we do not consider the notion of balanced environment [Can20, (Section 4.2)].[25]

To be defined, the UC-emulation relation requires three preliminary notions.

Let $\xi$ a predicate on extended identities. An environment is said $\xi$-*identity-bounded* [Can20, (Section 4.2)] if it only impersonates extended identities $M$ for which $\xi(M)$ is true.

A binary probability distribution ensemble [Can20, (Section 4.2)] is an infinite set $X = \{X(k,z)\}_{k\in\mathbb{N},z\in\{0,1\}^*}$ of binary probability distributions, indexed by $\mathbb{N}\times\{0,1\}^*$.

Two binary probability distribution ensembles $X$ and $Y$ are *indistinguishable* [Can20, (Definition 8)] – written $X \approx Y$ – if for any $c,d\in\mathbb{N}$, there exists $k_0\in\mathbb{N}$ such that for all $k > k_0$ and all $z\in\bigcup_{\kappa\leq k^d}\{0,1\}^\kappa$, we have: $|\Pr[X(k,z)=1]-\Pr[Y(k,z)=1]| < k^{-c}$. They are said to be *perfectly indistinguishable* if we always have $\Pr[X(k,z)=1]=\Pr[Y(k,z)=1]$. The execution of a parametrized system $\boldsymbol{\rho}$ naturally gives a binary probability distribution ensemble $(\mathsf{EXEC}_{\boldsymbol{\rho},\mathcal{A},\mathcal{E}}(k,z))_{k\in\mathbb{N},z\in\{0,1\}^*}$ that is used in the definition of UC-emulation.

**Definition 1** (UC-emulation [Can20, (Definition 9)])**.** We consider $\boldsymbol{\pi}$ and $\boldsymbol{\phi}$ two PPT parametrized systems and $\xi$ a predicate on extended identities. We say that $\boldsymbol{\pi}$ UC-emulates $\boldsymbol{\phi}$ with respect to $\xi$-identity-bounded environments (or, $\boldsymbol{\pi}$ $\xi$-UC-emulates $\boldsymbol{\phi}$) if for any PPT adversary $\mathcal{A}$, there exists a PPT adversary $\mathcal{S}$ such that for any PPT, $\xi$-identity-bounded environment $\mathcal{E}$, we have $\mathsf{EXEC}_{\boldsymbol{\pi},\mathcal{A},\mathcal{E}}\approx\mathsf{EXEC}_{\boldsymbol{\phi},\mathcal{S},\mathcal{E}}$. If $\xi$ is the predicate which is true for all ITIs, then $\boldsymbol{\pi}$ is said to UC-emulate $\boldsymbol{\phi}$. The UC-emulation is said to be perfect if the two probability distributions ensembles are perfectly indistinguishable. If this relation is only verified for a fixed adversary $\mathcal{A}$, we say that $\boldsymbol{\pi}$ $\xi$-UC-emulates $\boldsymbol{\phi}$ with respect to $\mathcal{A}$.

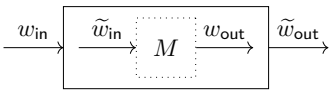
In the description of our counter-examples, we will consider UC-emulation for ITMs $\pi$ and $\phi$ instead of parametrized systems. When we state that $\pi$ UC-emulates $\phi$, it should be understood as $\boldsymbol{\pi}$ UC-emulates $\boldsymbol{\phi}$ for $\pi_k=\pi$ and $\phi_k=\phi$ for each $k\in\mathbb{N}$.

**Dummy Adversary.** In plain UC, an adversary $\mathcal{D}$, called the dummy adversary [Can20, Section 4.3.1], is of special importance as it can be shown that UC-emulation with respect to $\mathcal{D}$ implies UC-emulation [Can20, (Claim 11)] (*i.e.* UC-emulation with respect to any PPT adversary).

Specifically, the dummy adversary proceeds as follows. When activated with an input "$(\mathsf{m},(\mu,\mathsf{id}))$" from the environment $(\mathcal{E},0)$, where $\mathsf{m}$ is a message, and $(\mu,\mathsf{id})$ an extended identity, $(\mathcal{D},\Diamond)$ sends backdoor "$\mathsf{m}$" to ITI $(\mu,\mathsf{id})$. When activated with a backdoor "$\mathsf{m}$" sent by an ITI $(\mu,\mathsf{id})$, $(\mathcal{D},\Diamond)$ sends backdoor "$(\mathsf{m},(\mu,\mathsf{id}))$" to the environment $(\mathcal{E},0)$.

$\mathcal{D}$ is a PPT adversary, as its execution time for each activation is linear in the size of the received message.

**Universal Composition Operator (UC Operator).** The UC operator $\rho^{\phi\to\pi}$ is defined in [Can20, (Section 6.1)] and relies on the notion of *structured protocol* [Can20, (Section 5.1)]. A structured protocol is an ITM whose functioning can be divided into two parts: a body, which is an ITI, and a shell that simulates the body and can modify the external-writes it receives or intends to send. In the following figure, a structured ITI $N$ is shown with a body $M$. We observe that $N$ can modify both the external-writes that $M$ receives and those that $M$ intends to send. The shell is responsible for simulating the body and applying these modifications.




In a nutshell, the ITIs of an extended session $\mathsf{Ext}(\rho^{\phi\to\pi},\mathsf{s})$ are structured ITIs whose bodies correspond to ITIs from the extended session $\mathsf{Ext}(\rho,\mathsf{s})$ or from extended sessions of $\pi$. The shells of these ITIs are used to replace codes in certain sent and received

[25] This is justified by a weakness described in the extended version [CS25, Appendix D.6].

external-write operations, in such a way that the execution of $\rho^{\phi\to\pi}$ corresponds to an execution of $\rho$ in messages to sub-parties with code $\pi$ are replaced by messages to ITIs with the same identity but code $\phi$.

We now provide a more detailed explanation. To simplify it, we make the following three assumptions. First, no execution of $\rho$ invokes a main party of $\pi$. Second, for any sids $\mathsf{s} \neq \mathsf{s}'$ and $\mu \in \{\phi, \pi\}$, $\mathsf{ExtSub}(\mu, \mathsf{s}) \cap \mathsf{ExtSub}(\mu, \mathsf{s}') = \emptyset$. Third, for any sids $\mathsf{s}, \mathsf{s}'$, $\mathsf{Ext}(\phi, \mathsf{s}) \cap \mathsf{Ext}(\pi, \mathsf{s}') = \emptyset$.

To describe the rules of replacement, we call an ITI *outside* if it does not belong to any extended session of $\pi$. The rules are as follows:

- *Replacements $\phi \to \pi$:* Inputs or outputs from an outside ITI to a main party of an extended session $\mathsf{Ext}(\phi, \mathsf{s})$ are redirected to the main party of $\mathsf{Ext}(\pi, \mathsf{s})$ with the same pid.
- *Replacements $\pi \to \phi$:* When a main party of an extended session $\mathsf{Ext}(\pi, \mathsf{s})$ sends an output to an ITI $M$ outside this extended session, the external-write is modified so that the sender code, originally $\pi$, is replaced by $\phi$.

To apply these replacements, the shells store information through local variables and communicate with other shells. More precisely, any message sent by a structured protocol to another one can be seen as a couple $(\mathsf{m}_{\mathsf{body}}, \mathsf{m}_{\mathsf{shell}})$ where $\mathsf{m}_{\mathsf{body}}$ is aimed to the body of the recipient (we call it a body message) and $\mathsf{m}_{\mathsf{shell}}$ to its shell (we call it a shell message). The only aim of the shell messages is to communicate enough information to ensure good application of the replacements detailed above.

Finally, note that when applied to parametrized systems $\boldsymbol{\rho}, \boldsymbol{\phi}$ and $\boldsymbol{\pi}$, $\boldsymbol{\rho}^{\boldsymbol{\phi}\to\boldsymbol{\pi}}$ is itself a parametrized system with $\left(\rho^{\phi\to\pi}\right)_k = \rho_k^{\phi_k\to\pi_k}$.

**UC Theorem.** The UC Theorem [Can20, (Theorem 22)] states that, under certain hypotheses on PPT parametrized systems $\boldsymbol{\rho}$, $\boldsymbol{\phi}$, $\boldsymbol{\pi}$ and on a predicate $\xi$ on extended identities, if $\boldsymbol{\pi}$ $\xi$-UC-emulates $\boldsymbol{\phi}$, then $\boldsymbol{\rho}^{\boldsymbol{\phi}\to\boldsymbol{\pi}}$ UC-emulates $\boldsymbol{\rho}$.

Concretely, they need to satisfy some conditions : (1) $\boldsymbol{\phi}$ and $\boldsymbol{\pi}$ are both subroutine respecting [Can20, (Definition 19)] and subroutine exposing [Can20, (Definition 21)], and (2) $\boldsymbol{\rho}$ is $(\boldsymbol{\pi}, \boldsymbol{\phi}, \xi)$-compliant [Can20, (Section 6.1)]. The subroutine respecting, subroutine exposing, and compliance notions are respectively linked to the problems of the UC Theorem described in sections 5.1, 5.3 and 5.4. They are detailed within those sections.

# 5 Problems Within The Universal Composition Theorem

This section provides a detailed account of the issues, limitations, and weaknesses in the UC theorem of Plain UC [Can20, (Theorem 22)] listed in Table 1.

While these points help in understanding the changes introduced by Updated UC, they are also of independent interest.

## 5.1 Problems Related to Extended Sessions and Subroutine Respecting Property

We begin by reviewing the concept of subroutine respecting protocols.

A protocol $\mu$ is said *subroutine respecting* [Can20, (Definition 19)] if, in any prefix of execution of any protocol $\alpha$ and for any sid $\mathsf{s} \in \{0,1\}^*$, certain restrictions apply to communications with the extended session $\mathsf{Ext}(\mu, \mathsf{s})$ – we emphasize that we are not restricted to executions of $\mu$ itself, but rather consider how $\mu$ behaves within the execution of any protocol $\alpha$. For the problems we are addressing, we will focus on these four restrictions: (1) the main parties of $\mathsf{Ext}(\mu, \mathsf{s})$ reject any output sent by an ITI outside $\mathsf{Ext}(\mu, \mathsf{s})$; (2) the sub-parties of $\mathsf{Ext}(\mu, \mathsf{s})$ reject any input or output sent by an ITI outside

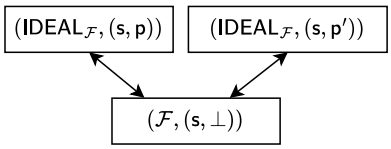



**Figure 7:** Illustration of an ideal protocol.

$\mathsf{Ext}(\mu, \mathsf{s})$; (3) no member of $\mathsf{Ext}(\mu, \mathsf{s})$ sends an input to an existing ITI that is not already a member of $\mathsf{Ext}(\mu, \mathsf{s})$; and (4) no sub-party of $\mathsf{Ext}(\mu, \mathsf{s})$ sends an output to an existing ITI that is not already a member of $\mathsf{Ext}(\mu, \mathsf{s})$. Moreover, neither of the required conditions impose any restriction on received or sent backdoors.

**(L.1) Extended Session Definition Excludes Too Many ITIs.** The notions of extended session and of subroutine respecting protocol are overly restrictive.

Looking at the definition of an extended session recalled in Section 4, page 17, we observe that an ITI which is neither a main party nor invoked by a member of the extended session will never become part of that session, even if it rejects the message that invoked it. We will show that this property implies that a broad class of protocols that should be subroutine respecting are, in fact, not subroutine respecting. This class consists of:

- most of the ideal protocols associated with ideal functionalities [Can20, Section 5.3];
- certain protocols constructed according to the method described in [Can20, Section 5.2, Paragraph "Writing subroutine respecting protocols"], whose stated goal was precisely to ensure that the resulting protocols are subroutine respecting.

We demonstrate this here for the case of ideal protocols; the second case is addressed in Appendix A.1.1. First, let's recall the necessary notions about the ideal protocols. For any ITM $\mathcal{F}$, we associate a protocol $\mathsf{IDEAL}_\mathcal{F}$, referred to as the ideal protocol of $\mathcal{F}$ (in this context, $\mathcal{F}$ is called an ideal functionality). By definition of the ideal protocol $\mathsf{IDEAL}_\mathcal{F}$ [Can20, Section 5.3], for any $\mathsf{s} \in \{0,1\}^*$, the ITIs that may appear in an extended session $(\mathsf{IDEAL}_\mathcal{F}, \mathsf{s})$ are of two types:

- The main parties of $(\mathsf{IDEAL}_\mathcal{F}, \mathsf{s})$, which act as relays between the environment and the ideal functionality $(\mathcal{F}, (\mathsf{s}, \bot))$. In particular, when a main party $(\mathsf{IDEAL}_\mathcal{F}, (\mathsf{s}, \mathsf{p}))$ receives an input "$m$" from an ITI $M$, they send the input "$(\mathsf{m}, M)$" to $(\mathcal{F}, (\mathsf{s}, \bot))$ ([Can20, Item (1) of Section 5.3]).
- The ideal functionality ITI $(\mathcal{F}, (\mathsf{s}, \bot))$, and its sub-parties.

This is illustrated in Figure 7.

Finally, we make a mild assumption on $\mathcal{F}$, which is satisfied by most ideal functionalities studied in the literature: there exist a message $\tilde{\mathsf{m}} \in \{0,1\}^*$ and a session identifier $\tilde{\mathsf{s}} \in \{0,1\}^*$ such that $(\mathcal{F}, (\tilde{\mathsf{s}}, \bot))$ rejects when invoked with input message $\tilde{\mathsf{m}}$. Indeed, the inputs that such functionalities accept generally need to follow a specific format, which makes it easy to find such a message $\tilde{\mathsf{m}}$.

**Proposition 1.** $\mathsf{IDEAL}_\mathcal{F}$ *is not subroutine respecting.*

*Proof.* Consider an ITI $(\rho, (\tilde{\mathsf{s}}, 0)), \rho \notin \{\mathsf{IDEAL}_\mathcal{F}, \mathcal{F}\}$, which, upon its first activation, sends the input "$\tilde{\mathsf{m}}$" to $(\mathcal{F}, (\tilde{\mathsf{s}}, \bot))$, and upon its second activation, sends the input "0" to $(\mathsf{IDEAL}_\mathcal{F}, (\tilde{\mathsf{s}}, 0))$. Next, consider an environment $\mathcal{E}$ which, at its first activation, sends the input "0" to the protocol with identifier $(\tilde{\mathsf{s}}, 0)$, and at its second activation, again sends the input "0" to the protocol with identifier $(\tilde{\mathsf{s}}, 0)$. Let $\theta$ be a prefix of the execution of $\rho$ with environment $\mathcal{E}$, some adversary, and some input.

We first note that $(\mathcal{F}, (\tilde{\mathsf{s}}, \bot)) \notin \mathsf{Ext}(\mathsf{IDEAL}_\mathcal{F}, \tilde{\mathsf{s}}, \theta)$. This follows directly from the definition of extended sessions, since $(\mathcal{F}, (\tilde{\mathsf{s}}, \bot))$ is invoked by an input from $(\rho, (\tilde{\mathsf{s}}, 0))$, which itself does not belong to $\mathsf{Ext}(\mathsf{IDEAL}_\mathcal{F}, \tilde{\mathsf{s}}, \theta)$.

We now use this observation to show that $\mathsf{IDEAL}_{\mathcal{F}}$ violates one of the conditions required for a protocol to be subroutine respecting.

Let us analyze the execution more closely. After being invoked by $(\rho, (\tilde{\mathsf{s}}, 0))$, $(\mathcal{F}, (\tilde{\mathsf{s}}, \perp))$ rejects, since it is invoked by an input with message $\tilde{\mathsf{m}}$ (by the assumption made on $\mathcal{F}$). We now describe the subsequent sequence of activations and messages:

- The environment $(\mathcal{E}, 0)$ is activated again, without receiving any message.
- Upon activation, the environment $(\mathcal{E}, 0)$ sends input "$\mathsf{0}$" to $(\rho, (\tilde{\mathsf{s}}, 0))$.
- In response, $(\rho, (\tilde{\mathsf{s}}, 0))$ invokes $(\mathsf{IDEAL}_{\mathcal{F}}, (\tilde{\mathsf{s}}, 0))$ with input "$\mathsf{0}$".
- Then, $(\mathsf{IDEAL}_{\mathcal{F}}, (\tilde{\mathsf{s}}, 0))$ sends input "$(\mathsf{0}, (\rho, (\tilde{\mathsf{s}}, 0)))$" to $(\mathcal{F}, (\tilde{\mathsf{s}}, \perp))$.

This final input demonstrates that $\mathsf{IDEAL}_{\mathcal{F}}$ is not subroutine respecting, since a main party of $\mathsf{Ext}(\mathsf{IDEAL}_{\mathcal{F}}, \tilde{\mathsf{s}}, \theta)$ sends an input to an existing ITI that does not belong to $\mathsf{Ext}(\mathsf{IDEAL}_{\mathcal{F}}, \tilde{\mathsf{s}}, , \theta)$, thereby violating condition (3) of the subroutine respecting property. □

To address this issue, Updated UC adopts an updated definition of the extended session (Section 6.1). The main change in this definition is that the fact that an ITI has rejected all messages it has received since its invocation no longer prevents it from later belonging to an extended session. As a result, Proposition 1 does not apply under this updated definition.

Note that this issue is not directly related to the other problems discussed in this section (Section 5.1). In particular, addressing it by adopting the updated definition of extended sessions from Updated UC does not resolve the other concerns identified in this section.

**(I.1) Subroutine Respecting Property and Backdoors.**

We recall that in the execution of a protocol, the adversary cannot invoke ITIs. Nevertheless, once an ITI has been invoked it becomes accessible to the adversary – even if it rejected all received messages.

Additionally, in the full definition of the subroutine respecting notion [Can20, Definition 19], no explicit condition is imposed on backdoor messages sent to main parties. The only message-related requirements in that definition concern inputs and outputs.

Together, these observations yield the following counterexample to the UC theorem.

We create an ITM $\pi$ such that:

- For any $\mathsf{s}, \mathsf{p}$, $(\pi, (\mathsf{s}, \mathsf{p}))$ always rejects any output it receives.
- For any $(\mathsf{s}, \mathsf{p}) \neq (0, 0)$, $(\pi, (\mathsf{s}, \mathsf{p}))$ always rejects any message it receives (input, output or backdoor).
- At reception of its first input, $(\pi, (0, 0))$ initializes a variable $\mathsf{ReceivedInput}$, then deactivates and rejects any subsequent input, output or backdoor it receives.
- At reception of a backdoor message, if $\mathsf{ReceivedInput}$ is initialized, $(\pi, (\mathsf{s}, \mathsf{p}))$ rejects it (as already implied by the previous item). Else, if $\mathsf{ReceivedInput}$ is uninitialized, $(\pi, (\mathsf{s}, \mathsf{p}))$ sends an output to an ITI with code $\mu \neq \pi$.

It can be readily verified that $\pi$ is subroutine respecting. Indeed, in the execution of any protocol,[26] every ITI with code $\pi$ has no subroutines and rejects all output messages.

Moreover, in any execution of $\pi$ with an adversary and an environment, all main parties of the extended test session are invoked through inputs originating from the environment. In particular, if $(\pi, (0, 0))$ exists in the execution, it must have initialized the variable $\mathsf{ReceivedInput}$ and therefore rejects any subsequent input, output, or backdoor message. As a consequence, one can show that $\pi$ UC-emulates the protocol $\phi$ that rejects any incoming message.

However, we can straightforwardly construct a protocol $\rho$ such that $\rho^{\phi \to \pi}$ does not UC-emulate $\rho$. This can be achieved by exploiting the fact that $\rho$ may invoke $(\phi, (0, 0))$

[26] Recall that the conditions ensuring that a protocol $\pi$ is subroutine respecting must hold not only in executions of $\pi$ itself, but also in executions of any protocol $\rho$ in which $\pi$ may appear as a sub-party.

through an output message, together with the behavior of $(\pi, (0,0))$ upon receiving a backdoor message when $\mathsf{ReceivedInput}$ is uninitialized.

Although we believe this issue does not affect real-world protocols, it highlights the need to refine the notion of subroutine respecting protocols. This refinement is introduced in Section 6.5.

**(I.2) The Case of "Dynamic Sub-parties".** We introduce two concepts to explain this issue. An ITI $M$ is said to be *awake* in an execution prefix if it has been invoked and has not rejected all received messages. Additionally, we say that an ITI $M$ is a *dynamic sub-party* of a protocol $\phi$ if there is a sid $\mathsf{s}$ and two execution prefixes $\theta_{\mathsf{in}}, \theta_{\mathsf{out}}$ such that $M \in \mathsf{Ext}(\phi, \mathsf{s}, \theta_{\mathsf{in}})$, but $M \notin \mathsf{Ext}(\phi, \mathsf{s}, \theta_{\mathsf{out}})$ and $M$ is awake in $\theta_{\mathsf{out}}$.

Dynamic sub-parties do not align well with Plain UC because, during the execution of $\phi$ with an environment and adversary, the execution rules indicate that if $M$ exists, it will always be a sub-party of the extended test session. Consequently, these executions do not consider scenarios where $M$ is awake but outside the extended session. In particular, some protocols with dynamic sub-parties could lead to counterexamples of the UC Theorem.

The main difficulty in constructing a protocol with dynamic sub-parties that can be used to contradict the UC Theorem lies in finding one that is subroutine respecting. Intuitively, one might expect this property to exclude any protocol with dynamic sub-parties, given the multiple constraints it imposes on the sub-parties of a protocol. However, the definition of a dynamic sub-party is not inherently incompatible with the requirements of a subroutine respecting protocol. For a protocol $\phi$ with a dynamic sub-party $M$ to be subroutine respecting, the following must hold:

- $M$ satisfies all requirements imposed on sub-parties in every execution of any protocol in which $M$ appears as a sub-party of $(\phi, \mathsf{s})$ for some $\mathsf{s}$;
- $M$ is not required to satisfy these requirements in executions of protocols in which $M$ does not appear as a sub-party of $(\phi, \mathsf{s})$ for any $\mathsf{s}$.

We present such a protocol $\phi$ in Figure 8. We claim that it is a subroutine respecting protocol with a dynamic sub-party $(\alpha, (0,0))$. Specifically:

- $(\alpha, (0,0))$ is a sub-party of $\mathsf{Ext}(\phi, 0)$ if it is invoked with an input by $(\phi, (0,0))$.
- $(\alpha, (0,0))$ is awake and not a sub-party of $\mathsf{Ext}(\phi, 0)$ if it is invoked with an input by another ITI. Moreover, it is never a sub-party of $\mathsf{Ext}(\phi, \mathsf{s})$ for $\mathsf{s} \neq 0$.

We explain in this sidebar the key reason why $\phi$ is subroutine respecting.

> We fix the execution of any protocol[27]. In this execution, note that if $(\alpha, (0,0))$ is not invoked by an input from $(\phi, (0,0))$, it sends an input to $(\phi, (0,0))$. This input can be interpreted as a "warning" to inform $(\phi, (0,0))$ that $(\alpha, (0,0))$ is awake and does not belong to $\mathsf{Ext}(\phi, 0)$. Once $(\phi, (0,0))$ is "warned", it refrains from sending any input to $(\alpha, (0,0))$, thereby maintaining the conditions of the subroutine respecting property[28].

In Appendix A.2, we show that there exists a protocol $\pi$ that UC-emulates a slight modification of $\phi$, and a protocol $\rho$ such that all the conditions of the UC theorem are satisfied. However, $\rho^{\phi \to \pi}$ does not UC-emulate $\rho$, thereby contradicting the UC theorem.

The core reason for the counterexample lies in the difference in the observable behavior[29] of $\phi$ when executed alone versus as a subroutine within another protocol. We detail it in the following sidebar.

[26] Recall that the conditions ensuring that a protocol $\phi$ is subroutine respecting must hold not only in executions of $\phi$ itself, but also in executions of any protocol in which $\phi$ may appear as a sub-party.

[27] Note that a complete proof (for a slight modification of $\phi$) is provided in Appendix A.2.

[29] *i.e.*, observable by the environment.

$\rho$ is a protocol distinct of $\alpha$ and $\phi$.
Moreover, all external writes are made with flags reveal-ext-id and forced-write set.

$(\phi, \mathsf{id})$

**local variable:** boolean FirstInputMade, initialized to false
**if** id $\neq (0,0)$ **then reject**
At reception of an input $m$ from ITI $M$ :
**if** FirstInputMade is false **then**
  FirstInputMade $\leftarrow$ true
  **if** $M \neq (\alpha, (0,0))$
    $\wedge$ first bit of randomness is 0 **then**
      **send input** "0" to $(\alpha, (0,0))$
  **else**
      **send output** "0" to $(\rho, (0,0))$
**send output** "0" to $(\rho, (0,0))$
At reception of output $m$ from ITI M :
**if** $M = (\alpha, (0,0))$ **then**
  **send output** "0" to $(\rho, (0,0))$
**else reject**
At reception of backdoor : **reject**

$(\alpha, \mathsf{id})$

**local variable:** boolean FirstInputMade, initialized to false
**if** id $\neq (0,0)$ **then reject**
At reception of input $m$ from ITI $M$ :
**if** FirstInputMade is false **then**
  FirstInputMade $\leftarrow$ true
  **if** $M = (\phi, (0,0))$ **then**
    **send output** "0" to $(\phi, (0,0))$
  **else**
    **send input** "0" to $(\phi, (0,0))$
**else reject**
At reception of backdoor : **reject**
At reception of output : **reject**

**Figure 8:** Algorithms of $\phi$ with dynamic sub-party $\alpha$, for Issue (I.2).

In any execution of $\phi$ with test sid 0, $(\phi, (0,0))$ behaves essentially like a protocol that sends an output to $(\rho, (0,0))$ upon receiving any input – regardless of whether it creates the subroutine $(\alpha, (0,0))$. In this context, the creation of the subroutine has no observable effect.

However, this changes when $\phi$ is used as a subroutine by a protocol. A new observable behavior arises. Suppose, for example, that $(\rho, (0,0))$ first sends an input to $(\phi, (0,0))$, and then sends an output to $(\alpha, (0,0))$. The result of this second input, sent to $(\alpha, (0,0))$, depends on the first bit of randomness used by $\phi$:

- If it is 0, then $(\alpha, (0,0))$ has already been invoked by $(\phi, (0,0))$. As a consequence, $(\alpha, (0,0))$ rejects the input.
- If it is 1, then this message actually invokes $(\alpha, (0,0))$. Because $(\alpha, (0,0))$ is invoked by an ITI distinct from $(\phi, (0,0))$ – namely, $(\rho, (0,0))$ – it subsequently sends an input to $(\phi, (0,0))$, which in turn sends an output to $(\rho, (0,0))$.

To address this issue, Updated UC's variant of the subroutine respecting property (Section 6.1) excludes protocols with dynamic sub-parties. This is achieved through the introduction of the Panoptic Session concept, described in Section 6.3.

**(I.3) Subroutine Respecting Property and Universal Composition Operator.** This issue is particularly subtle, and we will only provide a brief explanation of the mechanism it exploits, with a full counterexample of the UC theorem presented in Appendix A.3.

When considering the UC theorem for protocols $\rho, \phi, \pi$, an important hypothesis is that $\pi$ is subroutine respecting. This property gives restrictions to the ITIs with code $\pi$ and its sub-parties in any protocol execution.

In demonstrating the UC theorem, it's crucial to verify these conditions within the

execution of $\rho^{\phi \to \pi}$. However, such executions use structured protocols to simulate special kind of executions of $\rho$, where instances of $\phi$ are substituted with instances of $\pi$. In these executions, we can demonstrate that the simulated extended sessions of $\pi$ may encounter situations that do not arise in any real extended sessions of $\pi$ during any protocol execution. By definition, the subroutine respecting property of $\pi$ does not cover such cases, and this absence of constraints can be exploited to construct a counterexample to the UC theorem.

While we did not find a concrete protocol with this issue, it is crucial to address it to ensure the correctness of the UC Theorem.

In Updated UC, the notion of sub-party respecting (Section 6.5) addresses this issue by requiring that its restrictions hold not only for any protocol execution but also for any extended execution.

## 5.2 Problems Related to the Universal Composition Operator

In this section, we present an issue and a weakness linked to the UC operator [Can20, (Section 6.1, item 2)].

The issue highlights the necessity of replacing the UC operator, while the weakness underscores the value of having a modular formulation of the UC theorem that does not rely on a specific UC Operator implementation. These considerations are further discussed for Updated UC in Section 6.7.

**(I.4) An Issue in Handling Outputs Forwarded to the Environment.**

In this section, we examine a subtle mechanism required by the UC operator that was omitted from the overview for simplicity (see Section 4, page 18). We show that the UC operator in Plain UC does not implement this mechanism correctly, which we exploit to construct a counterexample to the UC theorem.

Recall that, in the execution of a protocol $\rho$ with environment $\mathcal{E}$ and some adversary, if a main party $(\rho, (\mathsf{s}_{\mathsf{test}}, \mathsf{p}))$ of the extended test session sends an output to an ITI $M$ that does not exist in the execution, the output is instead sent to $(\mathcal{E}, 0)$. Additionally, the identity $(\mathsf{s}_{\mathsf{test}}, \mathsf{p})$ and the extended identity of $M$ are also forwarded to $(\mathcal{E}, 0)$. Recall further that the ITIs of an extended session $\mathsf{Ext}(\rho^{\phi \to \pi}, \mathsf{s})$ are structured ITIs whose bodies correspond either to ITIs of $\mathsf{Ext}(\rho, \mathsf{s})$ or of extended sessions of $\pi$.

As a result, when the body of a structured ITI attempts to send such an output to an ITI $M$, it is crucial for the shell to determine whether a structured ITI $S$ with body $M$ exists in the execution. If such a structured ITI $S$ exists, the message must be directed to $S$. Otherwise, the recipient must remain unmodified, as the output will be forwarded to $(\mathcal{E}, 0)$ along with the identity of the intended recipient. In this latter case, using $S$ as the recipient instead of $M$ would allow the environment to distinguish between executions of $\rho^{\phi \to \pi}$ and $\rho$.

However, the definition of $\rho^{\phi \to \pi}$ does not address this situation correctly. In an execution of $\rho^{\phi \to \pi}$ with test sid $\mathsf{s}_{\mathsf{test}}$, when the body of a main party of $\mathsf{Ext}\left(\rho^{\phi \to \pi}, \mathsf{s}_{\mathsf{test}}\right)$ attempts to send an output to an ITI $(\alpha, (\mathsf{s}', \mathsf{p}'))$, the procedure described in [Can20, (Section 6.1, item 2)] is as follows:

- If $(\alpha, \mathsf{s}') \neq (\rho, \mathsf{s}_{\mathsf{test}})$, then $\left(\rho^{\phi \to \pi}, (\mathsf{s}_{\mathsf{test}}, \mathsf{p})\right)$ sends the output to $(\alpha, (\mathsf{s}', \mathsf{p}'))$.
- Otherwise, if $(\alpha, \mathsf{s}') = (\rho, \mathsf{s}_{\mathsf{test}})$, then the output is sent to $\left(\rho^{\phi \to \pi}, (\mathsf{s}_{\mathsf{test}}, \mathsf{p}')\right)$ – *i.e.*, the structured ITI with body $(\rho, (\mathsf{s}_{\mathsf{test}}, \mathsf{p}'))$.

Consequently, $\left(\rho^{\phi \to \pi}, (\mathsf{s}_{\mathsf{test}}, \mathsf{p})\right)$ behaves as if, whenever $(\alpha, \mathsf{s}') \neq (\rho, \mathsf{s}_{\mathsf{test}})$, no structured ITI with body $(\alpha, (\mathsf{s}', \mathsf{p}'))$ exists. This assumption is incorrect and leads to counterexamples to the UC theorem, as detailed in Appendix A.4.

To correctly select the recipient in this situation, $\left(\rho^{\phi \to \pi}, (\mathsf{s}_{\mathsf{test}}, \mathsf{p})\right)$ must be able to determine whether a structured ITI with body $M$ is present in the execution. To enable this, the UC operator proposed in Updated UC uses a centralized approach: it employs a

special ITI that tracks exchanged messages and maintains global awareness of instantiated structured protocols.

**(W.1) Proof of UC theorem and structured ITIs.**

In the proof of the UC Theorem [Can20, (Theorem 22)] for protocols $\rho$, $\phi$, and $\pi$, the objective is to establish the existence of a PPT adversary $\mathcal{S}$ such that the following relation [Can20, (Equation 2, Section 6.1)] holds for any PPT environment $\mathcal{E}$: $\mathsf{EXEC}_{\rho^{\phi\to\pi},\mathcal{D},\mathcal{E}} \approx \mathsf{EXEC}_{\rho,\mathcal{S},\mathcal{E}}$.

In this section, we briefly highlight a subtle flaw in the definition of $\mathcal{S}$ [Can20, Figure 10], which renders the proof of the UC Theorem incorrect. A full description of the problem and its resolution is provided in Appendix A.5. However, the precise description of it is not our main focus here. What matters is that both the problem and its resolution depend on the specific construction of the UC operator $\rho^{\phi\to\pi}$ – in particular, on the fact that $\rho^{\phi\to\pi}$ is a structured protocol and on its shell.

We argue that the existence of this problem stems from the inherent complexity of the proof, which simultaneously involves several subtle notions, including the internal structure of the UC operator. We believe it would be advantageous to formulate the UC Theorem in a way that does not rely on a specific operator construction. Such a formulation offers two key benefits: it simplifies the analysis of the proof and makes it easier to accommodate alternative operator constructions. This is precisely the approach adopted in Updated UC: we define the UC operator abstractly, by highlighting the conditions it must satisfy, rather than relying on a specific instantiation.

We now briefly describe the flaw. In essence, it arises when the simulator $\mathcal{S}$ receives an input of the form $(m, M)$—where $m$ is a message and $M$ an ITI—and performs certain operations and checks directly on $M$ (see Items 1.(a), 1.(b), and 1.(c) in [Can20, Figure 10]). These steps should instead be handled as follows. First, $\mathcal{S}$ should reject whenever $M$ is not a structured protocol sharing the same shell as $\rho^{\phi\to\pi}$. Second, if this condition holds and $M$ indeed represents such a structured protocol, the corresponding operations and verifications must be performed on the body of $M$, rather than on $M$ itself.

As shown in Appendix A.5, without this correction, the proof of the UC Theorem does not hold. That appendix also provides a detailed account of the required modifications.

## 5.3 Problems Related to the Subroutine Exposing Property

This section presents a limitation and a weakness related to the *subroutine exposing property* ([Can20, (Definition 21)]), which is required for $\phi$ and $\pi$ to apply the UC theorem.

To ease the reading, we recall the definition of a subroutine exposing protocol as provided in the technical overview (Section 2.4). For a protocol $\alpha$, the subroutine exposing property requires that for each sid $\mathsf{s}$, one of the ITIs in the extended session $\mathsf{Ext}(\alpha, \mathsf{s})$ can inform – upon request – the adversary whether an ITI $M$ belongs to the extended session $\mathsf{Ext}(\alpha, \mathsf{s})$ at the time of the request. We refer to this ITI as the exposing ITI of the extended session $\mathsf{Ext}(\alpha, \mathsf{s})$, or simply the exposing ITI when the context is clear. It is also required that the extended identity of the exposing ITI for the extended session $\mathsf{Ext}(\alpha, \mathsf{s})$ can be determined using only $\alpha$ and $\mathsf{s}$.[30]

**(L.2) Adding the Subroutine Exposing Property to a Protocol Does Not Preserve UC-Emulation.**

To improve readability, we restate here part of the explanation previously given in the technical overview (Section 2.4.1). Nevertheless, we encourage the reader to refer to that section for the full context of this limitation.

There is a straightforward mechanism, described in [Can20, (Section 6.1)], that transforms any protocol $\phi$ into a subroutine exposing protocol $\phi_{\mathsf{SE}}$. In summary, $\phi_{\mathsf{SE}}$ introduces,

[30] The exposing ITI is sometimes called a "directory ITI" in Plain UC.

All external writes are made with flags reveal-ext-id and forced-write set.

| $(\pi, \mathsf{id})$ | $(\alpha, \mathsf{id})$ |
|---|---|
| **if** $\mathsf{id} \neq (0,0)$ **then reject** | **reject** |
| At reception of an output or a backdoor : **reject** | |
| At reception of an input $\mathsf{m}$ from an ITI $M$ : | $(\phi, \mathsf{id})$ |
| **if** it is first activation $\wedge\ m = \mathsf{0}$ **then send input** "0" to $(\alpha, (0,0))$ | **reject** |
| **else reject** | |

**Figure 9:** Algorithms of $\pi, \alpha$ and $\phi$ for Limitation (L.2).

in any extended session $\mathsf{Ext}(\phi_{\mathsf{SE}}, \mathsf{s})$, a middle-man $M_{\phi,\mathsf{s}}$ that is aware of all ITIs belonging to the extended session and can therefore act as an exposing ITI, telling the adversary whether any ITI $N$ belongs to $\mathsf{Ext}(\phi_{\mathsf{SE}}, \mathsf{s})$.

To the best of our knowledge, most protocols in the UC framework are not subroutine exposing. We substantiate this claim in Appendix A.6 by exhibiting several ideal protocols that lack this property, including the ideal functionalities $\mathcal{F}_{\mathsf{AUTH}}$, $\mathcal{F}^{l}_{\mathsf{SMT}}$, $\mathcal{F}_{\mathsf{SYN}}$ and $\mathcal{F}_{\mathsf{NC}}$ of [Can20]. Consequently, applying the transformation is essential to endow them with it and to ensure they benefit from the security guarantees of the UC theorem.

For the transformation to be sound, it must preserve UC-emulation: whenever $\pi$ UC-emulates $\phi$, we should also have that $\pi_{\mathsf{SE}}$ UC-emulates $\phi_{\mathsf{SE}}$. However, this preservation is not guaranteed. We demonstrate this with a simple, intentionally constructed counterexample.

Let $\pi$ and $\phi$ be the protocols described in Figure 9. Since neither $\pi$ nor $\phi$ produces any output or interacts with the adversary, we know that $\pi$ UC-emulates $\phi$. Furthermore, both protocols are subroutine respecting. As a result, the only missing property for applying the UC theorem is subroutine exposing. Nevertheless, $\pi_{\mathsf{SE}}$ does not UC-emulate $\phi_{\mathsf{SE}}$. To see this, observe that $(\alpha, (0,0))$ belongs to $\mathsf{Ext}(\pi, 0)$ if and only if the environment invoked $(\pi, (0,0))$ with input "0". This implies that an adversary of $\pi_{\mathsf{SE}}$ can use the subroutine exposing ITI to determine whether the environment invoked the protocol with identifier $(0,0)$ using input "0". In contrast, an adversary of $\phi_{\mathsf{SE}}$ lacks such a mechanism.

This problem can arise in real-world protocols, where the existence of an ITI depends on an action taken by the environment. An example is given in the technical overview (Section 2.4).

The implications of this limitation are twofold. First, it shows that the subroutine-exposing property must be explicitly verified in protocols analyzed under Plain UC, since the generic transformation does not preserve UC-emulation – a verification that, to the best of our knowledge, is generally omitted. Second, it reveals that certain realistic examples of UC-emulation cannot be directly leveraged within the UC Theorem.

In Updated UC, we adopt a variant of subroutine exposing (Section 6.4) that eliminates the need for such a transformation and enables the analysis of more protocols, including the example from the technical overview (Section 2.4).

**(W.2) Determining the sids of Existing Sessions of $\phi$.** In the proof of UC theorem [Can20, (Theorem 22)] for protocols $\rho, \phi$, and $\pi$,[31] the author constructs a PPT adversary $\mathcal{S}$ [Can20, (Figure 10)] of $\rho$ that behaves differently when receiving a backdoor from an ITI $M$, depending on whether there exists a sid $\mathsf{s}$ such that $M \in \mathsf{Ext}(\phi, \mathsf{s})$.

$\mathcal{S}$ relies on the subroutine exposing property of $\phi$ to determine if an ITI belongs to an extended session of $\phi$ during the execution of $\rho$ (see [Can20, (2.(a) of Figure 10)]). Since

[31] In discussing this weakness, we simplify the explanation by omitting the concept of top-level sessions [Can20, (Section 6.1)], which is not essential for understanding and will not be required for the rest of the document.

using the exposing ITI of an extended session $\mathsf{Ext}(\phi, \mathsf{s})$ generally requires knowing $\mathsf{s}$, $\mathcal{S}$ must know the sids of the existing extended sessions of $\phi$ in the current execution of $\rho$. However, no mechanism is provided to identify them. This is critical because the number of possible sids could be exponential, meaning that the adversary would not be PPT if required to test all possible sids.

For instance, consider a protocol $\rho$ that, upon activation, samples a random $\mathsf{s} \in \{0,1\}^k$ and then invokes $(\phi, (\mathsf{s}, 0))$: the number of possible sids is exponential in the security parameter k.

We believe than this weakness cannot be solved in Plain UC without additional properties. A direct solution could be to only consider protocols $\rho$ that informs the adversary of these sids. Nevertheless, we think this would overly restrict the applicability of the UC theorem.

In Updated UC, this weakness does not exist due to our use of a variant of the subroutine exposing property, described in Section 6.4.

## 5.4 Problem Related to the Compliance Property

This section presents a weakness related to the *compliance* property ([Can20, (Section 6.1)]), which is required to apply the UC theorem. It highlights that this notion is not restrictive enough.

Compliance is the sole requirement concerning the protocol $\rho$ in the UC Theorem, aside from being PPT. Simply put, for protocols $\phi$ and $\pi$, and $\xi$ a predicate on extended identities, $\rho$ is $(\phi, \pi, \xi)$-compliant [Can20, (Section 6.1)] if, in any execution[32] of $\rho$: (1) some ITIs satisfy $\xi$; and (2) if two external-write made by protocol ITIs have target codes $\pi$ and $\phi$, they have distinct target sids.

**(W.3) Identifying Whether $\pi$ is the Result of a Replacement.**

In the proof of the UC theorem [Can20, (Theorem 22)] for protocols $\rho$, $\phi$, and $\pi$,[33] the author constructs a PPT adversary $\mathcal{S}$ [Can20, (Figure 10)] for $\rho$ that can behave in two ways upon receiving an input $(m, M)$ from the environment $(\mathcal{E}, 0)$: either as the dummy adversary or as a copy of an adversary $\mathcal{S}_\pi$ of $\phi$ such that $\mathsf{EXEC}_{\pi,\mathcal{D},\mathcal{E}} \approx \mathsf{EXEC}_{\phi,\mathcal{S}_\pi,\mathcal{E}}$.

The compliance notion does not prohibit $\rho$ from invoking ITMs with code $\pi$. In this setting, an ITI $M$ with body $(\pi, (\mathsf{s}, \mathsf{p}))$ appearing in an extended session of $\rho^{\phi \to \pi}$ can be classified into two types:

- Type 1: $M$ was invoked as the result of a replacement $\phi \to \pi$,
- Type 2: $M$ is not the result of a replacement.

We explain in the next paragraph that the existence of these two types leads to a problem in the proof of the UC Theorem.

As a consequence, in Updated UC we propose updating the compliance notion to exclude protocols $\rho$ that can invoke main parties of $\pi$, thereby preventing the occurrence of Type 2 ITIs. As a result, only Type 1 ITIs may appear, which avoids the problem. This restriction is explicitly enforced in the compliance definition introduced in Section 6.6.

**Detailed explanation of the problem.** Consider two protocols $\phi$ and $\pi$ such that $\pi$ UC-emulates $\phi$, with the simulator corresponding to the dummy adversary $\mathcal{D}$ denoted by $\mathcal{S}_\pi$. For simplicity, assume that $\phi$ and $\pi$ never send inputs, and that all outputs they produce are directed towards a protocol $\rho$.

The UC-emulation of $\phi$ by $\pi$ is illustrated in Figure 10 (identifiers omitted for simplicity):

[32] Note that our simplifications of protocol execution (Section 4, page 15) reduces the complexity of this notion, particularly because: (1) $(\mathcal{E}, 0)$ always sets reveal-ext-id, and (2) all protocol ITIs set flag forced-write.

[33] Same as note 31 on page 26.

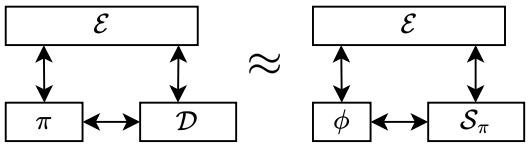



**Figure 10:** UC-emulation of $\phi$ by $\pi$. We omit the identifiers for simplicity.

Furthermore, assume that in any execution of $\rho$, $\rho$ interacts exclusively with a single main party of $\pi$. Under this assumption, in any execution of $\rho^{\phi\to\pi}$, no replacements $\phi \to \pi$ occur.

We further assume that $\phi$, $\pi$, and $\rho$ satisfy all additional conditions required for the UC theorem to apply. The UC-emulation of $\rho$ by $\rho^{\phi\to\pi}$ is depicted in Figure 11, where $\mathcal{S}$ [Can20, (Figure 10)] is the simulator constructed in the proof of the UC theorem.

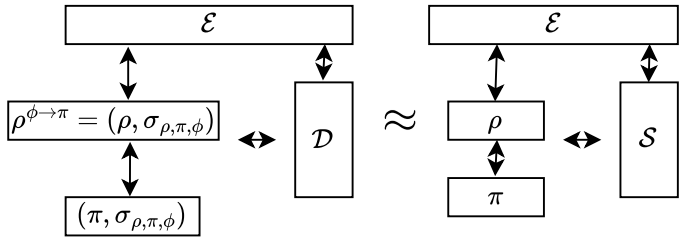



**Figure 11:** UC-emulation of $\rho$ by $\rho^{\phi\to\pi}$, with simulator $\mathcal{S}$. We omit the identifiers for simplicity.

In this situation, Figure 11 clearly illustrates that, for the indistinguishability of Figure 11 to hold, $\mathcal{S}$ should essentially behave like the dummy adversary $\mathcal{D}$. Indeed, the protocols in both executions are essentially identical, except for the presence or absence of shells; in particular, no replacement $\phi \to \pi$ is performed by $\rho^{\phi\to\pi}$.

Specifically, upon receiving an input $(\mathsf{m}, M)$, where $M = \pi$, $\mathcal{S}$ should simply forward a backdoor $\mathsf{m}$ to $\pi$ (we continue to omit identifiers for simplicity).

However, according to the current construction of $\mathcal{S}$ (specifically, Item 1.(b) of [Can20, (Figure 10)]), upon receiving such an input $(\mathsf{m}, M)$, $\mathcal{S}$ can behave as a copy of $\mathcal{S}_\pi$. There is then no guarantee that the behavior of this copy of $\mathcal{S}_\pi$ led to a result indistinguishable from sending the backdoor $\mathsf{m}$ to $\pi$. This discrepancy is precisely where the problem lies: even if subtle, it opens the door to environments that can easily distinguish the left and right executions of Figure 11, thus invalidating the proof of the UC Theorem.

**(W.4) Compliance Property and Universal Composition Operator.**

In Issue (I.3) (page 23), we have shown that the conditions required by the subroutine respecting notion should be verified not only in protocol executions but also in more general forms of execution.

A similar problem arises with the conditions of the compliance notion. We classify this as a weakness rather than an issue, as we have not provided a counterexample that exploits it.

Given the similarity of the two problems, one might logically attempt to address it in a similar manner—by requiring the conditions of the compliance property to hold not only for any protocol execution but also for any extended execution.

However, the conditions required by the subroutine respecting and compliance properties differ significantly, and directly applying this solution would be overly restrictive – making it nearly impossible for a protocol to be compliant.

To update the compliance notion without being overly restrictive, we need a more precise characterization of the set of extended execution that should be considered. To that end, Updated UC introduces a new kind of execution, called replacement execution (see Section 6.2).

# 6 Updated UC: An Update of Plain UC

This section introduces Updated UC, an update to the Plain UC framework that revises several core definitions to address the issues identified in earlier sections.

Each modification in Updated UC will be motivated by specific problems in Plain UC described in the preceding sections.

For clarity of exposition, the universal composition theorem is presented separately in Section 7.

## 6.1 A New Notion of Extended Session

As shown in Limitation (L.1) (Section 5.1), the fact that an ITI rejects a message should not prevent it from belonging to an extended session. In Updated UC, we modify the definition of extended session to account for this.

First, we introduce the notion of a sleeping ITI: an ITI is considered sleeping if it does not exist (*i.e.* it has not been invoked), or if it has always rejected every message it has received. Note that this implies that, upon receiving a message, a sleeping ITI reacts as if it were handling its first message.

Then, when constructing the extended session of a protocol, we no longer focus solely on messages received by newly invoked ITIs, as in the original definition (*i.e.*, ITIs that did not exist before receiving the message). Instead, we inspect each sleeping ITI that receives a message.

**Definition 2** (Sleep)**.** An ITI $M$ is sleeping in a prefix of execution $\theta$ of an extended system if one of the following holds:

- $M$ does not exist in $\theta$ (*i.e.* it has not been invoked).
- $M$ has rejected every message it received – including the most recent one. That is, if the prefix ends with a message being delivered to $M$, we use the full configuration of $M$ (as recorded in $\theta$) to determine whether it rejects the message.

An ITI $M$ is awake if it is not sleeping.

**Definition 3** (Main parties and extended session)**.** Let $\alpha$ be an ITM and $\mathsf{s} \in \{0,1\}^*$ a sid. The main parties of $\mathsf{Ext}(\alpha, \mathsf{s})$ are defined as $\mathsf{MainParty}(\alpha, \mathsf{s}) = \{(\alpha, (\mathsf{s}, \mathsf{p})) : \mathsf{p} \in \{0,1\}^*\}$.

Given a prefix $\theta$ of extended execution, the extended session $\mathsf{Ext}(\alpha, \mathsf{s}, \theta)$ is defined as a set of ITIs initialized [34] to $\mathsf{MainParty}(\alpha, \mathsf{s})$, and constructed inductively by inspecting each input or output received by a sleeping ITI, except the inputs sent by the environment. When a sleeping ITI $M$ receives an input or output with sender an ITI $N$:

- If the identifier of $M$ is not of the form $(\mathsf{s}', \mathsf{p}')$ for some $\mathsf{s}', \mathsf{p}' \in \{0,1\}^*$, nothing is done. In the next items, we assume this condition holds.
- If $M \in \mathsf{MainParty}(\alpha, \mathsf{s})$, then $M$ already belongs to $\mathsf{Ext}(\alpha, \mathsf{s}, \theta)$.
- If it is an input and $N \in \mathsf{Ext}(\alpha, \mathsf{s}, \theta)$, then $M$ is added to $\mathsf{Ext}(\alpha, \mathsf{s}, \theta)$.
- If it is an output and $N \in \mathsf{Ext}(\alpha, \mathsf{s}, \theta) \setminus \mathsf{MainParty}(\alpha, \mathsf{s})$, then $M$ is added to $\mathsf{Ext}(\alpha, \mathsf{s}, \theta)$.
- Otherwise, nothing is done.

We define $\mathsf{ExtSub}(\alpha, \mathsf{s}, \theta) = \mathsf{Ext}(\alpha, \mathsf{s}, \theta) \setminus \mathsf{MainParty}(\alpha, \mathsf{s})$, and call the ITIs in this set the sub-parties.

When the context is clear, we omit $\theta$ and simply write $\mathsf{Ext}(\alpha, \mathsf{s})$ and $\mathsf{ExtSub}(\alpha, \mathsf{s})$.

[34] In the original definition, $\mathsf{MainParty}(\alpha, \mathsf{s}) = \{(\alpha, (\mathsf{s}, \mathsf{p})) : \mathsf{p} \in \{0,1\}^*\}$ is not a subset of $\mathsf{Ext}(\alpha, \mathsf{s})$, since extended identities $(\alpha, (\mathsf{s}, \mathsf{p}))$ that have not been invoked are not regarded as main parties. Our change is a convenient convention.

## 6.2 Replacement Execution: A (Not Entirely) New Notion of Execution

Updated UC introduces a new notion of execution that depends on two protocols $\phi, \pi$ and a number $\ell \in \mathbb{N}$. Simply put, the $(\phi, \ell, \pi)$-execution of a protocol $\rho$ corresponds to executing $\rho$ while replacing the first $\ell$ calls to $\phi$ by calls to $\pi$, and informing the adversary of the sids of the replaced extended sessions.

Although variants of such executions already appear in the proof of the UC theorem [Can20, (Theorem 22)] within Plain UC, they were not used to define any protocol property. However, ignoring such executions leads to problems such as Issue (I.3). and Weakness (W.4). In Updated UC, the replacement executions will be used to update the compliance notion (Section 6.6) and define the notion of UC Operator (Section 6.7). Moreover, the proof of the UC Theorem made in the extended version [CS25, Appendix C] will extensively use them.

**Replacement Execution of an ITM.**

**Definition 4** ($(\phi, \ell, \pi)$-execution)**.** Let $\rho$, $\phi$, and $\pi$ be three ITMs, $\ell \in \mathbb{N} \cup \{\infty\}$, and let $\mathcal{A}, \mathcal{E}$ be two ITMs.

The $(\phi, \ell, \pi)$-execution of $\rho$ with environment $\mathcal{E}$ and adversary $\mathcal{A}$ is defined as the execution of the extended system $(\mathcal{E}, \mathcal{C}^{\rho,\mathcal{A},\mathcal{E}}_{(\phi,\ell,\pi)\text{-}\mathsf{EXEC}})$, where the control function $\mathcal{C}^{\rho,\mathcal{A},\mathcal{E}}_{(\phi,\ell,\pi)\text{-}\mathsf{EXEC}}$ is defined similarly to $\mathcal{C}^{\rho,\mathcal{A},\mathcal{E}}_{\mathsf{EXEC}}$, with some differences listed below:

1. *Tracking the sids of replaced extended sessions:* The execution maintains a set $\mathsf{ReplacedSession} \subseteq \{0,1\}^*$ (initialized to $\varnothing$) that tracks the sids of replaced sessions – as detailed in next items. Its size is bounded by $\ell$.
   In the following items, we will denote an ITI as "outside" if it is neither the environment $(\mathcal{E}, 0)$ nor the adversary $(\mathcal{A}, \Diamond)$, and if it does not belong to:
   - $\mathsf{Ext}(\phi, \mathsf{s})$ for any $\mathsf{s} \in \{0,1\}^*$,
   - $\mathsf{Ext}(\pi, \mathsf{s})$ for any $\mathsf{s} \in \mathsf{ReplacedSession}$.
2. *Replacement $\phi \to \pi$:* When an input or output is sent from an outside ITI to an ITI $(\phi, (\mathsf{s}, \mathsf{p}))$ for some $\mathsf{s}, \mathsf{p} \in \{0,1\}^*$:
   - If $\mathsf{s} \in \mathsf{ReplacedSession}$ or if $|\mathsf{ReplacedSession}| < \ell$, the message is instead sent to $(\pi, (\mathsf{s}, \mathsf{p}))$ and $\mathsf{s}$ is added to $\mathsf{ReplacedSession}$ (if not already present).
   - Otherwise, the message is not redirected and the execution continues.
3. *Replacement $\pi \to \phi$:* When an output is sent from an ITI $(\pi, (\mathsf{s}, \mathsf{p}))$ for some pid $\mathsf{p} \in \{0,1\}^*$ and sid $\mathsf{s} \in \mathsf{ReplacedSession}$, to an ITI $N \notin \mathsf{Ext}(\pi, \mathsf{s})$, the sender ITI $(\pi, (\mathsf{s}, \mathsf{p}))$ is replaced by $(\phi, (\mathsf{s}, \mathsf{p}))$.
4. *Adversary learns* $\mathsf{ReplacedSession}$*:* Whenever a message $m$ is sent to the adversary, the control function replaces it with the pair $(\mathsf{ReplacedSessions}, m)$. This ensures the adversary always knows the current value of $\mathsf{ReplacedSessions}$.

We denote by $(\phi, \ell, \pi)\text{-}\mathsf{EXEC}_{\rho,\mathcal{A},\mathcal{E}}(z)$ the random variable representing the result of executing $(\mathcal{E}, \mathcal{C}^{\rho,\mathcal{A},\mathcal{E}}_{(\phi,\ell,\pi)\text{-}\mathsf{EXEC}})$ on input $z$.

Note that replacement execution is a generalization of the standard notion of protocol execution. Indeed, for any protocol $\rho$, a protocol execution of $\rho$ corresponds to a $(\phi, 0, \pi)$-execution of $\rho$. The only difference is that, in the replacement execution, any message received by the adversary is a pair whose first element is the set $\varnothing$.

Note also that a replacement execution differs from applying the UC operator in two key respects:

- In a replacement execution, modifications are performed through control functions, which can directly substitute any external operation with another. In contrast, the UC operator must explicitly alter the involved protocols to simulate such changes.

- A replacement execution may perform partial replacements, whereas the UC operator applies a complete substitution.

Furthermore, the replacement execution should not be viewed as a substitute for the UC operator itself. Rather, it serves as a technical tool used to define certain notions within Updated UC. A closely related concept already appears in the proof of the UC Theorem in Plain UC [Can20, (Theorem 22)].

**Replacement Execution of a Parametrized System.**

Let $\ell \in \mathbb{N} \cup \{\infty\}$, and let $\boldsymbol{\rho}$, $\boldsymbol{\phi}$, and $\boldsymbol{\pi}$ be three parametrized systems, and $\mathcal{A}, \mathcal{E}$ two ITMs. The $(\boldsymbol{\phi}, \ell, \boldsymbol{\pi})$-execution of $\boldsymbol{\rho}$ with adversary $\mathcal{A}$ and environment $\mathcal{E}$ is defined analogously, with the following differences:

- All inputs to $\mathcal{E}$ must be pairs $(k, z)$ with $k \in \mathbb{N}$ and $z \in \{0,1\}^*$. The value $k$ is the security parameter.
- The adversary has read-only access to the security parameter.
- When the security parameter is $k$, the execution proceeds as a $(\phi_k, \ell, \pi_k)$-execution of $\rho_k$.

We denote the corresponding control function by $\mathcal{C}^{\boldsymbol{\rho},\mathcal{A},\mathcal{E}}_{(\boldsymbol{\phi},\ell,\boldsymbol{\pi})\text{-EXEC}}$ and the result of the execution by $(\boldsymbol{\phi}, \ell, \boldsymbol{\pi})\text{-}\mathsf{EXEC}_{\boldsymbol{\rho},\mathcal{A},\mathcal{E}}(k, z)$.

**PPT Triplet of Parametrized Systems.**

In Section 4, we defined the notion of a PPT protocol in the context of standard protocol execution. We now define an analogous notion for replacement executions.

A triplet $(\boldsymbol{\rho}, \boldsymbol{\phi}, \boldsymbol{\pi})$ of parametrized systems is said to be PPT if, for every PPT environment $\mathcal{E}$ and PPT adversary $\mathcal{A}$, there exists a polynomial $R$ such that for any $\ell \in \mathbb{N}$ and any input $(k, z) \in \mathbb{N} \times \{0,1\}^*$, any $(\boldsymbol{\phi}, \ell, \boldsymbol{\pi})$-execution of $\boldsymbol{\rho}$ with environment $\mathcal{E}$ and adversary $\mathcal{A}$ completes in time bounded by $R(k + |z|)$.

**Replacement Execution Prefixes.**

Just as we defined prefixes of protocol executions (Section 4, page 16), we now define prefixes for replacement executions.

A $(\rho, \phi, \ell, \pi)$-execution prefix is a prefix of an $(\phi, \ell, \pi)$-execution of $\rho$. We refer to any such prefix as a *replacement execution prefix*.

## 6.3 Panoptic Session

To define our variants of the subroutine respecting and subroutine exposing properties, we introduce a new construction whose use will be central in avoiding several issues identified in the first part of the paper—primarily Issue (I.2), Limitation (L.2), and Weakness (W.2).

**Definition 5** (Panoptic session)**.** Let $\alpha$ be an ITM, and let $\mathsf{s}$ be a sid. We define the panoptic session of $\alpha$ with sid $\mathsf{s}$, as

$$\mathsf{Pan}(\alpha, \mathsf{s}) = \bigcup_{\theta} \mathsf{Ext}(\alpha, \mathsf{s}, \theta),$$

where the union is taken over all $\alpha$-execution prefixes with test sid $\mathsf{s}$.

We define the set of panoptic sub-parties as

$$\mathsf{PanSub}(\alpha, \mathsf{s}) = \mathsf{Pan}(\alpha, \mathsf{s}) \setminus \mathsf{MainParty}(\alpha, \mathsf{s}) .$$

Additionally, for an ITM $\alpha$, we define

$$\mathsf{Pan}(\alpha) = \{\beta \text{ ITM} : \exists \mathsf{id}, \mathsf{s} \in \{0,1\}^*,\ (\beta, \mathsf{id}) \in \mathsf{Pan}(\alpha, \mathsf{s})\}.$$

We note that computing a panoptic session is no more difficult than computing the extended sessions already required by Plain UC (for example, to verify the subroutine respecting property), as it is simply a union over such sessions.

We also use this notion to define a restriction on the parameterized systems $\boldsymbol{\pi}$ and $\boldsymbol{\phi}$ considered in the UC Theorem. This restriction is particularly needed to formulate our update to the subroutine exposing notion and is employed in the proof of the UC Theorem in Updated UC. Although we do not know whether this condition can be relaxed in our setting, we argue that it is reasonable, as in practical settings $\boldsymbol{\phi}$ is typically an ideal protocol that does not share sub-parties with $\boldsymbol{\pi}$.

**Definition 6** (Disjunction)**.** Two protocols $\pi$ and $\phi$ are said to be disjoint if, for any sids $\mathsf{s}, \mathsf{s}' \in \{0,1\}^*$, we have $\mathsf{Pan}(\pi, \mathsf{s}) \cap \mathsf{Pan}(\phi, \mathsf{s}') = \varnothing$. Two parametrized systems $\boldsymbol{\phi}$ and $\boldsymbol{\pi}$ are said to be disjoint if, for all $k \in \mathbb{N}$, $\phi_k$ and $\pi_k$ are disjoint.

## 6.4 Sid-Exposing

To address the two issues discussed in Section 5.3 regarding the subroutine exposing property, we introduce a new concept called *sid-exposing*.

As a prerequisite for a protocol $\alpha$ to be sid-exposing, we require that all its panoptic sessions be separated, as formally defined below.[35]

**Definition 7** (Separation)**.** A protocol $\alpha$ is said to be *separated* if for any two distinct sids $\mathsf{s} \neq \mathsf{s}'$, $\mathsf{Pan}(\alpha, \mathsf{s}) \cap \mathsf{Pan}(\alpha, \mathsf{s}') = \varnothing$. A parametrized system $\boldsymbol{\alpha}$ is separated if each $\alpha_k$ is separated for all $k \in \mathbb{N}$.

Assuming that $\alpha$ is separated, we observe that any ITI $M$ belongs to at most one set $\mathsf{Pan}(\alpha, \mathsf{s})$ for some unique $\mathsf{s} \in \{0,1\}^*$, or to none of them.

The sid-exposing property asks that we can efficiently determine, given an ITI $M$, the corresponding $\mathsf{s}$, if it exists.

**Definition 8** (Sid-Exposing)**.** Let $\alpha$ be a separated protocol and $\mathsf{s}$ a sid. We say that $\alpha$ is *sid-exposing* if there exists a PPT Turing machine (TM) $\mathsf{Expose}^{\alpha}$, called the *exposing TM* of $\alpha$, which takes as input an ITI $M$ and returns the unique sid $\mathsf{s}$ such that $M \in \mathsf{Pan}(\alpha, \mathsf{s})$, if such a sid exists, and $\perp$ otherwise.

A separated parametrized system $\boldsymbol{\alpha}$ is sid-exposing if there exists a PPT Turing machine $\mathsf{PSExpose}^{\boldsymbol{\alpha}}$ whose inputs are pairs $(k, M) \in \mathbb{N} \times \mathsf{SetITI}$, and such that $\mathsf{PSExpose}^{\boldsymbol{\alpha}}(k, \cdot)$ is an exposing TM of $\alpha_k$ for each $k \in \mathbb{N}$.

We believe that this property is generally easy to verify, and we are not aware of any protocol in the literature that would fail to be sid-exposing.

This notion presents three main advantages over the subroutine exposing notion, effectively addressing the problems detailed in Section 5.3.

First, the exposing TM $\mathsf{Expose}^{\alpha}$ is not associated with a specific sid. In contrast, in the subroutine exposing property, there is a distinct exposing ITI for each sid. This resolves Weakness (W.2).

Second, the notion is *non-interactive*: the adversary can simulate $\mathsf{Expose}^{\alpha}$ locally without interacting with the protocol. This makes it more practical and avoids the need for the transformation discussed in (L.2).

Finally, this property reveals strictly less information to the adversary. Subroutine exposing allows the adversary to test whether an ITM $M$ exists in the *current execution*, whereas sid-exposing only reveals whether $M$ exists in *some execution* (possibly not the

[35] As with the disjunction property, we do not know whether this condition can be relaxed in our setting. We argue that it is reasonable, as we know no protocols in the literature under Plain UC that violate it. Generally, the subroutine of an extended session $(\alpha, \mathsf{s})$ have sid $\mathsf{s}$ too, which directly imply the separation property.

current one). A positive consequence is that the UC theorem in Updated UC can apply to concrete protocols such as $\pi^*$ and $\phi^*$ discussed in the technical details (Section 2.4).

## 6.5 Sub-party Respecting Protocol

As shown in Section 5.1, the subroutine respecting definition lacks a condition about backdoors (I.1), should not account only for protocol executions (I.3), and allows dynamic sub-parties (I.2).

In this section, we introduce a more robust concept called "sub-party respecting" to address these three issues. To achieve this, the sub-party respecting notion incorporates the following key changes compared to the original subroutine respecting definition:

- It prevents Issue (I.1) by adding a condition about the reception of backdoors by sleeping ITIs.
- To address Issue (I.2), we replace extended sessions with panoptic sessions and add a requirement regarding the reception of outputs by sleeping ITIs. Together, these measures prevent protocols with dynamic sub-parties to be sub-party respecting.
- To address Issue (I.3), the conditions are evaluated on any extended execution, not just any protocol execution.

**Definition 9** (Sub-party respecting)**.** A protocol $\alpha$ is said to be sub-party respecting if, for any prefix $\theta$ of any extended execution, and for each sid $\mathsf{s}$, the following conditions hold:

spr.1 All sleeping ITIs $N \in \mathsf{Pan}(\alpha, \mathsf{s})$ reject any backdoor.

spr.2 All sleeping ITIs $N \in \mathsf{PanSub}(\alpha, \mathsf{s})$ reject any output whose sender extended identity is an ITI $M \in \mathsf{MainParty}(\alpha, \mathsf{s})$.

spr.3 All ITIs of $\mathsf{PanSub}(\alpha, \mathsf{s})$ reject any input whose sender extended identity[36] is an ITI $M \notin \mathsf{Pan}(\alpha, \mathsf{s})$.

spr.4 All ITIs of $\mathsf{Pan}(\alpha, \mathsf{s})$ reject any output whose sender extended identity is an ITI $M \notin \mathsf{Pan}(\alpha, \mathsf{s})$.

spr.5 No ITI of $\mathsf{PanSub}(\alpha, \mathsf{s})$ sends an output to an ITI $M \notin \mathsf{Pan}(\alpha, \mathsf{s})$.

spr.6 No ITI of $\mathsf{Pan}(\alpha, \mathsf{s})$ sends an input to an ITI $M \notin \mathsf{Pan}(\alpha, \mathsf{s})$.

A parametrized system $\boldsymbol{\alpha}$ is sub-party respecting if $\alpha_k$ is sub-party respecting for each $k \in \mathbb{N}$.

Note that it may seem abstract to require these conditions for any prefix of any extended execution. A more concrete and informal interpretation is that they must hold for any possible internal state and any possible message (including those forged with impersonated sender identities).

## 6.6 Compliance

In Plain UC, compliance [Can20, (Section 6.1)] is the only property that constrains the protocol $\boldsymbol{\rho}$ when applying the UC theorem to protocols $\boldsymbol{\rho}$, $\boldsymbol{\pi}$, and $\boldsymbol{\phi}$[37]. The same principle applies in Updated UC, which employs a slight variant of this notion, as specified in Definition 10.

[36] Here, we specifically mean the sender's extended identity as indicated at the reception of the message. It is important to note that there is no guarantee the message was actually sent by an ITI with this extended identity, as impersonations are possible.

[37] Except time-execution constraints.

Our variant introduces two main changes. First, its conditions are not evaluated on any protocol execution, but rather on some replacement executions – this is motivated by Issue (I.3), Section 5.1. Second, to address Weakness (W.3) (also in Section 5.1), we forbid the presence of ITIs of Type 2, using the terminology introduced in that weakness.

Three additional changes are not directly related to the issues identified in Plain UC but are necessary to ensure the correctness of the UC Theorem in Updated UC. Determining whether the absence of these conditions causes problems within Plain UC is left for future work.

- We require that for each security parameter $k$, $\rho_k \notin \mathsf{Pan}(\phi_k) \cup \mathsf{Pan}(\pi_k)$. This assumption is reasonable, as $\boldsymbol{\rho}$ typically denotes a "larger protocol" that uses $\boldsymbol{\pi}$ or $\boldsymbol{\phi}$ as subroutines.
- Two additional conditions concerning certain output messages (Item 4 in the definition) is also required. We are not aware of any protocols studied in the literature that fail to satisfy it.

**Definition 10** (Compliance)**.** Let $\pi$, $\phi$, and $\rho$ be protocols such that $\pi$ and $\phi$ are separated and disjoint, and let $\xi$ be a predicate on extended identities. The protocol $\rho$ is said to be $(\pi, \phi, \xi)$-compliant if, for every $\ell \in \mathbb{N}$ and every $(\rho, \phi, \ell, \pi)$-execution prefix $\theta$, the following hold:

1. $\rho \notin \mathsf{Pan}(\phi) \cup \mathsf{Pan}(\pi)$.
2. There is no sid $\mathsf{s}$ such that an ITI $M \notin \mathsf{Ext}(\pi, \mathsf{s}, \theta)$ makes an external-write[38] targeting a recipient $N \in \mathsf{MainParty}(\pi, \mathsf{s})$.
3. The extended identities of all ITIs $M$ in $\theta$ that send inputs to ITIs with code $\phi$ satisfy the predicate $\xi$.[39]
4. There is no sid $\mathsf{s}$ such that an ITI $M \in \mathsf{MainParty}(\pi, \mathsf{s})$ intends to send an output to an ITI of $\mathsf{PanSub}(\phi, \mathsf{s})$, that lead to a replacement $\pi \to \phi$ by the control function.
5. There is no sids $\mathsf{s}, \mathsf{s}'$ such that an ITI $M \in \mathsf{MainParty}(\rho, \mathsf{s})$ sends an output to an ITI of $\mathsf{PanSub}(\phi, \mathsf{s}')$ or $\mathsf{PanSub}(\pi, \mathsf{s}')$.

If $\xi = \text{true}$, we simply say that $\rho$ is $(\pi, \phi)$-compliant.

Let $\boldsymbol{\pi}$, $\boldsymbol{\phi}$, and $\boldsymbol{\rho}$ be parametrized systems. $\boldsymbol{\rho}$ is said to be $(\boldsymbol{\pi}, \boldsymbol{\phi}, \xi)$-compliant if, for every $k \in \mathbb{N}$, the protocol $\rho_k$ is $(\pi_k, \phi_k, \xi)$-compliant.

## 6.7 Universal Composition Operator and Extension of UC-emulation

To address the concerns related to the universal composition operator (UC operator) in Plain UC (Section 5.2), we propose a general definition of the UC operator. This clarifies the concept and allows us to state the UC Theorem independently of any particular UC operator implementation, making it more general and modular. Since this definition relies on an extension of the UC-emulation relation, we first introduce this extension and then present the definition.

Additionally, we provide in the extended version [CS25, Appendix E] a new construction of UC operator that adheres to our definition and mitigates the issues identified with the UC operator of Plain UC. We will conclude this section by explaining the underlying idea behind its construction.

### 6.7.1 Extension of UC-emulation Relation.

Using the definitions of Section 6.1, we observe that for any parametrized systems $\boldsymbol{\pi}, \boldsymbol{\phi}, \boldsymbol{\rho}$, ITMs $\mathcal{E}, \mathcal{A}$, and $k \in \mathbb{N}, z \in \{0,1\}^*$, we can construct two binary probability distributions

[38] Here, we specifically refer to the external-write instruction issued by $M$, before any potential modification by the control function.

[39] Same as footnote 38.

ensembles:

$$(\mathsf{EXEC}_{\boldsymbol{\rho},\mathcal{A},\mathcal{E}}(k,z))_{k\in\mathbb{N},z\in\{0,1\}^*} \quad \text{and} \quad ((\boldsymbol{\phi},\infty,\boldsymbol{\pi})\text{-}\mathsf{EXEC}_{\boldsymbol{\rho},\mathcal{A},\mathcal{E}}(k,z))_{k\in\mathbb{N},z\in\{0,1\}^*}.$$

We want to use them to define a generalization of UC-emulation, adapted to $(\boldsymbol{\phi},\infty,\boldsymbol{\pi})$-executions. We want this extension to retain desirable properties of UC-emulation, notably transitivity, and the equivalence between UC-emulation and UC-emulation with respect to the dummy adversary [Can20, (Claim 11)].

To this end, we define $\mathcal{D}^{\mathsf{DROP}}$, an adaptation of the dummy adversary $\mathcal{D}$ for $(\phi,\infty,\pi)$-executions. When $\mathcal{D}^{\mathsf{DROP}}$ receives a message $(\mathsf{ReplacedSession},m)$, it behaves exactly like $\mathcal{D}$ on the message $m$. $\mathcal{D}^{\mathsf{DROP}}$ is said to be "dropping", as defined below:

**Definition 11** (Dropping Adversary)**.** An adversary $\mathcal{A}$ is said to be "dropping" if, upon receiving a message of the form $(\mathsf{ReplacedSession},m)$ — with $\mathsf{ReplacedSession}$ a set of sids and $m$ a message — it only uses $m$, thereby "dropping" $\mathsf{ReplacedSession}$.

In our extension of UC-emulation, we restrict attention to dropping adversaries in $(\boldsymbol{\phi},\infty,\boldsymbol{\pi})$-executions. Otherwise, a generalization of [Can20, (Claim 11)] would fail.

**Definition 12** (Extension of UC-emulation)**.** Let $\boldsymbol{\pi},\boldsymbol{\phi},\boldsymbol{\rho},\boldsymbol{\upsilon}$ be four PPT parametrized systems, and let $\xi$ be a predicate on extended identities. Suppose $(\boldsymbol{\rho},\boldsymbol{\phi},\boldsymbol{\pi})$ is a PPT triplet.

We say that $(\boldsymbol{\rho},\boldsymbol{\phi},\boldsymbol{\pi})$ UC-emulates $\boldsymbol{\upsilon}$ with respect to $\xi$-identity-bounded environments (or: $(\boldsymbol{\rho},\boldsymbol{\phi},\boldsymbol{\pi})$ $\xi$-UC-emulates $\boldsymbol{\upsilon}$) if for any dropping PPT adversary $\mathcal{A}$, there exists a PPT adversary $\mathcal{S}$ such that for any PPT $\xi$-identity-bounded environment $\mathcal{E}$, we have $(\boldsymbol{\phi},\infty,\boldsymbol{\pi})\text{-}\mathsf{EXEC}_{\boldsymbol{\rho},\mathcal{A},\mathcal{E}} \approx \mathsf{EXEC}_{\boldsymbol{\upsilon},\mathcal{S},\mathcal{E}}$.

We say that $\boldsymbol{\upsilon}$ UC-emulates $(\boldsymbol{\rho},\boldsymbol{\phi},\boldsymbol{\pi})$ with respect to $\xi$-identity-bounded environments (or: $\boldsymbol{\upsilon}$ $\xi$-UC-emulates $(\boldsymbol{\rho},\boldsymbol{\phi},\boldsymbol{\pi})$) if for any PPT adversary $\mathcal{A}$, there exists a dropping PPT adversary $\mathcal{S}$ such that for any PPT $\xi$-identity-bounded environment $\mathcal{E}$, we have $\mathsf{EXEC}_{\boldsymbol{\upsilon},\mathcal{A},\mathcal{E}} \approx (\boldsymbol{\phi},\infty,\boldsymbol{\pi})\text{-}\mathsf{EXEC}_{\boldsymbol{\rho},\mathcal{S},\mathcal{E}}$.

The UC-emulation is said to be perfect if the two distribution ensembles are perfectly indistinguishable. If the relation only holds for a specific adversary $\mathcal{A}$, we say that UC-emulation is with respect to $\mathcal{A}$.

UC-emulation for a fixed predicate $\xi$ is transitive. For example, if $\boldsymbol{\alpha}$ $\xi$-UC-emulates $(\boldsymbol{\rho},\boldsymbol{\phi},\boldsymbol{\pi})$ and $(\boldsymbol{\rho},\boldsymbol{\phi},\boldsymbol{\pi})$ $\xi$-UC-emulates $\boldsymbol{\upsilon}$, then $\boldsymbol{\alpha}$ $\xi$-UC-emulates $\boldsymbol{\upsilon}$.

We also generalize [Can20, (Claim 11)] to this setting:

**Proposition 2** (UC-emulation with respect to $\mathcal{D}^{\mathsf{DROP}}$ is equivalent to UC-emulation)**.** *Let $\boldsymbol{\pi},\boldsymbol{\phi},\boldsymbol{\rho},\boldsymbol{\upsilon}$ be four PPT parametrized systems and $\xi$ a predicate on extended identities. Suppose $(\boldsymbol{\rho},\boldsymbol{\phi},\boldsymbol{\pi})$ is a PPT triplet. Then, $(\boldsymbol{\rho},\boldsymbol{\phi},\boldsymbol{\pi})$ $\xi$-UC-emulates $\boldsymbol{\upsilon}$ if and only if $(\boldsymbol{\rho},\boldsymbol{\phi},\boldsymbol{\pi})$ $\xi$-UC-emulates $\boldsymbol{\upsilon}$ with respect to $\mathcal{D}^{\mathsf{DROP}}$.*

*Proof.* Direct adaptation of the proof of [Can20, (Claim 11)]. □

### 6.7.2 Definition of UC Operator.

We now define what we mean by a UC operator. Note that the conditions imposed on $\boldsymbol{\rho}$, $\boldsymbol{\phi}$, and $\boldsymbol{\pi}$ correspond to those required in our UC theorem.

**Definition 13** (UC Operator)**.** Let $\mathsf{PPT\text{-}Param}$ be the set of PPT parametrized systems. Let $S \subset \mathsf{PPT\text{-}Param}^3$ and $\mathsf{UC\text{-}OP} : S \to \mathsf{PPT\text{-}Param}$ a function. We say that $\mathsf{UC\text{-}OP}$ is a $S$-Universal Composition Operator (or $S$-UC Operator) if, for any $(\boldsymbol{\rho},\boldsymbol{\phi},\boldsymbol{\pi}) \in S$ such that:

- $\boldsymbol{\pi},\boldsymbol{\phi}$ are separated, sub-party respecting, sid-exposing, and disjoint,
- $\boldsymbol{\rho}$ is $(\boldsymbol{\pi},\boldsymbol{\phi})$-compliant,
- $(\boldsymbol{\rho},\boldsymbol{\phi},\boldsymbol{\pi})$ is a PPT triplet,

Then, $\mathsf{UC\text{-}OP}(\boldsymbol{\rho},\boldsymbol{\phi},\boldsymbol{\pi})$ perfectly UC-emulates $(\boldsymbol{\rho},\boldsymbol{\phi},\boldsymbol{\pi})$.

### 6.7.3 Description of our UC Operator Implementation.

As described in Issue (I.4) (page 24), the UC operator used in Plain UC suffers from flaws that lead to counterexamples to the UC theorem.[40] In the extended version [CS25, Appendix E], we describe a revised UC operator that meets our definition. Here, we summarize the core concept.

In every extended session $\mathsf{Ext}(\rho^{\phi\to\pi}, \mathsf{s}_{\mathsf{test}})$, we introduce an ITI $\mathsf{Coord}\,(\mathsf{s}_{\mathsf{test}})$, referred to as the coordinator. This ITI acts as an active middle-man within the session, observing nearly all messages and modifying them as needed to ensure correct replacements. In particular, it records enough information to avoid the problem described in Issue (I.4).

# 7 Universal Composition Theorem

This section states the UC theorem of Updated UC. It is both more general and more modular than the UC Theorem of Plain UC [Can20, (Theorem 22)], as it does not rely on a particular UC Operator implementation. Any choice of a UC operator (Definition 13) – including the one provided in the extended version [CS25, Appendix E] – immediately yields a concrete instantiation of the theorem, analogous to the UC Theorem in Plain UC.

A complete proof is provided in the extended version [CS25]. Its overall structure follows that of the original proof in Plain UC, building on the changes introduced in Updated UC to address the issues identified in this work.

**Theorem 1** (Universal Composition Theorem)**.** *Let $\boldsymbol{\pi}$, $\boldsymbol{\phi}$, and $\boldsymbol{\rho}$ be three PPT parametrized systems, and let $\xi$ be a predicate on extended identities. Assume that $(\boldsymbol{\rho}, \boldsymbol{\phi}, \boldsymbol{\pi})$ is a PPT triplet, and that $\boldsymbol{\phi}$ and $\boldsymbol{\pi}$ are sub-party respecting, separated, disjoint, and sid-exposing. Further assume that $\boldsymbol{\rho}$ is $(\boldsymbol{\pi}, \boldsymbol{\phi}, \xi)$-compliant and that $\boldsymbol{\pi}$ $\xi$-UC-emulates $\boldsymbol{\phi}$. Then, $(\boldsymbol{\rho}, \boldsymbol{\phi}, \boldsymbol{\pi})$ UC-emulates $\boldsymbol{\rho}$.*

*Sketch of proof.* The complete proof of the theorem is given in the extended version [CS25, Appendix C]. Its blueprint follows closely the original proof [Can20, (Theorem 22)], but incorporates the changes introduced in Updated UC to address the issues identified in this work, and is more detailed. In this proof sketch, we briefly explain the main points of the argument.

**Aim of the proof** Using UC-emulation with respect to the dropping dummy adversary $\mathcal{D}^{\mathsf{DROP}}$, we are looking for a PPT adversary $\mathcal{S}_\rho$ such that, for any PPT environment $\mathcal{E}$:

$$(\boldsymbol{\phi}, \infty, \boldsymbol{\pi})\text{-}\mathsf{EXEC}_{\boldsymbol{\rho},\mathcal{D}^{\mathsf{DROP}},\mathcal{E}} \approx \mathsf{EXEC}_{\boldsymbol{\rho},\mathcal{S}_\rho,\mathcal{E}}\ . \tag{1}$$

The fact that $\boldsymbol{\pi}$ $\xi$-UC-emulates $\boldsymbol{\phi}$ guarantees that there exists a PPT adversary $\mathcal{S}_\phi$ of $\boldsymbol{\phi}$, such that for any $\xi$-identity-bounded, PPT environment $\mathcal{E}$:

$$\mathsf{EXEC}_{\boldsymbol{\pi},\mathcal{D},\mathcal{E}} \approx \mathsf{EXEC}_{\boldsymbol{\phi},\mathcal{S}_\phi,\mathcal{E}}\ . \tag{2}$$

We construct a PPT adversary $\mathcal{S}_\rho$ from $\mathcal{S}_\phi$ and then show that it satisfies Equation 1.

**Description of $\mathcal{S}_\rho$** The complete code of the adversary $\mathcal{S}_\rho$ is provided in the extended version [CS25].

Before giving a summarized description of $\mathcal{S}_\rho$, we note that it simulates multiple variants of $\mathcal{S}_\phi$, each slightly modified. For each $\mathsf{s} \in \{0,1\}^*$, the adversary $\mathcal{S}_{\phi,\mathsf{s}}$ behaves identically to $\mathcal{S}_\phi$, with one exception: when $\mathcal{S}_\phi$ attempts to send a backdoor to an ITI $M \notin \mathsf{Pan}(\phi, \mathsf{s})$, $\mathcal{S}_{\phi,\mathsf{s}}$ deactivates instead of sending the message.

The behavior of $\mathcal{S}_\rho$, represented in Figure 12, can be summarized as follows:

[40] An additional issue is discussed in the extended version [CS25, Appendix D.1].

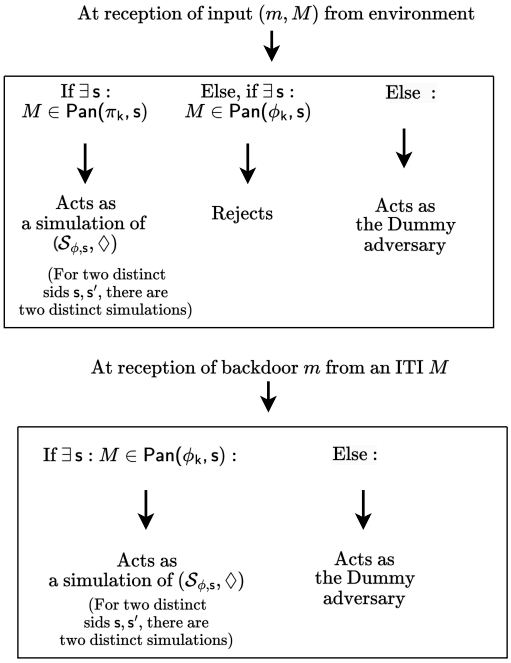



**Figure 12:** Simplified representation of the adversary $(\mathcal{S}_\rho, \Diamond)$ for security parameter $k$.

- Upon receiving an input $(m, M)$ from the environment, $\mathcal{S}_\rho$ uses the sid-exposing property of both $\boldsymbol{\pi}$ and $\boldsymbol{\phi}$ in order to compute $\mathsf{positionM}_\pi = \mathsf{PSExpose}^{\boldsymbol{\pi}}(k, M)$, and $\mathsf{positionM}_\phi = \mathsf{PSExpose}^{\boldsymbol{\phi}}(k, M)$. It then proceeds as follows:
  - If $\mathsf{positionM}_\pi \in \{0,1\}^*$, then $\mathcal{S}_\rho$ uses the simulation of $\mathcal{S}_{\phi,\mathsf{positionM}}$ to process the input $(m, M)$.
  - Else, if $\mathsf{positionM}_\phi \in \{0,1\}^*$, then $\mathcal{S}_\rho$ rejects.[41]
  - Otherwise, if $\mathsf{positionM}_\phi = \mathsf{positionM}_\pi = \perp$, then $\mathcal{S}_\rho$ acts as the dummy adversary and sends backdoor $m$ to $M$.
- Upon receiving a backdoor $m$ from some ITI $M$, $\mathcal{S}_\rho$ uses the sid-exposing property of $\boldsymbol{\phi}$ to compute $\mathsf{positionM}_\phi = \mathsf{PSExpose}^{\boldsymbol{\phi}}(k, M)$. It then proceeds as follows:
  - If $\mathsf{positionM}_\phi \in \{0,1\}^*$, then $\mathcal{S}_\rho$ uses the simulation of $\mathcal{S}_{\phi,\mathsf{positionM}}$ to process the backdoor $m$ received from $M$.
  - Otherwise, $\mathcal{S}_\rho$ acts as the dummy adversary and sends the backdoor $(m, M)$ to the environment.

Note that $\mathcal{S}_\rho$ is PPT because $\mathcal{S}_\phi$ is PPT, and both $\mathsf{Expose}^{\boldsymbol{\pi}}$ and $\mathsf{Expose}^{\boldsymbol{\phi}}$ are PPT Turing machines.

**Proof of Equation** (1) The proof proceeds by contraposition. We suppose that there exists a PPT environment $\mathcal{E}_\rho$ such that Equation (1) is false. By definition, this means: $\exists c, d \in \mathbb{N}$ such that $\forall k_0 \in \mathbb{N}, \exists k > k_0, \exists z \in \bigcup_{0<a\leq k^d} \{0,1\}^a$:

$$\left| \Pr\left[(\boldsymbol{\phi}, \infty, \boldsymbol{\pi})\text{-}\mathsf{EXEC}_{\boldsymbol{\rho}, \mathcal{D}^{\mathsf{DROP}}, \mathcal{E}_\rho}(k, z) = 1\right] - \Pr\left[\mathsf{EXEC}_{\boldsymbol{\rho}, \mathcal{S}_\rho, \mathcal{E}_\rho}(k, z) = 1\right] \right| \geq k^{-c}\,. \qquad (3)$$

[41] This case does not appear in the simulator used in the proof of the UC Theorem of Plain UC [Can20, (Theorem 22)]. Its omission is the source of a weakness discussed in the extended version [CS25, Appendix D.3].

We then use this environment $\mathcal{E}_\rho$ to construct a $\xi$-identity-bounded PPT environment $\mathcal{E}_\phi$ for which Equation (2) fails. More precisely, we show in the extended version [CS25, Appendix C, Lemma 1] that $\mathcal{E}_\phi$ satisfies the following central result:

> There is a polynomial $Q$ increasing on $[0,\infty[$, with values in $]0,\infty[$, such that, for any $(k,z) \in \mathbb{N} \times \bigcup_{0<a\leq k^d}\{0,1\}^a$:
>
> $$\Big|\Pr\big[\mathsf{EXEC}_{\boldsymbol{\pi},\mathcal{D},\mathcal{E}_\phi}(k,z)=1\big] - \Pr\big[\mathsf{EXEC}_{\boldsymbol{\phi},\mathcal{S}_\phi,\mathcal{E}_\phi}(k,z)=1\big]\Big|$$
> $$= \frac{1}{Q(k)}\left|\Pr\big[(\boldsymbol{\phi},\infty,\boldsymbol{\pi})\text{-}\mathsf{EXEC}_{\boldsymbol{\rho},\mathcal{D}^{\mathsf{DROP}},\mathcal{E}_\rho}(k,z)=1\big] - \Pr\big[\mathsf{EXEC}_{\rho,\mathcal{S}_\rho,\mathcal{E}_\rho}(k,z)=1\big]\right|.$$

Using this result together with Equation (3), we can conclude that Equation (2) is false. Indeed, they imply that $\forall k_0, \exists k > k_0, \exists z \in \bigcup_{0<a\leq k^d}\{0,1\}^a$ such that:

$$\Big|\Pr\big[\mathsf{EXEC}_{\boldsymbol{\pi},\mathcal{D},\mathcal{E}_\phi}(k,z)=1\big] - \Pr\big[\mathsf{EXEC}_{\boldsymbol{\phi},\mathcal{S}_\phi,\mathcal{E}_\phi}(k,z)=1\big]\Big| \geq \frac{k^{-c}}{Q(k)}\,.$$

Which completes the proof of the UC Theorem.

Note that establishing this central result requires showing that multiple executions have the same output distribution. More precisely, we perform an in-depth analysis in the extended version [CS25, Appendix C, Lemmas 2 and 3] to show that for $(k,z) \in \mathbb{N} \times \bigcup_{0<a\leq k^d}\{0,1\}^a$ and $\ell \in \{0,\cdots,Q(k)-2\}$:

$$\mathsf{EXEC}_{\boldsymbol{\pi},\mathcal{D},\mathcal{E}_\phi^\ell}(k,z) = \mathsf{EXEC}_{\boldsymbol{\phi},\mathcal{S}_\phi,\mathcal{E}_\phi^{\ell+1}}(k,z)\,,$$
$$\mathsf{EXEC}_{\boldsymbol{\pi},\mathcal{D},\mathcal{E}_\phi^{Q(k)-1}}(k,z) = (\boldsymbol{\phi},\infty,\boldsymbol{\pi})\text{-}\mathsf{EXEC}_{\boldsymbol{\rho},\mathcal{D}^{\mathsf{DROP}},\mathcal{E}_\rho}(k,z)\,,$$
$$\mathsf{EXEC}_{\boldsymbol{\phi},\mathcal{S}_\phi,\mathcal{E}_\phi^0}(k,z) = \mathsf{EXEC}_{\boldsymbol{\rho},\mathcal{S}_\rho,\mathcal{E}_\rho}(k,z)\,.$$

These equalities correspond to statements appearing in the proof of the UC Theorem in Plain UC, namely:

- Equation [Can20, (Equation (7), page 65)];
- The equivalences between inequalities [Can20, (Equation (4), page 64)] and [Can20, (Equation (6), page 65)], claimed on page 65 with the sentence "Consequently, inequality Equation (4) can be rewritten as".

These statements are asserted merely on the basis of an inspection of the involved ITMs. However, detailed demonstrations are not provided, and we believe that this omission substantially contributed to the counterexamples we identified. □

**Corollary 1** (Universal Composition Theorem with a UC Operator)**.** *Let $S \subset \mathsf{PPT\text{-}Param}^3$ and let $\mathsf{UC\text{-}OP}$ be an $S$-UC operator. Let $\boldsymbol{\pi}$, $\boldsymbol{\phi}$, and $\boldsymbol{\rho}$ be parametrized systems such that $(\boldsymbol{\rho},\boldsymbol{\phi},\boldsymbol{\pi}) \in S$, and let $\xi$ be a predicate on extended identities. Under the same assumptions as Theorem 1, $\mathsf{UC\text{-}OP}\,(\boldsymbol{\rho},\boldsymbol{\phi},\boldsymbol{\pi})$ UC-emulates $\boldsymbol{\rho}$.*

*Proof.* Immediate from transitivity of UC-emulation and Proposition 2. □

In typical use cases of Updated UC, the user aims to show that a protocol $\boldsymbol{\pi}$ UC-emulates another protocol $\boldsymbol{\phi}$, and wishes to preserve this security under composition. To that end, it suffices to verify that $\boldsymbol{\pi}$ and $\boldsymbol{\phi}$ are PPT, disjoint, separated, sid-exposing, and sub-party respecting. This ensures that the UC theorem applies to most natural choices of a PPT protocol $\boldsymbol{\rho}$, while the compliance and PPT-triplet requirements act as sanity conditions to exclude problematic edge cases.

## References


[BC15] Olivier Blazy and Céline Chevalier. Generic construction of UC-secure oblivious transfer. In Tal Malkin, Vladimir Kolesnikov, Allison Bishop Lewko, and Michalis Polychronakis, editors, *ACNS 2015: 13th International Conference on Applied Cryptography and Network Security*, volume 9092 of *Lecture Notes in Computer Science*, pages 65–86, New York, NY, USA, June 2–5, 2015. Springer, Cham, Switzerland. doi:10.1007/978-3-319-28166-7_4.

[BCH+20] Christian Badertscher, Ran Canetti, Julia Hesse, Björn Tackmann, and Vassilis Zikas. Universal composition with global subroutines: Capturing global setup within plain UC. In Rafael Pass and Krzysztof Pietrzak, editors, *TCC 2020: 18th Theory of Cryptography Conference, Part III*, volume 12552 of *Lecture Notes in Computer Science*, pages 1–30, Durham, NC, USA, November 16–19, 2020. Springer, Cham, Switzerland. doi:10.1007/978-3-030-64381-2_1.

[BDFH25] Ward Beullens, Lucas Dodgson, Sebastian H. Faller, and Julia Hesse. The 2Hash OPRF framework and efficient post-quantum instantiations. In Serge Fehr and Pierre-Alain Fouque, editors, *Advances in Cryptology – EUROCRYPT 2025, Part VIII*, volume 15608 of *Lecture Notes in Computer Science*, pages 332–362, Madrid, Spain, May 4–8, 2025. Springer, Cham, Switzerland. doi:10.1007/978-3-031-91101-9_12.

[BGHJ24] Manuel Barbosa, Kai Gellert, Julia Hesse, and Stanislaw Jarecki. Bare PAKE: Universally composable key exchange from just passwords. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology – CRYPTO 2024, Part II*, volume 14921 of *Lecture Notes in Computer Science*, pages 183–217, Santa Barbara, CA, USA, August 18–22, 2024. Springer, Cham, Switzerland. doi:10.1007/978-3-031-68379-4_6.

[BMTZ24] Christian Badertscher, Ueli Maurer, Daniel Tschudi, and Vassilis Zikas. Bitcoin as a transaction ledger: A composable treatment. *Journal of Cryptology*, 37(2):18, April 2024. doi:10.1007/s00145-024-09493-7.

[Can01] Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *42nd Annual Symposium on Foundations of Computer Science*, pages 136–145, Las Vegas, NV, USA, October 14–17, 2001. IEEE Computer Society Press. doi:10.1109/SFCS.2001.959888.

[Can20] Ran Canetti. Universally Composable Security. *J. ACM*, 67(5):28:1–28:94, 2020. doi:10.1145/3402457.

[CCL15] Ran Canetti, Asaf Cohen, and Yehuda Lindell. A simpler variant of universally composable security for standard multiparty computation. In Rosario Gennaro and Matthew J. B. Robshaw, editors, *Advances in Cryptology – CRYPTO 2015, Part II*, volume 9216 of *Lecture Notes in Computer Science*, pages 3–22, Santa Barbara, CA, USA, August 16–20, 2015. Springer Berlin Heidelberg, Germany. doi:10.1007/978-3-662-48000-7_1.

[CDG+25] Megan Chen, Pousali Dey, Chaya Ganesh, Pratyay Mukherjee, Pratik Sarkar, and Swagata Sasmal. Universally composable non-interactive zero-knowledge from sigma protocols via a new straight-line compiler, 2025. doi:10.1007/978-3-031-91820-9.

[CDPW07] Ran Canetti, Yevgeniy Dodis, Rafael Pass, and Shabsi Walfish. Universally composable security with global setup. In Salil P. Vadhan, editor, *TCC 2007:*

*4th Theory of Cryptography Conference*, volume 4392 of *Lecture Notes in Computer Science*, pages 61–85, Amsterdam, The Netherlands, February 21–24, 2007. Springer Berlin Heidelberg, Germany. doi:10.1007/978-3-540-70936-7_4.

[CDT19] Jan Camenisch, Manu Drijvers, and Björn Tackmann. Multi-protocol UC and its use for building modular and efficient protocols. Cryptology ePrint Archive, Report 2019/065, 2019. URL: https://eprint.iacr.org/2019/065.

[CJSV22] Ran Canetti, Palak Jain, Marika Swanberg, and Mayank Varia. Universally composable end-to-end secure messaging. In Yevgeniy Dodis and Thomas Shrimpton, editors, *Advances in Cryptology – CRYPTO 2022, Part II*, volume 13508 of *Lecture Notes in Computer Science*, pages 3–33, Santa Barbara, CA, USA, August 15–18, 2022. Springer, Cham, Switzerland. doi:10.1007/978-3-031-15979-4_1.

[CKKR19] Jan Camenisch, Stephan Krenn, Ralf Küsters, and Daniel Rausch. iUC: Flexible universal composability made simple. In Steven D. Galbraith and Shiho Moriai, editors, *Advances in Cryptology – ASIACRYPT 2019, Part III*, volume 11923 of *Lecture Notes in Computer Science*, pages 191–221, Kobe, Japan, December 8–12, 2019. Springer, Cham, Switzerland. doi:10.1007/978-3-030-34618-8_7.

[CNPR22] Cas Cremers, Moni Naor, Shahar Paz, and Eyal Ronen. CHIP and CRISP: Protecting all parties against compromise through identity-binding PAKEs. In Yevgeniy Dodis and Thomas Shrimpton, editors, *Advances in Cryptology – CRYPTO 2022, Part II*, volume 13508 of *Lecture Notes in Computer Science*, pages 668–698, Santa Barbara, CA, USA, August 15–18, 2022. Springer, Cham, Switzerland. doi:10.1007/978-3-031-15979-4_23.

[CR03] Ran Canetti and Tal Rabin. Universal composition with joint state. In Dan Boneh, editor, *Advances in Cryptology – CRYPTO 2003*, volume 2729 of *Lecture Notes in Computer Science*, pages 265–281, Santa Barbara, CA, USA, August 17–21, 2003. Springer Berlin Heidelberg, Germany. doi:10.1007/978-3-540-45146-4_16.

[CS25] Céline Chevalier and Éric Sageloli. Diving deep into UC: Uncovering and resolving issues in universal composability. Cryptology ePrint Archive, Report 2025/934, 2025. URL: https://eprint.iacr.org/2025/934.

[DFG+23] Gareth T. Davies, Sebastian H. Faller, Kai Gellert, Tobias Handirk, Julia Hesse, Máté Horváth, and Tibor Jager. Security analysis of the WhatsApp end-to-end encrypted backup protocol. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023, Part IV*, volume 14084 of *Lecture Notes in Computer Science*, pages 330–361, Santa Barbara, CA, USA, August 20–24, 2023. Springer, Cham, Switzerland. doi:10.1007/978-3-031-38551-3_11.

[Hes20] Julia Hesse. Separating symmetric and asymmetric password-authenticated key exchange. In Clemente Galdi and Vladimir Kolesnikov, editors, *SCN 20: 12th International Conference on Security in Communication Networks*, volume 12238 of *Lecture Notes in Computer Science*, pages 579–599, Amalfi, Italy, September 14–16, 2020. Springer, Cham, Switzerland. doi:10.1007/978-3-030-57990-6_29.

[HR25] Julia Hesse and Michael Rosenberg. PAKE combiners and efficient post-quantum instantiations. In Serge Fehr and Pierre-Alain Fouque, editors, *Advances in Cryptology – EUROCRYPT 2025, Part II*, volume 15602 of *Lecture Notes in Computer Science*, pages 395–420, Madrid, Spain, May 4–8, 2025. Springer, Cham, Switzerland. doi:10.1007/978-3-031-91124-8_14.

[HS15] Dennis Hofheinz and Victor Shoup. GNUC: A new universal composability framework. *Journal of Cryptology*, 28(3):423–508, July 2015. doi:10.1007/s00145-013-9160-y.

[HSS23] Julia Hesse, Nitin Singh, and Alessandro Sorniotti. How to bind anonymous credentials to humans. In Joseph A. Calandrino and Carmela Troncoso, editors, *USENIX Security 2023: 32nd USENIX Security Symposium*, pages 3047–3064, Anaheim, CA, USA, August 9–11, 2023. USENIX Association. URL: https://www.usenix.org/conference/usenixsecurity23/presentation/hesse.

[JKX18] Stanislaw Jarecki, Hugo Krawczyk, and Jiayu Xu. OPAQUE: An asymmetric PAKE protocol secure against pre-computation attacks. In Jesper Buus Nielsen and Vincent Rijmen, editors, *Advances in Cryptology – EUROCRYPT 2018, Part III*, volume 10822 of *Lecture Notes in Computer Science*, pages 456–486, Tel Aviv, Israel, April 29 – May 3, 2018. Springer, Cham, Switzerland. doi:10.1007/978-3-319-78372-7_15.

[KTR20] Ralf Küsters, Max Tuengerthal, and Daniel Rausch. The IITM model: A simple and expressive model for universal composability. *Journal of Cryptology*, 33(4):1461–1584, October 2020. doi:10.1007/s00145-020-09352-1.

[LL25] You Lyu and Shengli Liu. Hybrid password authentication key exchange in the UC framework. In Serge Fehr and Pierre-Alain Fouque, editors, *Advances in Cryptology – EUROCRYPT 2025, Part II*, volume 15602 of *Lecture Notes in Computer Science*, pages 421–450, Madrid, Spain, May 4–8, 2025. Springer, Cham, Switzerland. doi:10.1007/978-3-031-91124-8_15.

[Mau12] Ueli Maurer. Constructive cryptography – a new paradigm for security definitions and proofs. In Sebastian Mödersheim and Catuscia Palamidessi, editors, *Theory of Security and Applications*, pages 33–56, Berlin, Heidelberg, 2012. Springer Berlin Heidelberg. doi:10.1007/978-3-642-27375-9_3.

[RKC22] Daniel Rausch, Ralf Küsters, and Céline Chevalier. Embedding the UC model into the IITM model. In Orr Dunkelman and Stefan Dziembowski, editors, *Advances in Cryptology – EUROCRYPT 2022, Part II*, volume 13276 of *Lecture Notes in Computer Science*, pages 242–272, Trondheim, Norway, May 30 – June 3, 2022. Springer, Cham, Switzerland. doi:10.1007/978-3-031-07085-3_9.

# A Details About Issues, Limitations and Weaknesses Described in the Main Body

This section provides complete counterexamples and detailed proofs for Issue (I.2), Issue (I.3), and Issue (I.4) introduced in the main body.

Preliminary notations and conventions relevant to these constructions are given in Appendix A.1.

## A.1 Additional Notations and Conventions

**Additional Notations for ITIs.**

Let $\mathsf{SetITI}$ denote the set of all ITIs.

For an ITI $M$, we denote its code by $\mathsf{code}(M)$ and its identifier by $\mathsf{id}(M)$. If this identifier is a pair $(\mathsf{s},\mathsf{p}) \in \{0,1\}^* \times \{0,1\}^*$, we write $\mathsf{sid}(M)$ for its sid and $\mathsf{pid}(M)$ for its pid.

For any external-write $w$, $\mathsf{mess}(w)$ denotes its message and $\mathsf{tape}(w)$ its destination tape.

**Additional Conventions and Instructions for Algorithms.**

Local Functions. Each local function has access to the local variables of the main function and may modify them.

Sending Messages. The instructions **apply** and **send** both indicate that the ITI performs an external-write instruction $w$. More precisely, **apply** is used in the form "**apply** $w$", and **send** appears as "**send** '$m$' to tape $t$ of ITI $N$ with flags (...)". If the context is clear, flags may be omitted. When the destination tape $t$ is known, we often use **send input** , **send output** , or **send backdoor** .

Deactivation, Reject, and Halt. The instruction **deactivate** deactivates the ITI, **reject** causes it to reject, and **halt** places it in a halt state. Once halted, the ITI deactivates and rejects any subsequent messages.

Parsing. The instruction "**checkorchalt** $\mathsf{m}$ is (...)" checks whether a message $\mathsf{m}$ matches the specified pattern. If so, execution proceeds; otherwise, the ITI halts. This instruction is typically used to parse $m$ as a tuple and halt if parsing fails. The instruction **checkorreject** performs the same check but rejects instead of halting.

Ending Execution. In environment algorithms, for $b \in \{0,1\}$, the instruction "**return** $b$" causes $\mathcal{E}$ to write $b$ to the outgoing message tape and halt. This instruction thus terminates the execution with result $b$. The result of an execution with environment $\mathcal{E}$ is sometimes referred to as the "result of $(\mathcal{E},0)$".

Default Behavior. If an ITM's algorithm omits a response to certain events (*e.g.* it specifies a reaction to inputs but not to outputs or backdoors), the ITM is assumed to reject by default.

**Use of the New Extended Session Definition.** Due to the limitation highlighted in (L.1) (Section 5.1), we adopt our own extended session definition (Definition 3).

**No Use of Parametrized Systems.** The counterexamples we construct are not parametrized systems, but rather ITMs.

Although UC-emulation is formally defined only for parametrized systems, the notion naturally extends to ITMs. For any ITMs $\pi$ and $\phi$, we define $\pi_k = \pi$ and $\phi_k = \phi$ for all $k$, and apply the UC-emulation definition to the parametrized systems $\boldsymbol{\pi} = (\pi_k)_k$ and $\boldsymbol{\phi} = (\phi_k)_k$.

**Simplification of the Subroutine Exposing Property.** Certain protocols $\phi$ we will construct require the subroutine exposing property. In every such case, the exposing ITI for the extended session $\mathsf{Ext}(\phi,\mathsf{s})$ will be $(\phi,(\mathsf{s},0))$, and each ITI $(\phi,(\mathsf{s},\mathsf{p}))$ with $\mathsf{p} \neq 0$ will halt upon its first activation.

In our counterexamples, these latter ITIs are not monitored by the exposing ITI, as implementing such functionality would compromise readability and inflate the size of the algorithm.

Nevertheless, this feature could be added without affecting any relevant properties: an ITI $(\phi,(\mathsf{s},\mathsf{p}))$ may notify $(\phi,(\mathsf{s},0))$ of its invocation by sending it an output. This difference has no impact on any of the properties considered in our counterexamples.

**Use of Flags.** Unless explicitly stated otherwise, the following conventions apply:

- Messages exchanged between protocol ITIs (*i.e.* any ITI except the adversary and environment) have both flags forced-write and reveal-ext-id set.
- Backdoors sent by the adversary have both forced-write and reveal-ext-id unset.
- Inputs sent by the environment have both flags forced-write and reveal-ext-id set.

**Use of Autoreferences.** For clarity, we permit the code of an ITI to include its own code—despite this being formally undefined.

This abstraction simplifies the frequent situation involving two ITIs, say $(\psi, \mathsf{id})$ and $(\alpha, \mathsf{id}')$, such that:

- $(\psi, \mathsf{id})$ sends a message to $(\alpha, \mathsf{id}')$,
- and $(\alpha, \mathsf{id}')$ sends a message back to $(\psi, \mathsf{id})$.

In this scenario, the code of $\psi$ would embed that of $\alpha$, which in turn embeds the code of $\psi$, introducing a circular definition.

To circumvent this, we rely on the fact that an ITI has access to its own code via its identity tape, as defined in [Can20, (Section 3.1.1)]. We implement the following workaround in $\psi$:

- Replace every occurrence of $\psi$ in its own code with a reserved sequence seq.
- Modify $\psi$'s code so that, at runtime, the ITI replaces seq with the code stored on its identity tape.

**Structured Protocols' sid and pid Match Their Body.**

For structured protocols such as the UC operator $\rho^{\phi \to \pi}$, we require their sid and pid to to match those of their body.

This convention is not respected in Plain UC, as pointed out in Issue (I.5) (see extended version [CS25, Appendix D.1]), and leads to a counterexample of the UC Theorem.

**Compliance.** We omit the full definition of compliance from the main body of the article; it is reproduced here for completeness.

**Definition 14** (Compliance, ([Can20, (Section 6.1)])). A protocol $\rho$ is called $(\pi, \phi, \xi)$-compliant if:

- All inputs sent by any ITI of any extended session of $\rho$ are made with the flag forced-write set. All outputs sent to any ITI of any extended session of $\rho$ have the flag reveal-ext-id set.
- No two external-write instructions, out of all the external-write instructions made by the members of an extended session of $\rho$, where one instruction has target code $\pi$, and the other instruction has target code $\phi$, have the same target sid.
- The extended identities of all the ITIs in any extended session of $\rho$, that pass inputs to ITIs with code either $\pi$ or $\phi$, satisfy the predicate $\xi$.

When $\xi = \xi_{\mathsf{true}}$, the always-true predicate, we say that the protocol is $(\pi, \phi)$-compliant.

### A.1.1 (L.1): Detailed proof that the shell code [Can20, (Section 5.2)] does not turn any protocol into a subroutine respecting one

Consider two ITIs $M = (\pi, (s, p))$ and $N = (\nu, (s, p))$, where the only hypothesis we make is the following:

**(H)** When $M$ is invoked with an input $\mathsf{m}_1$, $M$ sends an input $\mathsf{m}_2$ to $N$.

We denote by $\sigma_\pi$ the shell construction defined in [Can20, (Section 5.2)], paragraph "Writing subroutine respecting protocols".

We will show that the structured protocol $(\pi, \sigma_\pi)$, with body $\pi$ and shell $\sigma_\pi$, is not subroutine respecting.

First, we introduce some notations for clarity: For any protocol $\mu$, we denote by $(\mu, \sigma_\pi)$ the structured protocol with body $\mu$ and shell $\sigma_\pi$. We adopt the convention that the sid of such a structured protocol is a pair $(s_{\mathsf{body}}, s_{\mathsf{shell}})$.

We also define:

- $M_{\sigma_\pi}$ as the structured ITI with shell $\sigma_\pi$, body $\pi$, shell sid $\top$, body sid $s$, and body pid $p$;[42]
- $N_{\sigma_\pi}$ as the structured ITI with shell $\sigma_\pi$, body $\nu$, shell sid $\top$, body sid $s$, and body pid $p$.

We now proceed to prove that $(\pi, \sigma_\pi)$ is not subroutine respecting. To this end, we describe an execution of a protocol $\rho$ with an environment $\mathcal{E}$ and some adversary, in which one of the subroutine-respecting conditions is violated.

For clarity and to keep the explanation concise, we define the actions of $\mathcal{E}$ and $\rho$ as the execution progresses.[43]

1. At first activation, $(\mathcal{E}, 0)$ sends an input to $(\rho, (s, p))$.
2. $(\rho, (s, p))$ then sends an input to $N_{\sigma_\pi}$.
3. This triggers the first activation of $N_{\sigma_\pi}$. By definition of the shell $\sigma_\pi$ [Can20, Section 5.2], Condition 1.(c) applies: $N_{\sigma_\pi}$ halts and remains inactive for all subsequent activations. In particular, *Because $N_{\sigma_\pi}$ has been invoked by an ITI that do not belong to an extended session of $(\pi, \sigma)$, $N_{\sigma_\pi}$ is outside of the extended session $((\pi, \sigma_\pi), (\mathsf{s}, \top))$.*
4. As $N_{\sigma_\pi}$ has halted, the environment $(\mathcal{E}, 0)$ is activated again. It sends another input to $(\rho, (s, p))$.
5. $(\rho, (s, p))$ is activated for the second time. We define its behavior so that it sends an input $\mathsf{m}_1$ to $M_{\sigma_\pi}$.
6. This is the first activation of $M_{\sigma_\pi}$. According to the shell specification, Condition 1.(a) applies, setting the main-party flag. Then, Condition 2 applies: since the flag main-party is set and the message is an input, the message $\mathsf{m}_1$ is directly transferred to the body $M$.
7. By hypothesis **(H)**, the body $M$ intends to send an input $\mathsf{m}_2$ to $N$. As a result, according to the last paragraph of Condition 3 of the shell specification, $M_{\sigma_\pi}$ produces an instruction to deliver input $\mathsf{m}_2$ to $N_{\sigma_\pi}$. *This contradicts the subroutine-property of $(\pi, \sigma_\pi)$, as $M_{\sigma_\pi}$, a main party of the extended session $((\pi, \sigma_\pi), (s, \top))$, sent an input to $N_{\sigma_\pi}$, an ITI that is outside the extended session $((\pi, \sigma_\pi), (s, \top))$.*[44]

## A.2 Details Regarding Issue (I.2)

In this section, we present a comprehensive counterexample for Issue (I.2).

Figure 13 defines the protocols $\phi$, $\pi$, and $\alpha$ – adaptations of those introduced in the discussion of Issue (I.2) (Section 5.1) – modified to satisfy the conditions required for the application of the UC theorem. These protocols rely on an additional protocol $\rho \notin \{\pi, \phi, \alpha\}$, whose definition is given in Figure 14.

**Proposition 3.** *$\phi$ and $\pi$ are subroutine respecting.*

*Proof.* It is clear that $\pi$ is subroutine respecting. We therefore focus on $\phi$, and fix an execution of an arbitrary protocol (not necessarily $\pi$) with any adversary and environment. For a sid $\mathsf{s} \neq 0$, the extended session $\mathsf{Ext}(\phi, \mathsf{s})$ contains no sub-party, so we focus on $(\phi, 0)$. The extended session $\mathsf{Ext}(\phi, 0)$ can have one sub-party $(\alpha, (0, 0))$, depending on which ITI sends first an input to $(\alpha, (0, 0))$. More precisely, there are two cases:

[42] We do not specify the value of the shell pid because we could not find its definition in the shell specification. We believe it can be set to $\top$, as is done for the shell sid.

[43] If needed, we would be happy to provide their code.

[44] More precisely, this contradicts the condition 4 of the subroutine respecting definition [Can20, (Definition 19)].

$(\phi, (\mathsf{s}, \mathsf{p}))$

**local variable:** boolean alphaIsSubparty, initialized to false
boolean FirstInputMade, initialized to false
1 : **if** $(\mathsf{s}, \mathsf{p}) \neq (0,0) \wedge \neg$IsBackdoor **then**
2 : **reject**
3 : At reception of an input $m$ from an ITI $M$ :
4 : **if** FirstInputMade is false **then**
5 : FirstInputMade $\leftarrow$ true
6 : **if** $M \neq (\alpha, (0,0))$ $\wedge$ first bit of randomness is 0 **then**
7 : alphaIsSubparty $\leftarrow$ true
8 : **send input** "0" to $(\alpha, (0,0))$
9 : **else**
10 : **send output** "0" to $(\rho, (0,0))$
11 : **send output** "0" to $(\rho, (0,0))$
12 : At reception of an output $m$ from an ITI M :
13 : **if** $M = (\alpha, (0,0))$ **then**
14 : **send output** "0" to $(\rho, (0,0))$
15 : **else reject**
16 : At reception of a backdoor :
// subroutine exposing interface:
17 : **if** $\mathsf{p} \neq 0$ **then reject**
18 : **checkorreject** message can be parsed as an ITI M
19 : **if** $M = (\phi, (\mathsf{s}, \mathsf{p}))$ **then**
20 : **send backdoor** "true" to adversary
21 : **if** $(\mathsf{s}, \mathsf{p}) = (0,0) \wedge M = (\alpha, (0,0))$ **then**
22 : **send backdoor** "alphaIsSubparty" to adversary
23 : **send backdoor** "false" to adversary

$(\pi, (\mathsf{s}, \mathsf{p}))$

1 : **if** $(\mathsf{s}, \mathsf{p}) \neq (0,0) \wedge \neg$IsBackdoor **then**
2 : **reject**
3 : At reception of an input $m$ from an ITI $M$ :
4 : **send output** "0" to $(\rho, (0,0))$
5 : **reject**
6 : At reception of output : **reject**
7 : At reception of a backdoor :
8 : // subroutine exposing interface:
9 : **if** $\mathsf{p} \neq 0$ **then reject**
10 : **checkorreject** message can be parsed as an ITI M
11 : **if** $M = (\phi, (\mathsf{s}, \mathsf{p}))$ **then**
12 : **send backdoor** "true" to adversary
13 : **send backdoor** "false" to adversary

$(\alpha, (\mathsf{s}, \mathsf{p}))$

1 : **local variable:** boolean FirstInputMade, initialized to false
2 : **if** $(\mathsf{s}, \mathsf{p}) \neq (0,0)$ **then reject**
3 : At reception of an input $m$ from an ITI $M$ :
4 : **if** FirstInputMade is false **then**
5 : FirstInputMade $\leftarrow$ true
6 : **if** $M = (\phi, (0,0))$ **then**
7 : **send output** "0" to $(\phi, (0,0))$
8 : **else**
9 : **send input** "0" to $(\phi, (0,0))$
10 : **else reject**
11 : At reception of output : **reject**
12 : At reception of backdoor : **reject**

**Figure 13:** Definition of $\phi$, $\pi$ and $\alpha$ for Issue (I.2).

- Case 1: if $(\alpha, (0,0))$ is activated by an input from $(\phi, (0,0))$, it is a sub-party of $(\phi, 0)$. We thus needs to ensure that:
  - $(\alpha, (0,0))$ does not send inputs or outputs to ITIs outside the extended session $\mathsf{Ext}(\phi, 0)$. This is indeed the case, as $(\alpha, (0,0))$ only sends an input to $(\phi, (0,0))$ then halts.
  - $(\alpha, (0,0))$ rejects any inputs or outputs from ITIs outside the extended session $\mathsf{Ext}(\phi, 0)$. This is indeed the case, as $(\alpha, (0,0))$ halts after its first activation.
- Case 2: if $(\alpha, (0,0))$ is activated by an input from an ITI that is not $(\phi, (0,0))$, we need to ensure that $(\alpha, (0,0))$ does not send outputs to main parties of the extended session $\mathsf{Ext}(\phi, 0)$, or input to the sub-parties of the extended session $\mathsf{Ext}(\phi, 0)$. This is the case, as it only sends an input to $(\phi, (0,0))$ and then halts.

□

**Proposition 4.** *$\phi$ and $\pi$ are subroutine exposing.*

*Proof.* It is direct by construction, the exposing ITI of session $(\pi, \mathsf{s})$ (*resp.* $(\phi, \mathsf{s})$) being $(\pi, (s, 0))$ (*resp.* $(\phi, (\mathsf{s}, 0))$). □

**Proposition 5.** *$\pi$ UC-emulates $\phi$.*

*Proof.* We consider an execution of $\pi$ with dummy adversary $\mathcal{D}$ and an environment $\mathcal{E}$. We note $\mathsf{s}_{\mathsf{test}}$ the test sid of the execution. The proof is made in two steps.
Step 1: We can assume that $\mathcal{D}$ neither receives nor sends any input or backdoor.

Note that the only interactions between the protocols $\phi$, $\pi$, and their adversaries occur through requests to the exposing ITI. We define an adversary $\mathcal{S}$ for $\phi$ by modifying $\mathcal{D}$ only in the way it handles messages related to the subroutine exposing property:

- The code of the exposing ITI $(\pi, (\mathsf{s}_{\mathsf{test}}, 0))$ is replaced by $\phi$, both for messages from the exposing ITI to the adversary and from the adversary to the exposing ITI.
- Requests about the presence of $(\pi, (\mathsf{s}_{\mathsf{test}}, 0))$ in the session are replaced by requests about the presence of $(\phi, (\mathsf{s}_{\mathsf{test}}, 0))$.
- Requests about the presence of an ITI $M$ with code $\alpha$ are not sent to the exposing ITI of $(\phi, (\mathsf{s}_{\mathsf{test}}, 0))$; instead, $\mathcal{S}$ simulates the response as if the request were made and the exposing ITI returned false.

It is immediate that $\mathcal{S}$ perfectly simulates the queries made by $\mathcal{D}$ to the exposing ITI $(\pi, (\mathsf{s}_{\mathsf{test}}, 0))$.

Furthermore, the environment $\mathcal{E}$ does not need to send any message to the dummy adversary in order to learn which ITIs belong to $\mathsf{Ext}(\phi, \mathsf{s}_{\mathsf{test}})$: the existence of these ITIs is entirely determined by the input messages sent by $\mathcal{E}$.

We have therefore established that:

- $\mathcal{E}$ cannot exploit any behavioral difference between $\mathcal{D}$ and $\mathcal{S}$ to distinguish the execution of $\pi$ from that of $\phi$.
- $\mathcal{E}$ does not need to interact with $\mathcal{D}$, as it already knows all the information that $\mathcal{D}$ could provide.

Consequently, we may assume without loss of generality that $\mathcal{D}$ never receives or sends any input or backdoor.
Step 2: $\mathcal{E}$ cannot distinguish the execution of $\pi$ or $\phi$ by sending inputs.

The only possible observable behavioral difference between $(\phi, (0,0))$ and $(\pi, (0,0))$ from the environment's perspective arises from the fact that $(\alpha, (0,0))$ may belong to the extended session $\mathsf{Ext}(\phi, 0)$, but never belongs to $\mathsf{Ext}(\pi, 0)$.

In particular, in the execution of $\phi$ with test sid 0, the behavior of $(\alpha, (0,0))$ upon receiving an input depends on whether it was previously invoked by $(\phi, (0,0))$.

$N$ is an ITI with code $\notin \{\phi, \pi, \alpha, \rho\}$

$(\widetilde{\mathcal{E}}, 0)$

```
1 : At first activation :
2 :    send input "0" to (ρ, (0,0))
3 : At activation with no message :
      // This condition occurs if:
      // - An ITI deactivates itself.
      // - The adversary sends a message
      //    to a non-existent ITI.
      // - The control function disallows
      //    an external write.
4 : return 0
5 : At reception of an output
    from an ITI M :
6 : return 1
```

$(\rho, \mathsf{id})$

```
1 : if id ≠ (0,0) then halt
2 : At reception of an input :
3 : if it is first activation then
4 :    send input "0" to (φ, (0,0))
5 : else halt
6 : At reception of an output
    from an ITI M :
7 : if it is second activation then
8 :    send input "0" to (α, (0,0))
9 : else if it is third activation then
10 :   send output "0" to N
11 : else halt
```

**Figure 14:** Definition of $\rho$ and $\widetilde{\mathcal{E}}$ for Issue (I.2).

However, for this behavioral difference to be visible to the environment, it would have to send an input to $(\alpha, (0, 0))$. This is impossible, since the environment is restricted to sending inputs only to main parties. □

**Proposition 6.** *The protocol $\rho$ defined in Figure 14 is $(\pi, \phi)$-compliant.*

*Proof.* All conditions can directly be verified. □

We have shown that $\phi$ and $\pi$ are subroutine respecting, subroutine exposing, that $\pi$ UC-emulates $\phi$ and that $\rho$ is $(\pi, \phi)$-compliant. The UC theorem of Plain UC should thus apply and ensure that $\rho^{\phi \to \pi}$ UC-emulates $\rho$.

Nevertheless, this conclusion is false, as shown by the next proposition.

**Proposition 7.** *$\rho^{\phi \to \pi}$ does not UC-emulate $\rho$.*

*Proof.* The environment $\widetilde{\mathcal{E}}$ defined in Figure 14 can distinguish them. More precisely, in the execution of $\rho$ with environment $\widetilde{\mathcal{E}}$, any adversary and any input, $\widetilde{\mathcal{E}}$ will return a uniformly random bit, whereas it will always return 1 in the execution of $\rho^{\phi \to \pi}$ with environment $\widetilde{\mathcal{E}}$ any adversary and any input.

Detail of messages sent in the execution of $\rho$ with environment $\widetilde{\mathcal{E}}$, any adversary and any input:

- $\widetilde{\mathcal{E}}$ sends input "0" to $(\rho, (0, 0))$.
- $(\rho, (0, 0))$ sends input "0" to $(\phi, (0, 0))$.
- If the first bit of randomness of $(\phi, (0, 0))$ is 0:
  1. $(\phi, (0, 0))$ sends input "0" to $(\alpha, (0, 0))$.
  2. $(\alpha, (0, 0))$ sends output "0" to $(\phi, (0, 0))$.
  3. $(\phi, (0, 0))$ sends output "0" to $(\rho, (0, 0))$.
  4. $(\rho, (0, 0))$ sends input "0" to $(\alpha, (0, 0))$.
  5. Because $\alpha$ halted, it deactivates.

6. $\widetilde{\mathcal{E}}$ is activated with no message and therefore returns 0.

- If the first bit of randomness of $(\phi, (0,0)$ is 1:
  1. $(\phi, (0,0))$ sends output "$\mathsf{0}$" to $(\rho, (0,0))$.
  2. $(\rho, (0,0))$ sends input "$\mathsf{0}$" to $(\alpha, (0,0))$.
  3. It is the first activation of $\alpha$ by an ITI that is not $(\phi, (0,0))$, it sends input "$\mathsf{0}$" to $(\phi, (0,0))$.
  4. $(\phi, (0,0))$ sends output "$\mathsf{0}$" to $(\rho, (0,0))$.
  5. $(\rho, (0,0))$ sends output "$\mathsf{0}$" to an ITI $N$ that does belong to the extended session $\mathsf{Ext}(\rho, 0)$. The output is thus sent to $\widetilde{\mathcal{E}}$, as well as extended identity $N$ and identity $(0,0)$.
  6. $\widetilde{\mathcal{E}}$ returns 1.

Detail of messages sent in the execution of $\rho^{\phi \to \pi} = (\rho, \sigma_{\pi,\phi})$ with environment $\widetilde{\mathcal{E}}$, any adversary and any input:

- $\widetilde{\mathcal{E}}$ sends input "$\mathsf{0}$" to $((\rho, \sigma_{\pi,\phi}), (0,0))$.
- $((\rho, \sigma_{\pi,\phi}), (0,0))$ sends input with message "$(\mathsf{0}, (\texttt{Shell:main:}\ \rho, 0\texttt{;}\ \texttt{subr:}\ \pi, 0))$" to the ITI $((\pi, \sigma_{\pi,\phi}), (0,0))$.
- $((\pi, \sigma_{\pi,\phi}), (0,0))$ sends output with message "$(\mathsf{0}, (\texttt{Shell:main:}\ \rho, 0\texttt{;}\ \texttt{subr:}\ \pi, 0))$" to the ITI $((\rho, \sigma_{\pi,\phi}), (0,0))$.
- $((\rho, \sigma_{\pi,\phi}), (0,0))$ sends input with message "$(\mathsf{0}, (\texttt{Shell:main:}\ \rho, 0\texttt{;}\ \texttt{subr:}\ \bot))$" to the ITI $((\alpha, \sigma_{\pi,\phi}), (0,0))$.
- $((\alpha, \sigma_{\pi,\phi}), (0,0))$ sends input with message "$(\mathsf{0}, (\texttt{Shell:main:}\ \rho, 0\texttt{;}\ \texttt{subr:}\ \pi, 0))$" to the ITI $((\pi, \sigma_{\pi,\phi}), (0,0))$.
- $((\pi, \sigma_{\pi,\phi}), (0,0))$ sends output with message "$(\mathsf{0}, (\texttt{Shell:main:}\ \rho, 0\texttt{;}\ \texttt{subr:}\ \pi, 0))$" to the ITI $((\rho, \sigma_{\pi,\phi}), (0,0))$.
- $((\rho, \sigma_{\pi,\phi}), (0,0))$ wants to send output with message "$\mathsf{0}$" to an ITI $N$ that does belong to the extended session $\mathsf{Ext}(\rho, \sigma_{\pi,\phi}, 0)$. The output is thus sent to $\widetilde{\mathcal{E}}$, as well as extended identity $N$ and identity $(0,0)$.
- $\widetilde{\mathcal{E}}$ returns 1.

□

## A.3 Details Regarding Issue (I.3)

In this section, we present a comprehensive counterexample to issue (I.3).

We consider the protocols $\phi$ and $\pi$ defined in Figure 15, the auxiliary protocols $\alpha$ and $\beta$ defined in Figure 16, and the protocol $\rho$ defined in Figure 17.

**Proposition 8.** *$\phi$ and $\pi$ are subroutine respecting.*

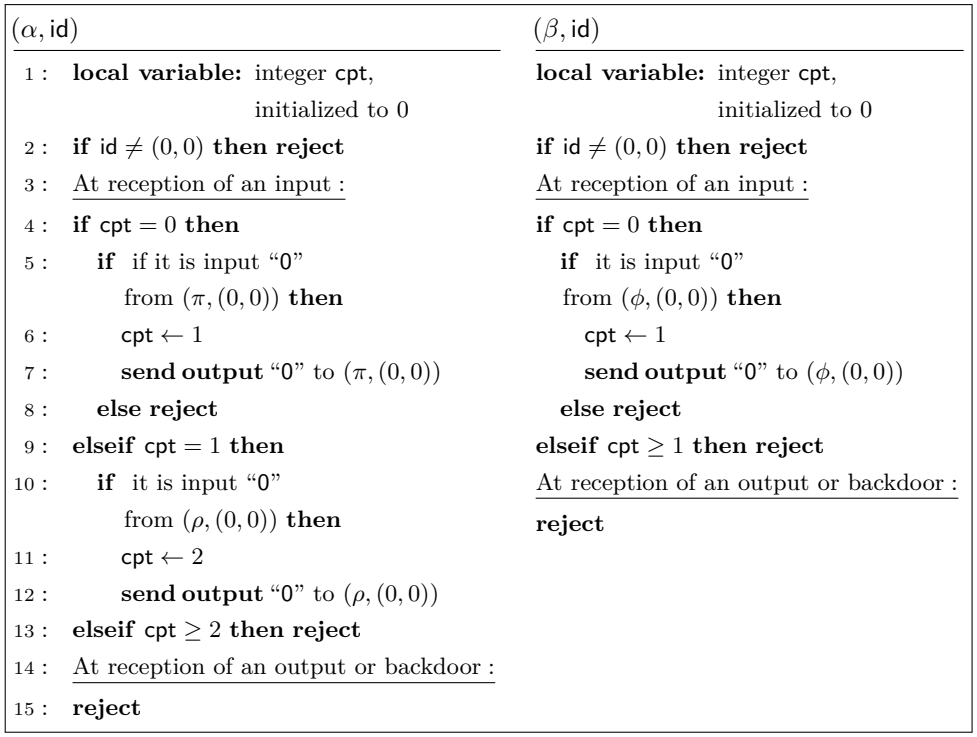
*Proof.* In any execution of an arbitrary protocol, the only extended session of $\phi$ that may contain a sub-party is $(\phi, 0)$, with sub-party $(\beta, (0,0))$. By definition, $(\beta, (0,0))$ rejects any message not originating from $(\phi, (0,0))$. It follows that $\phi$ is subroutine respecting.

We now focus on $\pi$. In any execution of an arbitrary protocol, the only session of $\pi$ that can contain a sub-party is $(\pi, 0)$, with sub-party $(\alpha, (0,0))$. All other main parties $(\pi, (0, \mathsf{s}))$, $\mathsf{s} \neq 0$, only accept backdoor and never send any input or output, so we can focus on the study of $(\pi, (0,0))$ and $(\alpha, (0,0))$.

Let $\theta$ be an execution prefix – of an arbitrary protocol with an environment $\mathcal{E}$ and some adversary – such that $(\alpha, (0,0))$ is a sub-party of $(\pi, 0)$. We will show that, within $\theta$, $(\alpha, (0,0))$ never receives an input from any ITI other than $(\pi, (0,0))$. This will complete the proof.

| $(\pi, (\mathsf{s}, \mathsf{p}))$ | | $(\phi, (\mathsf{s}, \mathsf{p}))$ | |
|---|---|---|---|
| | **local variable:** boolean alphaIsSubparty, initialized to false<br>integer cpt, initialized to 0 | **local variable:** boolean betaIsSubparty, initialized to false<br>integer cpt, initialized to 0 |
| 1 : | **if** $(\mathsf{s}, \mathsf{p}) \neq (0,0) \wedge \neg$IsBackdoor **then** | **if** $(\mathsf{s}, \mathsf{p}) \neq (0,0) \wedge \neg$IsBackdoor **then** |
| 2 : | **reject** | **reject** |
| 3 : | At reception of an input or output : | At reception of an input or output : |
| 4 : | **if** cpt $= 0$ **then** | **if** cpt $= 0$ **then** |
| 5 : | **if** it is input "0" from $(\rho, (0,0))$ **then** | **if** it is input "0" from $(\rho, (0,0))$ **then** |
| 6 : | alphaIsSubparty $\leftarrow$ true | betaIsSubparty $\leftarrow$ true |
| 7 : | cpt $\leftarrow 1$ | cpt $\leftarrow 1$ |
| 8 : | **send input** "0" to $(\alpha, (0,0))$ | **send input** "0" to $(\beta, (0,0))$ |
| 9 : | **else reject** | **else reject** |
| 10 : | **elseif** cpt $= 1$ **then** | **elseif** cpt $= 1$ **then** |
| 11 : | **if** it is output "0" from $(\alpha, (0,0))$ **then** | **if** it is output "0" from $(\beta, (0,0))$ **then** |
| 12 : | cpt $\leftarrow 2$ | cpt $\leftarrow 2$ |
| 13 : | **send output** "0" to $(\rho, (0,0))$ | **send output** "0" to $(\rho, (0,0))$ |
| 14 : | **else reject** | **else reject** |
| 15 : | **elseif** cpt $\geq 2$ **then** | **elseif** cpt $\geq 2$ **then** |
| 16 : | **reject** | **reject** |
| 17 : | At reception of a backdoor : | At reception of a backdoor : |
| | // subroutine exposing interface: | // subroutine exposing interface: |
| 18 : | **if** p $\neq 0$ **then reject** | **if** p $\neq 0$ **then reject** |
| 19 : | **checkorreject** message can be parsed as an ITI M | **checkorreject** message can be parsed as an ITI M |
| 20 : | **if** $M = (\pi, (\mathsf{s}, \mathsf{p}))$ **then** | **if** $M = (\phi, (\mathsf{s}, \mathsf{p}))$ **then** |
| 21 : | **send backdoor** "true" to adversary | **send backdoor** "true" to adversary |
| 22 : | **if** $(\mathsf{s}, \mathsf{p}) = (0,0) \wedge M = (\alpha, (0,0))$ **then** | **if** $(\mathsf{s}, \mathsf{p}) = (0,0) \wedge M = (\beta, (0,0))$ **then** |
| 23 : | **send backdoor** "alphaIsSubparty" to adversary | **send backdoor** "betaIsSubparty" to adversary |
| 24 : | **send backdoor** "false" to adversary | **send backdoor** "false" to adversary |

**Figure 15:** Definition of $\phi$, $\pi$ for Issue (I.3).



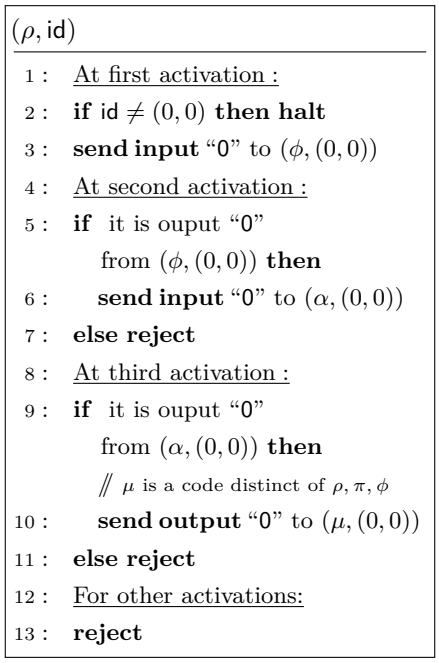
$(\alpha, \mathsf{id})$

**local variable:** integer cpt, initialized to 0
**if** id $\neq (0,0)$ **then reject**
At reception of an input :
**if** cpt $= 0$ **then**
  **if** if it is input "0" from $(\pi,(0,0))$ **then**
    cpt $\leftarrow 1$
    **send output** "0" to $(\pi,(0,0))$
  **else reject**
**elseif** cpt $= 1$ **then**
  **if** it is input "0" from $(\rho,(0,0))$ **then**
    cpt $\leftarrow 2$
    **send output** "0" to $(\rho,(0,0))$
**elseif** cpt $\geq 2$ **then reject**
At reception of an output or backdoor :
**reject**

$(\beta, \mathsf{id})$

**local variable:** integer cpt, initialized to 0
**if** id $\neq (0,0)$ **then reject**
At reception of an input :
**if** cpt $= 0$ **then**
  **if** it is input "0" from $(\phi,(0,0))$ **then**
    cpt $\leftarrow 1$
    **send output** "0" to $(\phi,(0,0))$
  **else reject**
**elseif** cpt $\geq 1$ **then reject**
At reception of an output or backdoor :
**reject**

**Figure 16:** Definition of $\alpha$, $\beta$ for Issue (I.3).



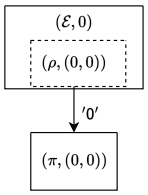
$(\rho, \mathsf{id})$

At first activation :
**if** id $\neq (0,0)$ **then halt**
**send input** "0" to $(\phi,(0,0))$
At second activation :
**if** it is ouput "0" from $(\phi,(0,0))$ **then**
  **send input** "0" to $(\alpha,(0,0))$
**else reject**
At third activation :
**if** it is ouput "0" from $(\alpha,(0,0))$ **then**
  // $\mu$ is a code distinct of $\rho, \pi, \phi$
  **send output** "0" to $(\mu,(0,0))$
**else reject**
For other activations:
**reject**

**Figure 17:** Definition of $\rho$ for Issue (I.3). All messages are sent with flags forced-write and reveal-ext-id set.

If $(\alpha,(0,0))$ is a sub-party of $(\pi,0)$, then the condition given in Line 5 of $\pi$'s definition must hold. In particular, this implies that $(\pi,(0,0))$ exists in $\theta$ and received an input "0" from some ITI with extended identity $(\rho,(0,0))$.

However, by construction, $\rho$ never sends inputs to $(\pi,(0,0))$. Therefore, this situation can only arise if $\theta$ is a prefix of an execution in which the environment $(\mathcal{E},0)$ impersonates $(\rho,(0,0))$ to send the input "0" to $(\pi,(0,0))$:




In this situation, $\theta$ is therefore a prefix of an execution of the protocol $\pi$ with test sid 0. By the rules governing protocol executions, $(\alpha,(0,0))$ can only receive inputs from $(\pi,(0,0))$, since the environment $(\mathcal{E},0)$ is not permitted to send inputs to the sub-party $(\alpha,(0,0))$. This completes the proof. □

**Proposition 9.** *$\phi$ and $\pi$ are subroutine exposing.*

*Proof.* It is direct by construction, the exposing ITI of session $(\pi,\mathsf{s})$ (*resp.* $(\phi,\mathsf{s})$) being $(\pi,(s,0))$ (*resp.* $(\phi,(\mathsf{s},0))$). □

**Proposition 10.** *$\pi$ UC-emulates $\phi$.*

*Proof.* We consider an execution of $\pi$ with dummy adversary $\mathcal{D}$ and an environment $\mathcal{E}$. We note $\mathsf{s}_{\mathsf{test}}$ the test sid of the execution.

The proof is made in two steps.

Step 1: We can assume that $\mathcal{D}$ neither receives nor sends any input or backdoor.

Observe that the only interactions between the protocols $\phi$, $\pi$ and their adversaries occur through requests to the exposing ITI. We construct an adversary $\mathcal{S}$ for $\phi$ as a variant of $\mathcal{D}$, with the only difference being that, whenever $\mathcal{D}$ sends or receives a message related to the subroutine exposing property, $\mathcal{S}$ modifies the message as follows:

- The code of the exposing ITI $(\pi,(\mathsf{s}_{\mathsf{test}},0))$ is replaced with $\phi$, both for messages from the exposing ITI to the adversary and from the adversary to the exposing ITI.
- Requests about the presence of $(\alpha,(0,0))$ in the session are replaced by requests about the presence of $(\beta,(0,0))$.

It is immediate that $\mathcal{S}$ perfectly simulates the requests made by $\mathcal{D}$ to the exposing ITI $(\pi,(\mathsf{s}_{\mathsf{test}},0))$, since the subroutine $(\alpha,(0,0))$ is invoked in the execution of $\phi$ under exactly the same conditions as $(\beta,(0,0))$ in the execution of $\pi$.

Moreover, the environment $\mathcal{E}$ does not need to interact with the dummy adversary to determine which ITIs belong to $\mathsf{Ext}(\phi,\mathsf{s}_{\mathsf{test}})$, as their existence depends solely on the input messages previously sent by $\mathcal{E}$.

We have therefore shown that:

- $\mathcal{E}$ cannot exploit any behavioral difference between $\mathcal{D}$ and $\mathcal{S}$ to distinguish the execution of $\pi$ from that of $\phi$.
- $\mathcal{E}$ does not need to communicate with $\mathcal{D}$, as it already knows all the information $\mathcal{D}$ could provide.

It follows that we may assume, without loss of generality, that $\mathcal{D}$ neither receives nor sends any input or backdoor.

Step 2: $(\mathcal{E},0)$ cannot distinguish the execution of $\pi$ or $\phi$ by sending inputs.

The only behavioral difference that $(\mathcal{E},0)$ could potentially exploit lies in the second activation of the sub-parties: $(\beta,(0,0))$ deactivates, whereas $(\alpha,(0,0))$ sends an output to

$(\rho, (0,0))$ if it receives the input "0" from $(\rho, (0,0))$. However, by construction, $(\rho, (0,0))$ never sends such a message. Moreover, the environment's impersonation ability cannot be leveraged in this context, as it is not allowed to send inputs to sub-parties. □

**Proposition 11.** *The protocol $\rho$ defined in Figure 17 is $(\pi, \phi)$-compliant.*

*Proof.* It is direct, as $\rho$ never writes to an ITI with code $\pi$ and that the flag forced-write is always set. □

We have shown that $\phi$ and $\pi$ are subroutine respecting, subroutine exposing, that $\pi$ UC-emulates $\phi$ and that $\rho$ is $(\pi, \phi)$-compliant. The UC theorem of Plain UC should thus apply and ensure that $\rho^{\phi \to \pi}$ UC-emulates $\rho$.

Nevertheless, this conclusion is false, as shown by the next proposition.

**Proposition 12.** *$\rho^{\phi \to \pi}$ does not UC-emulate $\rho$.*

*Proof.* Let us consider this environment:

$(\widetilde{\mathcal{E}}, 0)$

At first activation :

**send input** "0" to the protocol with identifier $(0,0)$

At second activation :

**if** it is activated with no message **then**

// This condition occurs if:

// - An ITI deactivates itself.

// - The adversary sends a message to a non-existent ITI.

// - The control function disallows an external write.

**return** 1

**else return** 0

We can see in the Figure 18 that for any adversary, $\widetilde{\mathcal{E}}$ returns 0 for the execution of $\rho^{\phi \to \pi}$ and 1 for the execution of $\rho$.

□

## A.4 Details Regarding Issue (I.4)

In this section, we present a comprehensive counterexample to issue (I.4).

We consider the protocol $\rho$ defined in Figure 19, which is constructed using an auxiliary protocol $\alpha$.

Additionally, we consider any PPT protocols $\phi$ and $\pi$ satisfying the following:

- $\phi \notin \{\pi, \rho, \alpha\}$,
- $\phi$ and $\pi$ are subroutine respecting and subroutine exposing,
- $\pi$ UC-emulates $\phi$.

It is clear that $\rho$ is $(\pi, \phi)$-compliant.

According to the UC theorem in Plain UC, this should imply that $\rho^{\phi \to \pi}$ UC-emulates $\rho$. However, we will show that this is not the case: $\rho^{\phi \to \pi}$ does not UC-emulate $\rho$.

**Proposition 13.** *$\rho^{\phi \to \pi}$ does not UC-emulate $\rho$.*

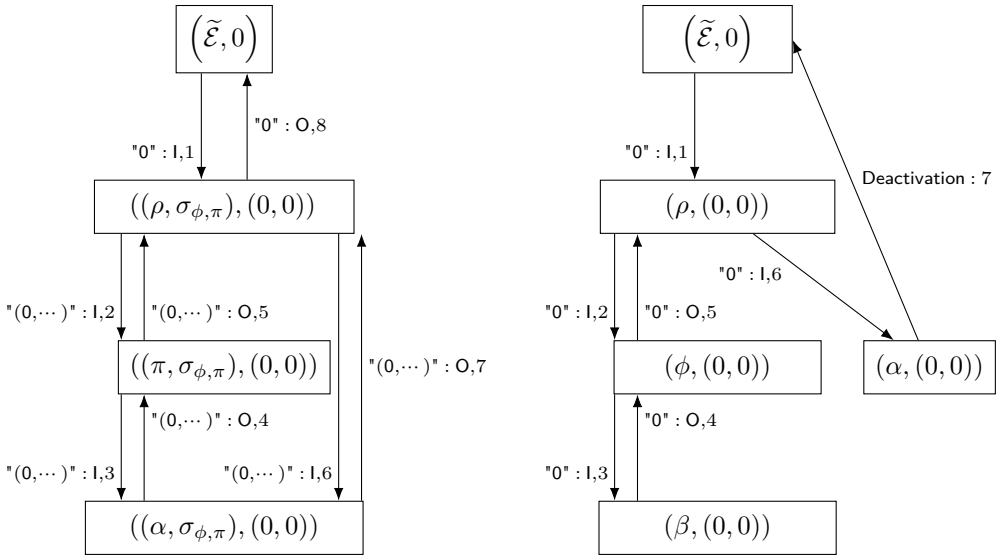



**Figure 18:** I/O differentiate input from output, and the numbers indicate execution order. The left diagram shows the message sequence in the execution of $\rho^{\phi \to \pi}$ with $\widetilde{\mathcal{E}}$, and the omitted right parts of messages indicated by "$\cdots$" are the shell messages $\mathsf{m}_{\mathsf{shell}}$ as discussed in the first paragraph of Section 5.2. The right diagram shows the sequence of messages in the execution of $\rho$ with $\widetilde{\mathcal{E}}$. The adversary is not used in both executions and is therefore not shown.

*Proof.* We recall that the UC operator of Plain UC is defined in [Can20, (Section 6.1)]. Considering the environment $\widetilde{\mathcal{E}}$ defined in Figure 20. We can see that, for any adversary $\mathcal{A}$:

- In the execution of $\rho^{\phi \to \pi} = (\rho, \sigma_{\pi,\phi})$ with $\widetilde{\mathcal{E}}$ and the dummy adversary $\mathcal{D}$, using the definition of the replacement operator [Can20, (Section 6.1)], we see:
  - At reception of input 0, $\big((\rho, \sigma_{\pi,\phi}), (0,0)\big)$ sends output "$(0, (\texttt{Shell:main:}\ \rho, 0\texttt{; subr:}\ \perp))$" to $((\alpha, \sigma_{\pi,\phi}), (0,0))$.
  - At reception of this input $((\alpha, \sigma_{\pi,\phi}), (0,0))$ deactivates.
  - $(\widetilde{\mathcal{E}}, 0)$ is then activated, and sends input 0 to $((\rho, \sigma_{\pi,\phi}), (0,1))$.
  - At reception of the input, $((\rho, \sigma_{\pi,\phi}), (0,1))$ initializes its variables main to $(\rho, 0)$ and subr to $\perp$. The body $(\rho, (0,1))$ is then activated with input 0 and intends to send output 0 to $(\alpha, (0,0))$: the condition 2.(c) of the definition of UC operator [Can20, (Section 6.1)] is then verified, which implies that $((\rho, \sigma_{\pi,\phi}), (0,1))$ sends output 0 to $(\alpha, (0,0))$. Because $(\alpha, (0,0))$ does not exist in the extended session $((\rho, \sigma_{\pi,\phi}), 0)$, this message is actually sent to $(\widetilde{\mathcal{E}}, 0)$, as well as the extended identity $(\alpha, (0,0))$ and the identity $(0,1)$.
  - $(\widetilde{\mathcal{E}}, 0)$ receives the output, and thus returns 1.
- In the execution of $\rho$ with $\widetilde{\mathcal{E}}$ and $\mathcal{A}$:
  - At reception of input 0, $(\rho, (0,0))$ sends input 0 to $(\alpha, (0,0))$.
  - At reception of this input, $(\alpha, (0,0))$ deactivates.
  - $\widetilde{\mathcal{E}}$ is then activated, and sends input 0 to $(\rho, (0,1))$.
  - At reception of the input, $(\rho, (0,1))$ sends output 0 to $(\alpha, (0,0))$.
  - At reception this output, $(\alpha, (0,0))$ deactivates.
  - As a result of this deactivation, $\widetilde{\mathcal{E}}$ is activated with no message and returns 0.

□

$(\rho, \mathsf{id})$

1 : At first activation:
2 : **if** $\mathsf{id} \notin \{(0,0),(0,1)\}$ **then reject**
3 : **if** $\mathsf{id} = (0,0)$ **then**
4 : **send input** "0" to $(\alpha, (0,0))$
5 : **if** $\mathsf{id} = (0,1)$ **then**
6 : **send output** "0" to $(\alpha, (0,0))$
7 : For other activations:
8 : **reject**

$(\alpha, \mathsf{id})$

1 : **reject**

**Figure 19:** Algorithm $\rho$ used for Issue (I.4).

$\left(\widetilde{\mathcal{E}}, 0\right)$

1 : At first activation:
2 : **send input** "0" to $(\rho, (0,0))$
3 : At second activation:
4 : **send input** "0" to $(\rho, (0,1))$
5 : At third activation:
6 : **if** it is activated with no message **then**
// This condition occurs if:
// - An ITI deactivates itself.
// - The adversary sends a message to a non-existent ITI.
// - The control function disallows an external write.
**return** 0
7 : **else return** 1

**Figure 20:** environment $\widetilde{\mathcal{E}}$ used for Issue (I.4).

## A.5 Details Regarding Weakness (W.1)

In thus subsection, we explain precisely the problem (W.1) and then describes how it can be mitigated.

**Detailed explanation of the problem.**
We denote by $\mathcal{S}_\pi$ the simulator corresponding to the dummy adversary in the UC-emulation of $\phi$ by $\pi$, and by $\mathcal{S}$ the simulator constructed in the proof of the UC Theorem [Can20, Figure 10] to ensure that $\rho^{\phi\to\pi}$ UC-emulates $\rho$.

We are concerned with the following indistinguishability from [Can20, Equation (2) of Section 6.2.1]:

$$\mathsf{EXEC}_{\rho^{\phi\to\pi},\mathcal{D},\mathcal{E}} \approx \mathsf{EXEC}_{\rho,\mathcal{S},\mathcal{E}}. \tag{4}$$

Our goal is to explain that, with the current definition of $\mathcal{S}$, this indistinguishability does not hold for all PPT environments $\mathcal{E}$.

For an ITI $M$, we define $\zeta(M)$ as the predicate that holds if and only if the code of $M$ corresponds to a structured protocol with the same shell as $\rho^{\phi\to\pi}$ (*i.e.*, the shell $\sigma_{\rho,\pi,\phi}$, following the precise definition of the UC Operator [Can20, (Section 6.1)]).

We make the following assumptions:

**(H1)** No execution of $\rho$ contains an ITI $M$ such that $\zeta(M)$ holds.

**(H2)** In the execution of any arbitrary protocol, no ITI $M$ satisfying $\zeta(M)$ belongs to an extended session of $\pi$ or $\phi$.

Both assumptions are quite natural and hold for a large class of protocols $\rho$, $\phi$, and $\pi$.

Let $m$ be any input, $M_\mathsf{T}$ any ITI such that $\zeta(M_\mathsf{T})$ holds and that is neither an adversary nor an environment (that is, its identifier is neither 0 nor $\Diamond$), and $M_\mathsf{F}$ any ITI such that $\zeta(M_\mathsf{F})$ does not hold and that is neither an adversary nor the environment.

We will temporarily admit the following two claims, which will be proven at the end of this subsection:

- Claim 1: In any execution of $\rho^{\phi\to\pi}$ with dummy adversary $\mathcal{D}$ and some environment $\mathcal{E}$, when $(\mathcal{E},0)$ sends the input $(m,M_\mathsf{F})$ to the adversary $(\mathcal{D},\Diamond)$, the next activation of $(\mathcal{E},0)$ is an activation with no message, resulting from a disallow by the control function.
- Claim 2: In any execution of $\rho$ with simulator $\mathcal{S}$ and some environment $\mathcal{E}$, when $(\mathcal{E},0)$ sends the input $(m,M_\mathsf{T})$ to the adversary $(\mathcal{S},\Diamond)$, the next activation of $(\mathcal{E},0)$ is an activation with no message, resulting from a disallow by the control function.

We now consider $\mathcal{E}$ a PPT environment such that Equation (4) is verified. Since, for every ITI $M$, either $\zeta(M)$ holds or it does not, we observe that when $(\mathcal{E},0)$ sends an input $(m,M)$ to the adversary, the environment $(\mathcal{E},0)$ is next activated with no message either in the left-hand execution or the right-hand execution of Equation (4). This follows from the fact that the conditions of either Claim 1 or Claim 2 are satisfied.

A direct consequence of that, because of the indistinguishability of (4), is that in both executions of (4), the following holds except with negligible probability:

**(C)** Whenever $(\mathcal{E},0)$ sends an input $(m,M)$ to the adversary, where $M$ is neither an adversary or an environment, the next activation of $(\mathcal{E},0)$ will be with no message.

Indeed, if this behavior occurs non-negligibly in one of the two executions but not in the other, it would be straightforward to construct an environment $\mathcal{E}'$, based on $\mathcal{E}$, that can non-negligibly distinguish between the two executions.

Nevertheless, **(C)** is clearly false: we can straightforwardly construct protocols $\rho, \pi, \phi$ and PPT environments $\mathcal{E}$ that does not satisfy it.[45] This invalidates the proof of UC theorem with the construction of simulator $\mathcal{S}$ given in [Can20, Figure 10].

We now proceed to prove Claim 1 and Claim 2.

*Proof. Proof of Claim 1*:
We note that in any execution of $\rho^{\phi \to \pi}$, by definition of $\rho^{\phi \to \pi}$ and of protocol execution, all ITIs $M$ distinct from the environment and adversary that exist in the execution satisfy $\zeta$. In particular, $M_{\mathsf{F}}$ does not exist in the execution.

As a result, when $\mathcal{E}$ sends input $(m, M_{\mathsf{F}})$ to the adversary $(\mathcal{D}, \Diamond)$:

1. $(\mathcal{D}, \Diamond)$ sends the backdoor message $m$ to $M_{\mathsf{F}}$.
2. As established at the beginning of this proof, $M_{\mathsf{F}}$ does not exist in the execution. The control function therefore disallows this backdoor, as it was sent with the `forced-write` flag unset.
3. As a consequence of the disallow, $(\mathcal{E}, 0)$ is activated with no message.

This completes the proof of Claim 1. □

*Proof. Proof of Claim 2*:
We note that by hypothesis **H1**, in any execution of $\rho$, all ITIs $M$ distinct from the environment and adversary that exist in the execution do not satisfy $\zeta$. In particular, $M_{\mathsf{T}}$ does not exist in the execution.

As a result, when $\mathcal{E}$ sends input $(m, M_{\mathsf{T}})$ to the adversary $(\mathcal{S}, \Diamond)$:

1. $(\mathcal{S}, \Diamond)$ is activated, and Item 1.(a) of [Can20, Figure 10] is applied: each existing copy of $\mathcal{S}_\pi$ is activated to check whether $M_{\mathsf{T}}$ belongs to its database.
2. By hypothesis **H2**, this is not the case, so no copy of $\mathcal{S}_\pi$ responds positively[46].Consequently, the actions described in Item 1.(b) of [Can20, Figure 10] are not executed.
3. Again by hypothesis **H2**, $M_{\mathsf{T}}$ is not a main party of a session of $\pi$. Thus, Item 1.(c).(i) of [Can20, Figure 10] does not apply. As a consequence, Item 1.(c).(ii) of [Can20, Figure 10] apply and $(\mathcal{S}, \Diamond)$ sends the backdoor message $m$ to $M_{\mathsf{T}}$.
4. As established at the beginning of this proof, $M_{\mathsf{T}}$ does not exist in the execution. The control function therefore disallows this backdoor, as it was sent with the `forced-write` flag unset.
5. As a consequence of the disallow, $(\mathcal{E}, 0)$ is activated with no message.

This completes the proof of Claim 2. □

**Mitigation of the problem.** The definition of the simulator $\mathcal{S}$ must be adjusted as follows. Upon receiving an input $(m, M)$, the simulator $\mathcal{S}$ should reject if $M$ does not satisfy $\zeta(M)$ (that is, if the code of $M$ does not correspond to a structured protocol sharing the same shell as $\rho^{\phi \to \pi}$). If $M$ satisfies $\zeta(M)$, it represents a structured protocol whose body is an ITI denoted $M_{\mathsf{body}}$. In this case, Items 1.(a), 1.(b), and 1.(c) of [Can20, Figure 10] must be applied to $M_{\mathsf{body}}$ rather than to $M$.

## A.6 Many examples of protocols studied in the literature that are not subroutine exposing

In this appendix, we substantiate the claim made on page 8 of Section 2 that most protocols used in the UC literature are not subroutine exposing, *i.e.*, they do not satisfy the subroutine-exposing property without additional modifications.[47]

[45] Said otherwise, our reasoning shows that any such choice of $\rho$, $\pi$, $\phi$, and $\mathcal{E}$ would constitute a concrete counter-example to Equation (4).

[46] A positive response could, in principle, occur with negligible probability if the UC simulation of $\phi$ by $\pi$ is not perfect. We omit this detail here, as it is not relevant to the argument.

[47] We do not claim that such modifications are impossible; for instance, a generic transformation achieving this is described in [Can20, Section 6.1].

To this end, we show that the ideal protocols associated with the following ideal functionalities are not subroutine exposing.[48]

- $\mathcal{F}_{\mathsf{AUTH}}$ [Can20, Figure 12], $\mathcal{F}^{l}_{\mathsf{SMT}}$ [Can20, Figure 13], $\mathcal{F}_{\mathsf{SYN}}$ [Can20, Figure 14] $\mathcal{F}_{\mathsf{NC}}$ [Can20, Figure 15].
- $\mathcal{F}_{\mathsf{PAKE}}, \mathcal{F}_{\mathsf{lePAKE}}, \mathcal{F}_{\mathsf{rlePAKE}}$ [HR25, Figure 1] (Eurocrypt 2024).
- $\mathcal{F}_{\mathsf{bPAKE}}$ and $\mathcal{F}_{\mathsf{bPAKE}^{\mathsf{LE}}}$ [LL25, Figure 5] (Eurocrypt 2025).
- $\mathcal{F}^{f}_{\mathsf{SFE}}$, $\mathcal{F}_{\mathsf{OPRF}}$ and $\mathcal{F}^{f}_{\mathsf{2H-OPRF}}$ [BDFH25, Figures 1 and 2] (Eurocrypt 2025).
- $\mathcal{F}_{\mathsf{cbAC}}$ and $\mathcal{F}^{\mathsf{blind}}_{\mathsf{cbAC}}$ [HSS23, Figure 4] (USENIX Security 2023).

We begin by recalling the complete definition of a subroutine exposing protocol as introduced in [Can20, Definition 21]:

"Protocol $\pi$ is subroutine exposing if each session s of $\pi$, occurring within an execution of any protocol with any environment, provides an interface to the adversary where the adversary can specify an extended identity $\alpha$ and is notified in response whether $\alpha$ is a member of the extended session $(\pi, s)$. The interface should take the form of a query on the backdoor tape of an ITI whose extended identity should be determined given $(\pi, s)$ alone."

As stated, this definition requires the existence of an ITI $U$—called an *exposing ITI*—that can receive queries on its backdoor tape. Each query contains an extended identity $\alpha$, and upon receiving such a query, $U$ must send a backdoor message to the adversary indicating whether $\alpha$ belongs to the extended session $(\pi, s)$.

We now explain why, for each ideal functionality $\mathcal{F}$ listed earlier in the short answer, the corresponding ideal protocol $\mathsf{IDEAL}_{\mathcal{F}}$ is not subroutine exposing. Specifically, we show that no ITI appearing in an execution of $\mathsf{IDEAL}_{\mathcal{F}}$ satisfies the role of an exposing ITI.

By definition of the ideal protocol $\mathsf{IDEAL}_{\mathcal{F}}$ [Can20, Section 5.3], the ITIs that may appear in an extended session $(\mathsf{IDEAL}_{\mathcal{F}}, s)$ are of two types:

- The main parties of $(\mathsf{IDEAL}_{\mathcal{F}}, s)$, which act as simple relays between the environment and the ideal functionality $\mathcal{F}$. More precisely, they relay messages between the environment and the ITI $(\mathcal{F}, (s, \perp))$. They cannot serve as exposing ITIs, as [Can20, Section 5.3, Item (3)] explicitly specifies that they reject any backdoor message.
- The ideal functionality ITI $(\mathcal{F}, (s, \perp))$ itself and any subroutines it may invoke. In all the ideal functionalities we listed earlier, $(\mathcal{F}, (s, \perp))$ does not invoke subroutines. Consequently, it suffices to check whether $(\mathcal{F}, (s, \perp))$ could serve as an exposing ITI. A direct inspection of the corresponding code for each functionality shows that $(\mathcal{F}, (s, \perp))$ is not an exposing ITI: $(\mathcal{F}, (s, \perp))$ never implements an interface allowing the adversary to query, for example, whether a ITI of the form $(\mathsf{IDEAL}_{\mathcal{F}}, (s, p))$ exists in the extended session $(\mathsf{IDEAL}_{\mathcal{F}}, s)$.

[48] We focus on ideal protocols because they play a central role in the UC framework. The selected functionalities come from [Can20] as well as from several works accepted at major conferences. The cited papers do not necessarily represent the earliest appearances of these functionalities.